

Local Reasoning Models for Mathematics

–From installation, through concepts, to workflows with Ollama–
1st release

Paweł Przybyłowicz
AGH University of Krakow
Faculty of Applied Mathematics

June 2026

Contents

Preface	1
Section 0: How to Read These Notes	2
0.1 What the Notes Are About	2
0.2 What the Notes Are Not About	3
0.3 Prerequisites	3
0.4 Formatting Conventions	4
0.5 Sources and Bibliography	4
1 Why Local LLMs for Mathematics?	5
1.1 Local LLMs as mathematical instruments	5
1.2 Why run models locally?	6
1.3 Mathematical exploration vs mathematical proof	8
1.4 Privacy, reproducibility, and independence from paid APIs . .	9
1.5 What local LLMs cannot guarantee	10
2 Installing Ollama and Running the First Model	12
2.1 What is Ollama?	12
2.2 Installation on Windows, macOS, and Linux	13
2.3 Basic commands: pull, run, list, show, remove	15
2.4 Running a first local model	16
2.5 Running GGUF models from Hugging Face	16
2.6 Checking available disk, RAM, and GPU memory	17
2.7 A first mathematical prompt	18
2.8 Common installation problems	20
3 Basic Vocabulary of Local Reasoning Models	22
3.1 Parameters and model size	22
3.2 Dense models and mixture-of-experts models	23
3.3 Active parameters versus total parameters	24
3.4 Quantization	25
3.5 Quantization labels: Q4, Q5, Q6, Q8, FP16	25
3.6 Context window and tokens	26
3.7 KV cache	27

3.8	RAM, VRAM, CPU, GPU, and offloading	28
3.9	GGUF and local inference formats	28
3.10	Temperature, seed, determinism, and reproducibility	29
3.11	Throughput, latency, and practical speed	31
3.12	Why bigger is not always better	31
4	A Field Guide to Free Local Reasoning Models	33
4.1	What does <i>free local model</i> mean?	33
4.2	Open-weight, open-source, permissive license, and local availability	34
4.3	Why model families matter more than single model rankings .	35
4.4	Qwen and Qwen reasoning models	36
4.5	QwQ	37
4.6	DeepSeek-R1 and distilled reasoning models	37
4.7	Phi-4 Reasoning	39
4.8	Nemotron and OpenReasoning Nemotron	40
4.9	gpt-oss	41
4.10	OLMo and open research models	42
4.11	Gemma, Mistral, GLM	42
4.12	Code-oriented local models	43
4.13	Small, medium, and large local models	44
4.14	Reproducibility checklist	45
4.15	Model roles in a mathematical workflow	45
5	Choosing Models for Mathematical Work	47
5.1	Task-based model selection	47
5.2	Explanation, proof search, proof audit, coding, and editing . .	49
5.3	Small fast models versus larger reasoning models	51
5.4	Quantization versus mathematical reliability	51
5.5	When to use several models	52
5.6	When not to use an LLM	54
5.7	Recommended minimal local setup	54
5.8	A model comparison worksheet	55
6	Prompting Principles for Mathematics	57
6.1	Anatomy of a mathematical prompt	57
6.2	Stating assumptions precisely	58
6.3	Stating definitions explicitly	59
6.4	Asking for proof or counterexample	60
6.5	Asking for strategy before proof	62
6.6	Asking for proof audit	62
6.7	Asking for repair of a flawed proof	63
6.8	Asking for LaTeX exposition	64
6.9	Asking for computational experiments	65

6.10	Asking for uncertainty and weak points	66
6.11	Bad prompts and improved prompts	67
7	Mathematical Workflows with Several Models	70
7.1	The single-model workflow	70
7.2	Why single-model workflows are risky	71
7.3	The two-model adversarial workflow	72
7.4	The cascade workflow for hard problems	73
7.5	Separating roles: explorer, skeptic, formalizer, programmer, editor	77
7.6	Human-in-the-loop verification	78
7.7	Optional formal verification with Lean	79
7.8	From model output to publication-quality proof	80
7.9	Documentation of prompts and model versions	80
8	Worked Mathematical Examples	82
8.1	Example 1: Every convergent real sequence is bounded	82
8.2	Example 2: A false claim about critical points	84
8.3	Example 3: Convexity and global minimizers	87
8.4	Example 4: The Polyak–Łojasiewicz inequality and minimizers	89
8.5	Example 5: A function of the form $f(x) = x^2 + a \sin^2(x)$. . .	93
8.6	Example 6: Numerical evidence versus proof	97
8.7	Example 7: Auditing a flawed model-generated proof	99
8.8	Example 8: From informal solution to L ^A T _E X proof	101
8.9	Example 9: Optional Lean formalization of a simple theorem	103
9	Limitations, Hallucinations, and Verification	106
9.1	Common mathematical failure modes	106
9.2	Definition drift	107
9.3	Hidden compactness assumptions	108
9.4	Hidden convexity assumptions	109
9.5	Direction errors in inequalities	110
9.6	Smoothness inflation	111
9.7	False theorem invocation	111
9.8	Local-global confusion	112
9.9	Numerical evidence mistaken for proof	113
9.10	Verification checklist	114
9.11	The human mathematician as final arbiter	116
10	Student Projects and Assignments	118
	General rules for all projects	118
10.1	Project 1: Compare three local models on elementary analysis	119
10.2	Project 2: Compare Q4 and Q8 quantization	121
10.3	Project 3: Audit a flawed proof	122

10.4	Project 4: Local LLM plus Python	124
10.5	Project 5: Local LLM plus L ^A T _E X	126
10.6	Project 6: Optional local LLM plus Lean	127
10.7	Final report template	128
	Transition to the tool-support chapters	130
11	Numerical Evidence, Python, and Sage	131
11.1	Numerical evidence as conjecture generation	131
11.2	Using Python for plots and sanity checks	132
11.3	Using Sage for exact or symbolic support	133
11.4	Worked example: plotting $x^2 + a \sin^2 x$	134
11.5	Reproducible computation logs	134
11.6	Failure modes in generated code	135
12	LaTeX Writing and Manuscript Support	136
12.1	What local models can help with in LaTeX	136
12.2	Definitions used verbatim	137
12.3	Theorem, proof, example, and remark environments	137
12.4	From audit notes to revised proof	138
12.5	LaTeX output as a handoff artifact	138
12.6	Common LaTeX failure modes	139
13	Lean and Formal Verification	140
13.1	Why formal verification is different from model agreement	140
13.2	What Lean can and cannot do	141
13.3	Installing or accessing Lean	141
13.4	Using mathlib	142
13.5	Example: formalizing a simple theorem	142
13.6	LLMs as Lean assistants	143
13.7	When not to formalize	143
14	Conclusion	144
14.1	What the human user should retain	144
14.2	A recommended first independent project	144
A	Ollama Command Reference	146
A.1	Conventions	146
A.2	Version and help commands	146
A.3	Core model-management commands	147
A.4	Running a first mathematical prompt	147
A.5	PowerShell examples on Windows	148
A.6	Hugging Face GGUF pattern	148
A.7	Context length and loaded models	149
A.8	Minimal reproducibility log	149

B	Example Modelfiles	150
B.1	General cautions	150
B.2	Minimal mathematical assistant Modelfile	150
B.3	Skeptical proof-auditor Modelfile	151
B.4	Counterexample-search Modelfile	152
B.5	Long-context reading Modelfile	152
B.6	Modelfile from a local GGUF file	153
B.7	Modelfile audit checklist	153
C	Prompt Templates	154
C.1	Problem card template	154
C.2	Proof or counterexample template	154
C.3	Proof-generation template	155
C.4	Strategy-before-proof template	155
C.5	Counterexample-search template	156
C.6	Proof-audit template	156
C.7	Proof-repair template	156
C.8	LaTeX rewriting template	157
C.9	Computational-experiment template	157
C.10	Uncertainty-analysis template	157
C.11	Bad prompt and improved prompt examples	158
D	Glossary	159
E	Model Comparison Table	162
E.1	Principles for comparison	162
E.2	Model comparison worksheet	162
E.3	Compact table for reports	163
E.4	Fields requiring primary sources	163
F	Verification Checklist	165
F.1	Compact checklist	165
F.2	Checklist for proof generation	166
F.3	Checklist for counterexamples	166
F.4	Checklist for numerical experiments	167
F.5	Checklist for Lean or formal verification	167
G	Suggested Reading and Documentation	168
G.1	Ollama documentation	168
G.2	Hugging Face GGUF documentation	168
G.3	Model cards used in these notes	169
G.4	Python, SageMath, LaTeX, and Lean	169
G.5	LLM limitations and hallucinations	169
G.6	Formal verification	169

G.7 How to update this appendix	170
---	-----

Preface

“Sed quis custodiet ipsos custodes?”

Who will guard the guardians themselves?

— Juvenal, *Satires*, Satire VI, lines 347–348

“Knowing where the trap is—that’s the first step in evading it.”

— Frank Herbert, *Dune*

These lecture notes are written for undergraduate or graduate mathematics students who want to use local large language models in a disciplined mathematical way. The intended reader is not assumed to have a background in machine learning or LLMs. However, we assume that the reader has mathematical maturity, is willing to read and write proofs, and is prepared to use a PowerShell terminal when necessary.

The notes aim to explain how free local or open-weight language models can be used as mathematical instruments: for exploration, proof attempts, counterexample search, proof auditing, LaTeX drafting, small Python or Sage experiments, and, where appropriate, support for formalization in Lean. The focus is on controlled workflows in which model output is inspected, challenged, and verified.

The central message (and warning) of the notes is the following:

Local LLMs are useful mathematical instruments, but they are not mathematical authorities. Their outputs should be treated as conjectural until verified by human(s).

This warning is the operating principle of the whole manuscript. A local model may suggest a proof strategy, find a plausible counterexample, or write useful code. None of these actions makes the result automatically correct. In particular, a fluent proof is not necessarily a valid proof, and a confident answer is not necessarily a theorem. Hence, the strengthened form of the principle is as follows:

The final arbiter of a mathematical proof remains the human mathematician.

Accordingly, the notes treat local models as tools for generating mathematical objects to be checked, like proposed lemmas, estimates, examples, counterexamples, formal statements, and code. They are not treated as a replacement for mathematicians.

We focus mainly on free local models and open-weight model families that can be used through tools such as Ollama or through GGUF model files. Paid cloud systems may occasionally be mentioned as external points of comparison, but they are not the main subject. The document is therefore closer to a mathematical laboratory manual than to a survey of commercial AI systems.

How to Read These Notes

0.1 What the Notes Are About

The notes are about building reliable habits around local reasoning models in mathematical work. In particular, we cover:

- installing and running local models,
- understanding the basic vocabulary needed to work with Ollama, GGUF files, model cards, parameters, and context length,
- designing prompts as small mathematical experiments,
- using models to explore definitions, conjectures, examples, and counterexamples,
- turning model-generated proof sketches into human-checkable arguments,
- auditing flawed proofs and locating the first invalid step,
- separating numerical evidence from proof;
- using LaTeX, Python, Sage, and optional Lean workflows as verification aids,
- coordinating several local models without confusing agreement with correctness.

The mathematical examples are deliberately elementary at the beginning. A proof that every convergent real sequence is bounded is a better first test of a mathematical workflow than a difficult theorem whose details cannot be independently checked by the reader. Later examples move toward optimization and analysis, including convexity, global minimizers, the Polyak-Łojasiewicz inequality, etc.

0.2 What the Notes Are Not About

These notes are not a benchmark report, and we do not attempt to establish a universal ranking of models for mathematics. Model performance depends on many things, like the exact checkpoint, quantization, prompt, context length, hardware, sampling parameters, and task design. When we later discuss particular model families, exact factual claims are supported by primary sources such as official documentation, model cards, or license pages. Moreover, these notes are also not a general machine learning textbook. They do not explain transformer architectures in depth, do not develop training objectives for language models, and do not present a broad theory of artificial intelligence. We discuss and introduce only the concepts needed for responsible local use. What is important, and what we want to underline here, is that the notes are not a substitute for a proof course, a numerical analysis course, or a formal methods course. We assume that the reader will still check definitions, hypotheses, estimates, edge cases, and logical dependencies. We also assume that the reader understands the difference between an observed pattern, a plausible conjecture, a numerical experiment, and a proof. Finally, these notes are not legal advice about software or model licensing. Whenever license status matters, the relevant model card or official license text must be consulted directly.

0.3 Prerequisites

The mathematical prerequisites needed to read these notes can be summarized as follows:

- familiarity with standard proof techniques such as direct proof, proof by contradiction, induction, and counterexample construction,
- basic real analysis, including sequences, limits, continuity, differentiability, and elementary convexity,
- elementary linear algebra and calculus,
- enough mathematical maturity to distinguish a definition from a theorem and a proof from a heuristic.

The computational prerequisites are also modest:

- basic command-line use: running a command, reading error messages, and copying output into a log,
- basic editing of plain-text files,
- basic LaTeX reading and compilation,
- optional familiarity with Python for numerical experiments,

- optional familiarity with Lean if the formal verification chapter is used.

Readers who do not yet know Python, Sage, or Lean can still read the main mathematical workflow chapters. Those tools are used as aids, not as prerequisites for understanding the central message.

0.4 Formatting Conventions

Commands are printed in monospaced text or in verbatim blocks. A typical command block has the form

```
ollama run model-name
```

Such blocks should be read as examples of the style of interaction, since we cannot guarantee that a particular model tag or command will remain available. Fast-changing details must be checked against current documentation.

Prompts are displayed as text that can be copied and adapted. The prompts in these notes can be seen as templates for controlled mathematical experiments. A good prompt states the problem, the assumptions, the desired output format, the forbidden shortcuts, and the verification task.

Mathematical examples are written in ordinary mathematical prose. When a model-generated proof is discussed, the notes distinguish between the model’s proposed argument and the corrected human argument. The reader should not skip this distinction.

Warnings mark points where local models commonly fail or where the user is especially likely to overtrust a fluent answer. A warning is a part of the workflow.

Exercises ask the reader to test, modify, or audit a workflow. Some exercises are mathematical, some are computational, and some are methodological. In all cases, the expected attitude is the same: produce an object, then verify it.

0.5 Sources and Bibliography

For software, model families, model cards, licenses, model tags, context lengths, parameters, and installation commands, the preferred sources are primary documentation pages and official model cards. Such information changes over time, so access dates matter. If a factual claim needs a source and the source has not yet been verified, the manuscript uses an explicit citation-needed marker rather than presenting the claim as established.

Chapter 1

Why Local LLMs for Mathematics?

This chapter explains why local large language models are worth studying in a mathematics course, and also why their use must remain subordinate to ordinary mathematical verification. The intended reader is a mathematics student. The goal is to learn how to use a local model as a controlled instrument for producing objects that can be checked: definitions, examples, possible counterexamples, proof sketches, mathematical programming fragments, LaTeX drafts, and audit notes.

Large language models have been studied as tools for mathematical reasoning. Still, the relevant literature also records persistent limitations, including brittle reasoning, factual errors, and difficulty separating fluent text from valid argument [63, 64, 65]. As it was underlined in the Preface section, these notes use a conservative rule:

Local LLMs are useful mathematical instruments, but they are not mathematical authorities. Their outputs should be treated as conjectural until verified by human(s).

This rule applies equally to a short answer, a polished proof, a generated Python script, a proposed Lean statement, and a comparison table between model outputs. A local model may be valuable precisely because it can generate candidate material quickly. However, that speed is useful only if the user retains responsibility for checking the mathematical content.

1.1 Local LLMs as mathematical instruments

A mathematical instrument is something that helps the mathematician see, compute, search, organize, or write. A ruler does not prove a theorem in Euclidean geometry. A numerical plot does not prove convexity on an interval. A computer algebra system can simplify an expression, but the user

still needs to understand what the expression means and whether the assumptions underlying the simplification are satisfied. Local LLMs should be understood and treated in the same way.

A local reasoning model can be asked (prompted) to:

- propose proof strategies for a stated theorem,
- identify missing assumptions in a proof attempt and support proof auditing,
- search for counterexamples to an overgeneralized claim,
- compare two different proof strategies;
- translate an informal argument into a structured proof draft,
- rewrite mathematical prose in clearer LaTeX,
- generate small Python or Sage scripts for numerical exploration and mathematical programming,
- suggest a Lean formalization route for a simple theorem.

The useful output is the candidate object it produces. A proposed proof becomes useful when its quantifiers, estimates, and dependencies can be checked. Similarly, a proposed counterexample becomes useful when all hypotheses and conclusions can be evaluated directly. Hence, this way of working is closer to a laboratory notebook than to an oracle. A good interaction records the problem statement, the assumptions, the model identifier, the prompt, the relevant settings, the answer, and the subsequent audit. Such records matter because model behavior depends on the exact model, interface, prompt, context window, and sampling parameters. Ollama’s documentation is a primary source for its command-line use, API usage, and context-length configuration, but local behavior still has to be checked in the user’s own environment [1, 4, 3].

1.2 Why run models locally?

Running a model locally means that the inference step is performed on the user’s own machine rather than through a paid remote model API. In practice, this usually requires a local runner, local model files or a local model cache, and enough CPU, RAM, or GPU memory for the selected model. Ollama is one local runner commonly used for this purpose. Its documentation covers command-line use, model creation, import mechanisms, and configuration objects such as Modelfiles [1, 2, 5]. Hugging Face documentation is also relevant when the model is obtained as a GGUF file, used through a local-application workflow, or documented through a model repository whose

model card states intended use, license, limitations, or other release meta-data [15, 16, 17, 18].

Control of the experimental setting

We can agree on a group of local models on a specific local setup: the same model family, the same interface, the same problem cards, and the same audit protocol. This does not guarantee identical outputs, because sampling and implementation details can still vary. It does, however, make it easier to discuss what happened. A local workflow can be logged, repeated, and compared more explicitly than an opaque remote interaction whose model version or system behavior may change without notice.

Independence from paid APIs

A course on the mathematical use of LLMs should not require to have access to a paid cloud model. Free local models make it possible to design exercises in proof exploration, proof auditing, and counterexample search without making a commercial account the center of the course. This does not mean that every local model is free of restrictions. Licensing, acceptable use, redistribution, and commercial-use terms must be checked in the relevant model card or license file. A model card is documentation, and it must be read for the exact release under discussion [18].

Classroom reproducibility

Local execution can support reproducibility in a limited but useful sense. A student can submit the prompt, the model identifier, the settings, and the output transcript. Another student can attempt to reproduce the same interaction or intentionally vary one component, for example the temperature, the model, or the prompt. This might be useful from the pedagogical point of view, since it makes model behavior an object of analysis. The word *reproducibility* must be used carefully. A priori, it does not mean that the obtained result is mathematically correct. It only means that the conditions under which a model produced the answer have been recorded well. A reproducible wrong proof is still a wrong proof.

Offline and low-connectivity work

After installation and model download, local workflows may be usable without continuous access to a remote inference service. This is convenient for private study. It also helps to understand that the model is a piece of local software with local resource constraints.

1.3 Mathematical exploration vs mathematical proof

We separate exploration from proving. Many failures in the mathematical use of LLMs come from collapsing several different categories into one. A fluent explanation, a proof sketch, a grid search, and a formal proof are different objects.

Category	Meaning and verification status
Heuristic	An informal idea, analogy, pattern, or possible route of attack. A heuristic may be useful for orientation/inspiration, but it is not proof and may hide a false assumption. It must be converted into precise mathematical claims before it can be checked.
Proof sketch	A compressed argument indicating the intended logical route. A proof sketch may identify the correct lemma structure, but omitted details can contain the actual difficulty. It must be expanded before it can be accepted as proof.
Candidate proof	A complete-looking argument that claims to prove the statement. This is the usual output requested from a model when one asks for a proof. It is not automatically correct because it is detailed, and it must be audited against definitions, assumptions, and logical dependencies.
Numerical evidence	Computations, plots, simulations, or finite tests suggesting a pattern or possible theorem. Numerical evidence is useful for conjecture generation, counterexample search, and error detection, but it does not prove a universal statement except in genuinely finite problems where all cases have been exhausted.
Formal proof	A proof term or formal derivation accepted by a proof assistant or formal proof system. This has the strongest machine-checkable status, relative to the trusted kernel, imported libraries, definitions, and the formal statement actually written. It still requires the user to check that the formal statement matches the intended informal theorem.

The distinction between these categories is especially important when working with LLMs. A model often produces text that looks like a candidate proof, even when it is only a heuristic with gaps. It may also produce numerical evidence in a style that sounds like a theorem. Conversely, a useful model answer may be short and incomplete but still point to the right lemma. The human’s task is to classify the output correctly before relying on it.

For example, suppose a model is asked whether a differentiable function with a critical point must have a global minimum. The model may answer with a plausible statement about stationary points. That is not enough. A mathematician should immediately ask: What is the domain? Is the function convex? Is the critical point local or global? Is there a counterexample?

The model can help generate candidate examples, but the verification is performed by evaluating the hypotheses and conclusion directly.

The same applies to numerical work. A plot of a function may suggest convexity over a displayed interval. It does not prove global convexity on $\mathbb{R}$. A finite grid may miss narrow intervals, asymptotic behavior, or exceptional points. Numerical evidence can be very helpful, but only after its status is correctly named.

Formal proof systems occupy a different position. A proof assistant such as Lean can check formal derivations within a specified logical environment, and Lean 4 and its mathematical ecosystem are documented in primary sources and standard tutorials [52, 53, 66, 54]. But formalization has its own boundary: the formal statement must match the intended theorem. A proof of the wrong formal statement is not a proof of the intended informal theorem.

1.4 Privacy, reproducibility, and independence from paid APIs

Local execution does not remove every risk. It is better to separate three issues: privacy, reproducibility, and independence.

Privacy

If inference is truly local, the prompt and generated text need not be sent to a remote model API during the generation step. This can be useful when prompts contain unpublished mathematical ideas, draft manuscripts, student work, or internal project notes. However, local execution is not a complete privacy guarantee. The user still has to consider how the model files were obtained, whether a graphical interface or extension adds network behavior, whether logs are stored, and whether the machine itself is appropriately secured.

This leads to a simple rule: do not put confidential material into any tool merely because it is called local. Verify the actual software path. If a workflow uses a third-party interface, synchronization service, or plugin layer, the privacy analysis must include that component.

Reproducibility

A reproducible local experiment should record at least the following data:

- the exact problem statement and assumptions,
- the model identifier as displayed locally,
- the source of the model or model card,

- the runner and version if available,
- relevant settings such as temperature, top-p, seed if supported, and context length,
- the full prompt,
- the full model output,
- the human audit result.

This record is needed because model output is fallible. A precise log makes it possible to identify whether an error came from a bad prompt, a missing assumption, a context limitation, a stochastic variation, or a misunderstanding by the human user.

Independence

Independence from paid APIs has a practical value. It allows experimentation without recurring API costs, without rate limits imposed by a remote provider, and without designing the course around one commercial system. It also allows comparisons among several free local or open-weight model families.

This independence should not be confused with self-sufficiency. A local model still depends on its training, its release terms, its quantization, the local runner, and the user’s hardware. It may fail on elementary mathematics. It may give a correct answer for the wrong reason. It may produce different answers under small prompt changes. Independence from a paid API is a property of the workflow, but not a property of mathematical correctness.

1.5 What local LLMs cannot guarantee

The most important limitations are mathematical, not technical. A local model cannot guarantee that a theorem is true, that a proof is valid, that a counterexample satisfies every hypothesis, or that a generated program correctly implements the intended mathematics. Research on LLM mathematical reasoning and factuality gives ample reason for this caution [63, 64, 65].

In particular, local LLMs cannot guarantee the following.

- **Correctness.** A generated proof may contain a false implication, a hidden compactness assumption, an invalid limit interchange, a mis-used theorem, etc.
- **Completeness.** A proof sketch may omit exactly the hard case. A counterexample search may stop before finding the relevant boundary case.

- **Faithfulness to definitions.** A model may silently replace the given definition with a more familiar but inequivalent one.
- **Global reasoning from local evidence.** A plot or derivative calculation at sample points does not establish a global statement.
- **License certainty.** A model’s availability in a local runner does not by itself determine its license, redistribution status, or acceptable use.
- **Reproducibility of answers.** Even under a local setup, small changes in parameters, prompt wording, quantization, or runner behavior may change the answer.
- **Privacy of the whole workflow.** Local inference is only one component. Interfaces, logs, backups, and plugins must also be considered.
- **Formal verification.** A LLM-assisted proof is not a formal proof merely because it is structured. Formal verification requires a precise formal statement and a proof accepted by the human. Again, we recall that:

The final arbiter of a mathematical proof remains the human mathematician.

This is the condition under which local models become useful. A human who treats model output as an answer key will learn little mathematics and may accept false proofs. However, those who treat model output as conjectural material to be tested can use the same model as a serious instrument for exploration, criticism, and writing.

Chapter 2

Installing Ollama and Running the First Model

In this chapter, we give a path from an empty terminal to a first local model session. We show how to make the local workflow concrete enough that a mathematics student can run a model, record what was run, ask a small mathematical question, and audit the answer.

Ollama, operating systems, GPU drivers, model repositories, Hugging Face Hub conventions, and model tags change over time. Hence, before using these notes, we advise rechecking the current official documentation and running the commands on the machines that students will use. The installed command's own help output, for example `ollama help`, should be treated as the operational reference for the classroom machines.

2.1 What is Ollama?

Ollama is a local runner for large language models. It provides a command-line interface for pulling, running, listing, inspecting, and removing local models. It also exposes a local service interface for more advanced workflows, but this chapter stays with terminal commands [1, 4].

Ollama supports a reproducible working pattern:

1. choose a model identifier from a verified source,
2. record the model name, tag, and command used,
3. run a prompt locally,
4. save the output or transcript,
5. audit the mathematical content independently.

Ollama also supports model definitions through a **Modelfile**. Later chapters use Modelfiles as reproducibility objects - they can record a base model, a system prompt, and selected parameters. At this point in the notes, however, the priority is the basic command-line lifecycle [2].

Remark 2.1. *Local inference avoids sending a prompt to a paid remote model API during that inference step, but the user still has to consider model provenance, downloaded files, local logs, shared machines, backups, copied transcripts, screenshots, and institutional data rules.*

Minimal first laboratory path

For a first laboratory, keep the operational path short. Install Ollama from the official operating-system page, open a terminal, check the installed version or help output, pull or run one verified placeholder model identifier, ask the elementary bounded-sequence prompt from Section 2.7, and then ask for an audit of the answer. Deeper diagnostics about disk space, RAM, VRAM, offloading, and service logs are useful when something fails. However, they are not prerequisites for understanding the mathematical workflow.

2.2 Installation on Windows, macOS, and Linux

Use the official installation page for the operating system in question. We do not advise to copy an installation command from an old blog post into a laboratory handout without rechecking it. Ollama maintains separate documentation pages for Windows, macOS, and Linux, and these pages contain operating-system-specific notes about installation, command-line availability, storage locations, and troubleshooting [7, 9, 10, 12].

Windows

On Windows, the official documentation describes Ollama as a native Windows application and states that the `ollama` command is available in `cmd`, PowerShell, or another terminal application after installation [7]. The official Windows download page also presents a PowerShell installation command [8]:

```
irm https://ollama.com/install.ps1 | iex
```

This command downloads and executes an installation script. A first PowerShell check is:

```
ollama --version
```

If that exact form is not accepted by the installed CLI, try the short form shown in Ollama's Linux verification instructions:

```
ollama -v
```

The current Ollama command source is included in the bibliography as an additional primary source for command and flag definitions that are not always displayed in every public documentation page [14]. If PowerShell reports that `ollama` is not recognized, close and reopen PowerShell after installation. If the problem remains, inspect the Windows documentation and troubleshooting notes rather than downloading executables from unofficial sites [7, 12].

macOS

On macOS, the official documentation describes installation by mounting the `ollama.dmg` file and moving the Ollama application into the Applications folder. It also documents filesystem locations and the command-line link behavior [9]. Again, the first terminal check is

```
ollama --version
```

If needed, also try:

```
ollama -v
```

If neither command is found, open the Ollama application once and check the command-line link instructions in the official macOS documentation.

Linux

On Linux, the official documentation gives an installation route and also discusses service setup and GPU driver verification. At the access date recorded in the source registry, the documented one-line installation command was [10]:

```
curl -fsSL https://ollama.com/install.sh | sh
```

This command downloads and runs an installation script. After installation, try

```
ollama -v
```

If Ollama is not running as a background service, the documentation also discusses starting the server explicitly:

```
ollama serve
```

Use a second terminal for model commands when `ollama serve` occupies the first terminal. On Linux systems using `systemd`, troubleshooting often begins with logs and service status, as described in the official troubleshooting documentation [12].

2.3 Basic commands: pull, run, list, show, remove

The placeholder `modelname` means a model identifier that has been checked against a current model source.

Command	Purpose	Comments
<code>ollama --version</code>	Print version information.	Record this in experiment notes. If the long form is not accepted, try <code>ollama -v</code> and record what worked on the installed system [10, 14].
<code>ollama pull modelname</code>	Download a model before running it.	Use only a checked model identifier.
<code>ollama run modelname</code>	Start an interactive local model session.	This is the basic terminal entry point [1].
<code>ollama list</code>	List locally available models.	Current public documentation displays <code>ollama ls</code> for listing local models, while some installations also accept <code>ollama list</code> . Record the exact form accepted by <code>ollama help</code> on the installed CLI [1, 14].
<code>ollama show modelname</code>	Inspect local model information.	Useful before recording a reproducibility log. For Modelfile-related inspection, official documentation also uses <code>ollama show -modelfile modelname</code> [2, 14].
<code>ollama rm modelname</code>	Remove a local model.	Check that the intended model and tag are being removed [1].

It might be helpful, for reproducing purposes, to keep a small text log:

Date:
Operating system:
Ollama version:
Model identifier:
Command used to run the model:
Prompt:
Output file or transcript location:
Human audit status:

2.4 Running a first local model

Choose a model identifier from a current, documented source that plausibly fits the available storage and memory. Do not infer that a model fits your computer from its name alone.

A starting PowerShell session on Windows might look like this:

```
ollama --version
ollama pull modelname
ollama run modelname
```

(A macOS or Linux terminal session has the same structure.)

Inside the interactive session, begin with a harmless mathematical task whose answer can be checked from first principles, for example:

State the definition of convergence of a real sequence.

Then ask a follow-up question that requires proof rather than a definition. The model's response should be treated as a draft to inspect.

Remark 2.2. *If the model is too slow, produces memory errors, or causes the machine to swap heavily, stop the session and choose a smaller or more compressed model. Poor performance may be a mismatch between the selected checkpoint, quantization, context length, and local hardware.*

2.5 Running GGUF models from Hugging Face

Hugging Face documents a direct pattern for using the GGUF model repositories with Ollama. The documented generic forms are [16]:

```
ollama run hf.co/{username}/{repository}
ollama run hf.co/{username}/{repository}:{quantization}
```

In these notes we write the second form schematically as:

```
ollama run hf.co/username/model-repository:quantization-tag
```

A concrete command should be used only after checking the actual model repository, its files, its model card, and any license or usage restrictions. Hugging Face's GGUF documentation explains that GGUF repositories can contain model tensors together with standardized metadata and that the Hub provides tools for finding and inspecting GGUF files [15]. A model card can document intended use, limitations, license metadata, and other release information [18].

Do not replace the placeholder by guessing a plausible-looking tag:

```
ollama run hf.co/someone/some-model:Q4_K_M
```

Instead, open the repository page, inspect the available files or the Hub’s Ollama snippet, read the model card, and record the exact command. Third-party GGUF conversions require particular care: they may differ from the original model release in format, quantization, prompt template, license presentation, or metadata completeness.

Ollama also documents a separate local import route for GGUF files. Namely, create a `Modelfile` containing a `FROM /path/to/file.gguf` line, then use `ollama create` to build a local model name [5]. The direct Hugging Face route and the local import route should not be conflated in reproducibility notes.

2.6 Checking available disk, RAM, and GPU memory

A local model can fail for ordinary resource reasons. Simply, it may not fit on the disk. It may fit on disk but not in memory. It may run on the CPU when the user expected GPU acceleration. And it may start correctly but slow down severely when the context length is increased. Ollama’s hardware support, FAQ, and context-length documentation should be checked before making hardware-sensitive claims [6, 11, 3].

Windows PowerShell checks

The following commands are local inspection commands, and they give a first inventory only. They should be supplemented by Windows Settings, Task Manager, or vendor tools when precision matters.

```
Get-PSDrive -PSProvider FileSystem
Get-CimInstance Win32_ComputerSystem
Get-CimInstance Win32_VideoController
nvidia-smi
ollama ps
```

The first command lists filesystem drives available to the PowerShell session. The second and third query Windows system-management information. Microsoft documents `Get-PSDrive` and `Get-CimInstance` - the latter is the modern CIM-based command for querying system classes on Windows [56, 57]. For NVIDIA GPUs, `nvidia-smi` is the usual vendor command-line tool. Ollama’s own `ps` can be helpful after a model is running. The FAQ explains that `ollama ps` shows loaded models and a processor column indicating whether the model is loaded on CPU, GPU, or a mixture of CPU and GPU memory [11].

macOS checks

A minimal macOS check is:

```
df -h
sysctl hw.memsize
ollama ps
```

The first command checks disk usage. The second prints physical memory in bytes. The third becomes informative after a model is loaded. The registered macOS manual-page sources document `df -h` and the `hw.memsize` key for `sysctl` for a specific macOS manpath. Nevertheless, one should prefer `man df` and `man sysctl` on their own installed system when exact semantics matter [59, 60]. Do not translate a GPU-memory rule from an NVIDIA desktop directly to a Mac without checking the current Ollama macOS and hardware documentation [9, 6].

Linux checks

A minimal Linux check is:

```
df -h
free -h
nvidia-smi
ollama ps
```

The first command checks disk usage, and the second gives a compact memory summary; GNU Coreutils documents `df`, and the Linux manual page for `free` documents physical and swap memory reporting [61, 62]. The NVIDIA command is relevant only when an NVIDIA GPU and driver are installed. The official Linux documentation explicitly uses `nvidia-smi` as a driver verification command in its optional CUDA-driver discussion [10]. For AMD, Vulkan, or container-based configurations, consult the current hardware support page rather than adapting NVIDIA-specific instructions [6].

2.7 A first mathematical prompt

The first mathematical prompt should be easy enough to check without trusting the reasoning model. The following example is suitable because the theorem is standard, short, and exposes whether the model knows how to use the definition of convergence:

Prove that every convergent sequence of real numbers is bounded.
Give a rigorous proof and explicitly state where convergence is used.

For short prompts or interactive session one may pass the text directly to `ollama run modelname`. For longer mathematical prompts, however, it might be helpful to use a PowerShell *here-string*. This avoids quoting problems and makes the prompt readable. For example:

```
$prompt = @"
Prove that every convergent sequence of real numbers is bounded.
Give a rigorous proof and explicitly state where convergence is used.
"@
```

```
ollama run modelname $prompt
```

When a prompt is passed directly as an argument, like above, Ollama treats this as a one-shot request: it prints an answer and then returns to PowerShell. This is useful for scripts, but it is not convenient when one wants to ask follow-up questions in the same conversation. To preserve the interactive session, first start the model

```
ollama run modelname
```

and then enter the long prompt inside the Ollama session. For multiline input in the Ollama prompt, wrap the text in triple quotes:

```
>>> """
Prove that every convergent sequence of real numbers is bounded.
Give a rigorous proof and explicitly state where convergence is used.
"""
```

After the model answers, the session remains open and further questions can be asked in the same context.

After the model answers, ask it to audit itself:

```
Audit your previous proof. Identify every hidden assumption.
```

The expected mathematical structure is known. Namely, if $(x_n)_{n \geq 1}$ converges to x , then for $\varepsilon = 1$ there exists N such that $|x_n - x| < 1$ for all $n \geq N$. Therefore $|x_n| \leq |x| + 1$ for all $n \geq N$. If $N \geq 2$, take

$$M = \max\{|x| + 1, |x_1|, \dots, |x_{N-1}|\}.$$

If $N = 1$, take $M = |x| + 1$. Then $|x_n| \leq M$ for all $n \geq 1$. The point at which convergence is used is the existence of a tail bound for $\varepsilon = 1$.

An audit should check at least the following:

- Did the proof state which norm or absolute value is being used?
- Did it use convergence with a concrete $\varepsilon > 0$?

- Did it handle the finitely many initial terms before N ?
- Did it justify the existence of a maximum over finitely many real numbers?
- Did it accidentally assume monotonicity, positivity, or a known limit bound that was not given?

This example is intentionally elementary, and the aim was to establish a workflow: prompt, proof attempt, audit prompt, human verification, and a final corrected proof if needed.

2.8 Common installation problems

Most first-run failures are operational, and the following list is a triage guide.

The command is not found

Open a new terminal after installation. On Windows, the documentation states that the command is available in terminal applications after installation and gives local application locations, while on macOS, the documentation describes the application bundle and command-line link behavior [7, 9]. Do not add arbitrary directories to `PATH` unless you know which binary they contain.

The server is not running

If the command-line client cannot connect to the local service, open the Ollama application or start the service according to the operating-system documentation. On Linux systems using `systemd`, useful first checks are:

```
sudo systemctl status ollama
journalctl -u ollama --no-pager --follow --pager-end
```

The official troubleshooting page gives log locations for macOS, Linux, containers, and Windows. Logs are often more informative than repeated guessing at prompts [12].

The model download fails

Check network access, proxy configuration, and available disk space. Ollama’s FAQ includes proxy and environment-variable notes, while the troubleshooting page explains where logs are stored on supported platforms [11, 12]. Sometimes the network filtering may be the actual cause.

The model name or tag is not found

Check the exact spelling of the model identifier. For Hugging Face GGUF repositories, open the repository page and confirm that the selected GGUF file or quantization tag exists. The command pattern `hf.co/username/model-repository:quantization-tag` is documented, but the placeholder does not make every possible tag valid [16].

The machine runs out of disk space

List local models and remove unneeded ones by using:

```
ollama list
ollama rm modelname
```

Then inspect disk usage with operating-system tools. The FAQ discusses where models are stored and how to set a different model directory. The Windows documentation also discusses changing the model location through `OLLAMA_MODELS` when the default home-directory location is unsuitable [11, 7].

The model is slow or the GPU is not used

Slowness may reflect model size, quantization, context length, CPU offloading, driver configuration, or lack of a supported accelerator. It is not evidence that the mathematical prompt gave the model a complicated and deep mathematical problem to solve. Use:

```
ollama ps
nvidia-smi
```

where applicable. Ollama’s context-length documentation gives an example of `ollama ps` reporting processor allocation and context information, and the hardware page should be checked for current GPU support notes [3, 6].

The output contains odd boxes or glyphs

The Windows documentation notes that some older terminal fonts may display Ollama progress characters as unknown squares. This is a terminal-display issue, not a mathematical or model-quality issue [7].

Increasing context length causes failures

A larger context length increases memory requirements. Do not increase context length merely because a model card advertises a large context window. For the first mathematical exercises, short prompts plus saved transcripts are usually preferable to very long contexts [3].

Chapter 3

Basic Vocabulary of Local Reasoning Models

This chapter introduces the vocabulary needed to read model cards, inspect local models, and make practical choices about locally running reasoning models. The intended reader is a mathematics student rather than a machine learning specialist. We are aiming at making them precise enough that later chapters can discuss model choice, prompt design, reproducibility, and hardware limits without repeatedly stopping for terminology.

A useful habit is to separate three levels of description. First, there is the mathematical object in the user’s work: a theorem, a definition, a counterexample, a program, or a proof obligation. Second, there is the local inference system: the model file, quantization, context window, memory allocation, and sampling settings. Third, there is the written record of the experiment: model identifier, exact prompt, relevant settings, and audit result. Confusing these levels is a common source of overclaiming. A model may be large, local, quantized carefully, and run reproducibly, but none of that makes its proof output automatically correct.

3.1 Parameters and model size

A parameter of a neural language model is a learned numerical coefficient used by the model during inference. In a transformer-like language model these coefficients occur in large arrays, usually called weights. The phrase “14B model” or “32B model” is shorthand for approximately 14 billion or 32 billion learned parameters. It is a statement about the scale of the stored model.

A useful analogy is a large finite-dimensional approximation space. Sup-

pose a function is represented as

$$g(x) = \sum_{j=1}^N c_j \phi_j(x),$$

where the basis functions ϕ_j are fixed and the coefficients c_j have been learned from data. Increasing N may increase expressive capacity, but it does not by itself guarantee that a particular approximation is correct on a particular problem. Similarly, increasing the number of model parameters may enhance representational capacity, but a model with more parameters can still perform a false algebraic manipulation, omit a quantifier, or invent an invalid lemma.

For local use, model size matters because model weights must be stored on disk and loaded into system memory, GPU memory, or both. Ollama’s model-detail API reports fields such as format, family, parameter size, and quantization level for a model in its example response [13].

When recording a local model experiment, write down at least:

- the model name and tag as used by the runner,
- the source of the model or GGUF file, if it came from an external repository,
- the reported parameter size, if available,
- the reported quantization level, if available,
- the date and local software version.

Therefore, a parameter count should be considered in conjunction with the architecture type, quantization, context setting, and model card.

3.2 Dense models and mixture-of-experts models

A dense model is one in which, informally, the main body of model parameters is used for each token processed by the model. Hence, in this case, the model is not organized as a large set of alternative expert submodules of which only a small subset is selected for a token. A mixture-of-experts model, usually abbreviated MoE, uses a routing mechanism that selects only some expert components for a given token. The stored model may contain many experts, while each token is processed through only a subset of them. This sparse activation idea is standard in the MoE literature and appears in modern language-model examples such as Switch Transformers and Mixtral [67, 68]. A mathematical analogy is sparse representation. Instead of always using a full sum

$$\sum_{j=1}^N c_j \phi_j(x),$$

one may use a smaller selected set

$$\sum_{j \in I(x)} c_j \phi_j(x),$$

where $I(x)$ depends on the input. This analogy is not exact, but it explains the basic local-computation issue. Namely, total stored capacity and per-token activated computation need not coincide.

For using local models, the dense/MoE distinction matters for the following three reasons:

1. It affects how to read model-card metadata or repository names that distinguish total parameter count from activated parameter count.
2. It affects rough expectations about memory and speed, because the stored weights and the active computation are not the same object.
3. It affects reproducibility notes, because an MoE model’s exact behavior is still determined by the complete implementation, tokenizer, router, quantization, and inference engine.

We do not claim that MoE models are intrinsically better for mathematics. A dense model can outperform an MoE model on a particular mathematical prompt, and an MoE model can outperform a dense model on another. The local user must test and audit each particular usage and experiment.

3.3 Active parameters versus total parameters

For dense models, the reported parameter count is usually the main size number that users discuss. For MoE models, model cards may distinguish total parameters from activated parameters. The total parameters refer to the parameters stored in the full model, while the activated parameters refer to the subset used for a token or inference step, depending on the model-card convention. For example a model-card for the Qwen3.6-35B-A3B model card reports a language model with 35B total parameters and 3B activated parameters, and separately describes the number of experts and activated experts [25]. This kind of statement should be copied only from the relevant model card.

For local inference, total and active parameters affect different practical questions:

- **Disk storage:** the stored file is related to the full set of weights after serialization and quantization.
- **RAM or VRAM loading:** the runner may need to load all or part of the stored model into memory, even if only some experts are activated for a token.

- **Per-token compute:** for an MoE model, the activated part can be much smaller than the total stored model, but the actual speed still depends on implementation, hardware, batch size, context length, and offloading.
- **Mathematical reliability:** neither total nor active parameters certify correctness of a proof.

If a model card states a total parameter count and a separate activated parameter count, write exactly those two numbers and cite the card. If the model source does not state an activated parameter count, do not infer one from the repository name. If a third-party GGUF repository uses shorthand that appears to encode architectural information, check the original model card and the conversion repository separately.

3.4 Quantization

Quantization is the use of lower-precision representations for model weights or related tensors. For example, one might replace

$$0.137492 \mapsto 0.14.$$

This illustrates the idea of storing less numerical detail, however, it is not a description of real GGUF quantization. Actual quantization methods use block structures, scales, minima, and other conventions. Hugging Face’s GGUF documentation describes GGUF metadata and lists multiple quantization types, while the llama.cpp quantization README describes a workflow in which a model is converted to GGUF and then quantized [15, 20].

Quantization is attractive for local work because it can reduce storage and memory requirements. The trade-off is that weights are approximated. However, the effect of that approximation is not a statement of the form “Q5 is always enough” or “Q8 is always better”. It depends on the base model, the quantization method, the task, the prompt, the inference engine, and the hardware. Quantization should be treated as a numerical approximation layer. It is analogous to replacing exact arithmetic by finite-precision arithmetic in a numerical computation.

3.5 Quantization labels: Q4, Q5, Q6, Q8, FP16

GGUF repositories commonly use labels such as Q4_K_M, Q5_K_M, Q6_K, Q8_0, or FP16. These labels describe the numerical format or quantization choice used for a particular model file. Hugging Face documents GGUF quantization types such as Q8_0, Q6_K, Q5_K, Q4_K, lower-bit K-quants, and IQ variants. llama.cpp examples also show labels such as Q4_K_M, Q5_K_M, Q3_K_M, IQ2_M, and IQ1_M in quantization tables and commands [15, 20].

Label	Interpretation
FP16	A 16-bit floating point representation. Usually larger than common 4-bit or 5-bit GGUF quantizations. It is closer to the original half-precision weights when such weights are the source format. However, it is not automatically a better local choice if the file does not fit the available memory.
Q8_0	An 8-bit GGUF quantization label. It is often used when the user wants a relatively high-precision quantized model.
Q6_K	A K-quant label around the 6-bit family. It is usually a compromise between file size and numerical fidelity.
Q5_K_M	A commonly used 5-bit K-quant variant in GGUF repositories. The suffix should be read as part of the file’s quantization convention.
Q4_K_M	A commonly used 4-bit K-quant variant. It often makes larger models locally feasible.
Q3/Q2/IQ2/IQ1	Lower-bit or importance-quantized families. These may be useful under tight memory constraints, but they involve stronger compression and should be tested carefully before use in proof-oriented workflows.

There are two common mistakes to avoid. First, **Q8** is not “twice as intelligent” as **Q4**. The labels refer only to numerical storage and quantization conventions, but not to a scale of intelligence. A model file with a higher-bit quantization may preserve weights more accurately, but proof quality is affected by the base model, training procedure, training set, architecture, prompt, context, sampling, etc. Furthermore, a smaller model with a higher-bit quantization is not automatically better than a larger model with a lower-bit quantization. A 14B model in **Q8_0** and a 32B model in **Q4_K_M** differ in at least two dimensions: base parameter count and numerical representation. The 32B **Q4_K_M** model may give better answers on some mathematical tasks, because the underlying model capacity and training may matter more than the extra precision of the smaller model. It is also possible that the 14B **Q8_0** model is preferable on a given machine because it is faster, more stable, or better aligned with the prompt style. Hence, it is important to experiment and test the concrete models under recorded conditions and audit the results.

3.6 Context window and tokens

A token is a unit of text used internally by the model’s tokenizer, and they are not the same as words. Depending on the tokenizer, a word may be one token, several subword tokens, or combined with punctuation in a way that is not obvious from ordinary writing. Hugging Face’s Transformers documentation describes common subword tokenization algorithms such as byte pair encoding, Unigram, and WordPiece, and emphasizes that tokenization splits text into units between words and characters [22]. The context win-

dow is the maximum number of tokens that the model has access to when generating the next token. Ollama’s context-length documentation states this directly and also notes that increasing context length increases memory requirements [3].

For mathematical work, context length matters because proofs are often dependency-heavy. If the prompt contains a theorem statement, definitions, a candidate proof, an audit checklist, and previous model output, the model may need many tokens simply to keep all relevant material available. Unfortunately, a longer context does not guarantee better reasoning. A long prompt can also bury the main assumptions, include contradictory instructions, or increase memory use enough to force CPU offloading. It is useful to distinguish between:

- **problem context:** the theorem, assumptions, definitions, and notation,
- **workflow context:** instructions about role, audit style, output format, and forbidden shortcuts,
- **conversation context:** previous attempts, model answers, and human comments,
- **irrelevant context:** material that consumes tokens without helping the current mathematical task.

For rigorous work, a shorter and cleaner context might be better than maximal context. Moreover, a theorem statement with exact hypotheses and a compact audit checklist may outperform a long conversation transcript.

3.7 KV cache

During autoregressive generation, a large language model generates text one token at a time. Transformer attention uses key-and-value (KV) vectors associated with previous tokens. A KV cache stores these key-value computations so that they can be reused instead of recomputed from scratch at every generation step. Hugging Face’s Transformers documentation describes the role of key-value vectors in attention, explains that caches store these calculations for reuse, and notes that cache choices trade memory use and speed [21].

The KV cache is important because it grows with context and generation. A long mathematical prompt plus a long generated proof can require substantial cache memory. Because of this, context length and maximum generated tokens affect RAM or VRAM use. Ollama’s context-length documentation explicitly warns that setting a larger context length increases memory requirements [3]. Hence, when a model fits at a short context length

but fails or slows down at a long context length, the problem may not be the stored model file alone. It may be the runtime memory required for the context and cache. This is especially relevant in proof auditing via LLMs, where the input may contain several pages of argument and the output may be another long document.

3.8 RAM, VRAM, CPU, GPU, and offloading

Recall that RAM is system memory, and VRAM is memory on the GPU. CPU inference uses the processor and system memory. GPU inference uses the graphics processor and VRAM, when supported by the software stack and hardware. On machines with limited VRAM, a local runner may place only part of the model on the GPU and leave the rest in system memory. This is commonly called *offloading*.

Ollama’s FAQ explains that `ollama ps` can be used to see which models are loaded and that the processor column may report cases such as a model loaded entirely on GPU, entirely on CPU, or split between CPU and GPU [11]. Ollama’s context-length page also instructs users to verify context length and model offloading with `ollama ps` [3]. Offloading should be seen as a memory-management compromise. If the GPU cannot hold the complete runtime state, part of the work may move through slower memory paths. The usual practical effect is slower generation (sometimes much slower). In the case of offloading, a proof-audit workflow may require several long generations. A model that is theoretically runnable but practically very slow may be a poor choice. A local workflow, therefore, should record:

- available disk space before pulling or importing a model,
- system RAM,
- GPU name and VRAM, if a discrete GPU is used;
- whether the model was loaded on CPU, GPU, or split,
- context length and sampling settings.

These details are needed because they help the experiment to be reproducible, and explain practical failures such as out-of-memory errors, severe latency, or inconsistent results.

3.9 GGUF and local inference formats

GGUF is a file format used in local inference workflows. Hugging Face describes GGUF as a binary format designed for use with GGML and other executors, and notes that GGUF encodes both tensors and standardized

metadata [15]. This metadata can include information useful to local tools, such as tokenizer-related information, tensor names, tensor shapes, and precision metadata, depending on the file.

In the Ollama workflow there are two common GGUF routes. One route is to build a local Ollama model from a GGUF file through a Modelfile; the Ollama Modelfile documentation includes a `FROM ./ollama-model.gguf` pattern and specifies that the GGUF file location should be absolute or relative to the Modelfile location [2]. The other route is to run a GGUF repository from the Hugging Face Hub using the Hub’s Ollama integration. Hugging Face documents the pattern `ollama run hf.co/{username}/{repository}` and, for a chosen quantization, `ollama run hf.co/{username}/{repository}:{quantization}` [16].

3.10 Temperature, seed, determinism, and reproducibility

Many local runners expose sampling parameters. Ollama’s Modelfile documentation lists parameters including `temperature`, `seed`, `num_ctx`, `num_predict`, `top_k`, `top_p`, and others [2]. These parameters influence (proof and text) generation. They should be recorded in mathematical experiments.

Temperature is a sampling parameter. Lower temperature usually makes generation more concentrated, while higher temperature usually allows more variety. For proof drafting, a lower temperature may make output more stable, but stability does not force correctness by default.

A **seed** is intended to make generation more reproducible for the same prompt and settings. Ollama’s documentation says that setting a specific seed makes the model generate the same text for the same prompt [2]. In practice, reproducibility should still be treated cautiously. Different software versions, hardware paths, model files, templates, and settings may change outputs. Therefore, the reproducibility record should include more information than just the seed value. A reproducibility note record can include:

Model: `modelname`; runner: Ollama; date: yyyy-mm-dd; context: 4096 or recorded value; temperature: recorded value; seed: recorded value if set; quantization: recorded value if shown; prompt file: recorded path; output file: recorded path.

The parameter `num_ctx` sets the size of the context window used by Ollama when the model generates the next token. Informally, this is the maximum number of tokens from the current prompt and conversation state that the model is allowed to keep available as context during generation. It should not be confused with a guarantee of mathematical correctness: increasing

`num_ctx` may allow the model to see more of the problem statement or previous discussion, but it does not make the model a proof checker. A larger context window can also increase memory use and may reduce practical speed. See the Ollama Modelfile reference and context-length documentation [2, 3].

The parameter `num_predict` sets an upper bound on the number of new tokens that the model is allowed to generate in its answer. It is therefore an output-length control, not an input-context control. For mathematical work, too small a value may truncate a proof in the middle of an argument, while too large a value may encourage unnecessarily long answers. It should be treated as a practical stopping parameter rather than as a parameter controlling the validity of the reasoning. The Ollama Modelfile reference describes `num_predict` as the maximum number of tokens to predict when generating text [2].

At a single decoding step, suppose that the model assigns probabilities

$$p(v \mid x_{1:t})$$

to possible next tokens v in the vocabulary V , given the previous tokens $x_{1:t}$. The parameter `top_k` restricts sampling to the k tokens with the largest probabilities. If $V_k \subset V$ denotes this set of k most probable next tokens, then all tokens outside V_k are discarded and the remaining probabilities are renormalized:

$$\tilde{p}(v \mid x_{1:t}) = \begin{cases} \frac{p(v \mid x_{1:t})}{\sum_{u \in V_k} p(u \mid x_{1:t})}, & v \in V_k, \\ 0, & v \notin V_k. \end{cases}$$

The next token is then sampled from $\tilde{p}$, assuming stochastic sampling is being used. Smaller values of `top_k` make the sampling more restrictive; larger values allow more candidate tokens. This is a decoding parameter, not a mathematical-reliability parameter. A proof produced with a smaller or larger `top_k` still requires human or formal verification. See the Ollama Modelfile reference and the Hugging Face generation documentation [2, 72].

The parameter `top_p`, restricts sampling to a dynamically chosen set of high-probability tokens. Let the vocabulary tokens be ordered so that

$$p(v_1 \mid x_{1:t}) \geq p(v_2 \mid x_{1:t}) \geq p(v_3 \mid x_{1:t}) \geq \dots$$

For a threshold $p_{\text{nuc}} \in (0, 1]$, define

$$m = \min \left\{ r \geq 1 : \sum_{i=1}^r p(v_i \mid x_{1:t}) \geq p_{\text{nuc}} \right\}.$$

The nucleus is then

$$V_{p_{\text{nuc}}} = \{v_1, \dots, v_m\}.$$

Tokens outside $V_{p_{\text{nuc}}}$ are discarded and the probabilities on $V_{p_{\text{nuc}}}$ are renormalized:

$$\tilde{p}(v \mid x_{1:t}) = \begin{cases} \frac{p(v \mid x_{1:t})}{\sum_{u \in V_{p_{\text{nuc}}}} p(u \mid x_{1:t})}, & v \in V_{p_{\text{nuc}}}, \\ 0, & v \notin V_{p_{\text{nuc}}}. \end{cases}$$

The next token is sampled from this truncated and renormalized distribution. Unlike `top_k`, the number of allowed tokens is not fixed in advance: it depends on how concentrated the model’s next-token probability distribution is at that step. In Ollama, `top_p` may be used together with `top_k`, so one should not interpret it in isolation when several decoding filters are active. The method is commonly known as nucleus sampling [73, 72].

3.11 Throughput, latency, and practical speed

Users of local reasoning models often speak about model speed as if it were one number. It is better to separate at least two quantities. Namely:

- **Latency** is the delay before useful output appears. For an interactive proof discussion, high latency makes the workflow unpleasant even if the eventual tokens per second are acceptable.
- **Throughput** is the rate at which tokens are processed or generated. It is often reported in tokens per second, but the exact number depends on prompt length, generated length, hardware, quantization, context, batching, and whether the model is already loaded.

Ollama’s API documentation includes timing and token-count fields such as prompt evaluation count, prompt evaluation duration, generation evaluation count, and generation evaluation duration in example responses [4].

For proofs generation, speed has a special role. A slow model may still be useful for a difficult audit if it produces a careful candidate critique. A fast model may be better for exploratory counterexample search or LaTeX rewriting. Testing only a short conversational prompt does not measure the cost of a long proof audit. On the other hand, testing only the generation speed does not measure the cost of reading a long theorem statement.

3.12 Why bigger is not always better

The phrase “bigger model” can mean several different things: more total parameters, more active parameters, a larger file, a longer context window, or a less compressed quantization. For mathematical workflows, bigger is not always better for at least six reasons stated below.

1. **Local fit:** a model that does not fit in memory, or runs mostly through CPU offloading, may be too slow for the intended workflow.
2. **Prompt discipline:** a larger model can still fail if the prompt omits hypotheses, uses ambiguous notation, or asks for an invalid conclusion.
3. **Quantization trade-off:** a larger but heavily quantized model and a smaller but lightly quantized model are not ordered by a simple rule.
4. **Training and specialization:** a smaller model trained or tuned for reasoning may outperform a larger general model on some proof-auditing tasks.
5. **Context cost:** a longer context setting can increase memory use and reduce speed. It can also invite undisciplined prompt construction.
6. **Verification burden:** a more fluent long proof may require more human auditing than a short, explicit proof sketch.

Therefore, before running the experiment, we should ask ourselves *Which locally runnable model, with which quantization and settings, is useful for this mathematical task, on this hardware, under this verification protocol?* Unfortunately, this question has no permanent answer. Model cards change, GGUF conversions appear and disappear, Ollama behavior evolves, and classroom hardware differs. It seems that the only stable principle is the one stated in the Preface, i.e., local LLM outputs are conjectural until verified, and the final arbiter of a mathematical proof remains the human mathematician.

Chapter 4

A Field Guide to Free Local Reasoning Models

This chapter is dedicated to help a mathematician decide how a local reasoning model might be used in a mathematical workflow, what must be checked before using it, and what should never be inferred from the word *reasoning*. The main object of interest is a benchmark score or an attractive model page. Moreover, we aim at a reproducible mathematical practice in which machine-generated text is treated as a source of conjectures, drafts, counterexamples, code, and possible failure modes.

The reasoning model ecosystem changes rapidly. Exact repository names, quantization tags, licenses, context windows, and recommended inference settings must therefore be checked against the current model card before a lecture or laboratory session. The examples below are source-verified examples as of the access date recorded in the source registry. However, they should not be read as permanent recommendations.

4.1 What does *free local model* mean?

In these notes a *free local reasoning model* means a model that can be used without paying a per-token cloud API fee, and that can be run on hardware controlled by the (human) user, in at least some quantized or otherwise locally deployable form. This is a practical definition, and it says something about use on a user’s machine. However, it does not by itself say whether the model is open-source, whether redistribution is allowed, whether commercial use is allowed, or whether the training data are available. For mathematical work, local execution has (at least) three practical advantages. First, we can repeat a prompt without depending on a paid account, a remote rate limit, or a proprietary interface. Second, we can compare several models under similar prompts and record the exact local configuration. Third, sensitive draft material need not be sent to a cloud model merely to obtain a proof

sketch, a counterexample attempt, or a $\text{\LaTeX}$ rewrite. A model can be free to run locally and still be unsuitable. For instance, the files may be too large, the hardware requirements may be unreasonable, the license may not fit the intended use, or the model may be weak on the needed mathematical style. Conversely, a small model that is not strong enough to write a complete proof may still be useful as a syntax assistant, a counterexample generator for elementary claims, or a second independent reader of a candidate proof.

We use the following distinctions.

Free to run locally. The user can run the model on local or self-controlled hardware without a per-token paid cloud API. Hardware, electricity, storage, and maintenance are still costs.

Open-weight. The trained weights are available for download or inspection under the stated terms. This does not automatically mean that the training code, full training data, or data curation pipeline are all open.

Open-source. In software, this term is normally tied to criteria such as source-code availability, redistribution, derived works, and non-discrimination. The Open Source Initiative gives a formal Open Source Definition for software, and also maintains an Open Source AI Definition for AI systems [44, 45]. These definitions should not be replaced by informal marketing usage.

Permissive license. A license such as Apache License 2.0 or MIT usually grants broad rights to use, copy, modify, and redistribute under stated conditions [46, 47].

Local availability. The model can be run in a local inference stack such as Ollama, llama.cpp, LM Studio, MLX, vLLM, or a compatible application. Local availability may depend on third-party quantizations, a particular GGUF conversion, or hardware-specific support. It must be verified for the exact model and quantization.

4.2 Open-weight, open-source, permissive license, and local availability

The distinction between open weights and open source matters in mathematics because it affects reproducibility. A public weight file may allow a user to reproduce outputs under the same local inference settings. It may not allow a researcher to reproduce the training procedure, audit the training data, or understand why the model behaves in a particular way on proof tasks. The source registry, therefore, records the type of source used: model card, official repository, platform documentation, license text, or research paper.

Several recurring patterns should be checked before using any local reasoning model. Namely:

1. Check whether the model card states a license and whether the files are actually accessible under that license.
2. Check whether the model card and repository agree. If they disagree, treat the situation as ambiguous until verified.
3. Check whether the model is a base model, an instruction model, a reasoning model, a distilled model, a code model, or a domain-specialized model.
4. Check whether the model card describes intended use and limitations. A model intended for general chat may be a poor proof auditor, while a model intended for mathematics may be poor outside mathematics.
5. Check whether the local version is official or a third-party conversion. A GGUF quantization can be very useful, but it is not the same artifact as the original model card unless the source explicitly says so.
6. Record the exact repository, quantization, Ollama version, operating system, and relevant parameters before presenting outputs as classroom examples.

4.3 Why model families matter more than single model rankings

A single model tag may be replaced, renamed, quantized by different people, or run with different context settings. A family, by contrast, often has a recognizable design culture such as general instruction following, explicit reasoning traces, code orientation, open research transparency, small-device deployment, or multilingual capability. For mathematical work, this family-level view is usually more useful than a leaderboard. The useful question is

Which model, under which local configuration, is useful for which mathematical subtask, with which independent check?

A model card is, therefore, evidence about intended use, architecture family, release terms, and known limitations. Hugging Face model cards are designed to document model characteristics and intended use, but the responsibility for mathematical verification remains outside the model card [18].

The following family notes use the same five-field template. The user should read the template as a checklist. In each case the important questions

are: what role can this family play, what must be verified, and what must not be inferred from the family name?

4.4 Qwen and Qwen reasoning models

Qwen is a broad model family rather than a single model. Qwen model cards include both dense and mixture-of-experts (MoE) variants. Recent Qwen releases describe both thinking and non-thinking modes in some models [24]. This makes the family useful as a comparison point in local mathematical experiments. Namely, one can compare a general instruction model, a reasoning-oriented model, and a code-oriented model without leaving the same broad ecosystem.

Typical role in mathematical work. Qwen-family models are often suitable as general mathematical explorers. They can restate definitions, propose proof strategies, produce examples, and compare approaches. Reasoning-oriented variants can be useful for longer proof sketches or for decomposing a problem into lemmas. Code-oriented variants can be useful for Python, Sage, or Lean-adjacent syntax tasks, subject to compilation or execution.

Strengths. The broad family structure is a practical strength. A user can test a small general model, a reasoning model, and a code model under related prompts. Some model cards also make local-app and quantization discovery visible through the Hugging Face interface, which helps students locate local inference options [23, 26].

Cautions. A Qwen-family label does not imply a specific license, context length, memory requirement, or mathematical ability. These must be read from the exact model card. Mixture-of-experts models require special care because the total number of parameters and active parameters are not the same quantity.

What should be verified. One should verify the exact repository, the license, whether the model is dense or MoE, whether it is instruction-tuned or reasoning-tuned, and whether a local GGUF or Ollama-compatible version is official or third-party. The user should also test whether the model preserves hypotheses in a multi-step proof.

Suitable workflow role. In a multi-model workflow, a Qwen-family model can serve as an *explorer*, *programmer*, or *first-draft proof writer*. It should not be the final proof checker.

4.5 QwQ

QwQ is a reasoning-oriented member of the Qwen ecosystem. Its model card describes it as a reasoning model and identifies local-app quantization pathways through the Hugging Face interface [23]. In mathematical use, this makes QwQ a natural candidate for tasks where the user wants a longer chain of intermediate reasoning rather than a short final answer.

Typical role in mathematical work. QwQ is suitable for proof exploration, lemma discovery, and comparison of approaches. For example, on a problem about the Polyak–Łojasiewicz inequality, it can be asked to separate definitions, examples, possible counterexamples, and proof attempts.

Strengths. The main strength is willingness to produce extended reasoning. This is useful when the user wants to expose hidden steps and then audit them. A long output can reveal where the model used assumptions on, for example, compactness without justification, assumed the existence of a minimizer, reversed an inequality, or confused a stationary point with a global minimizer.

Cautions. A plausible chain of text can contain a false lemma, an unjustified limit passage, or a hidden assumption. In some cases, a longer answer creates more opportunities for error. Hence, the user should treat the reasoning trace as audit material, but not as evidence of correctness.

What should be verified. One should verify every invoked theorem, every quantifier, and every transition from local to global information.

Suitable workflow role. In this course, QwQ is a plausible candidate for the *explorer* or *candidate proof writer* role. It should be paired with a different model as a skeptic and, for formalizable results, with a proof assistant or executable check.

4.6 DeepSeek-R1 and distilled reasoning models

DeepSeek-R1 model cards distinguish DeepSeek-R1-Zero, DeepSeek-R1, and distilled dense models based on other model families. Moreover, they note limitations of the early zero-RL variant, such as repetition, readability problems, and language mixing [27]. Later model-card releases also record licensing statements for the relevant release, and these statements should be checked for the exact model used [28].

A *distilled model* is a model trained to imitate, compress, or inherit part of the behaviour of another model, usually called the *teacher*. The

model being trained is called the *student*. In the classical form of knowledge distillation, the student model is often a smaller or cheaper model to run than the teacher, and it is trained so that its output distribution is close to the teacher’s output distribution [74, 75]. Thus, distillation is another training procedure.

A simplified mathematical description is as follows. Suppose that, for a given input x , the teacher produces logits $z^{(T)}(x) \in \mathbb{R}^m$, and the student produces logits $z^{(S)}(x) \in \mathbb{R}^m$, where m is the number of possible output classes or next-token candidates. For a temperature $\tau > 0$, define the softened distributions

$$q_i^{(T)}(x) = \frac{\exp(z_i^{(T)}(x)/\tau)}{\sum_{j=1}^m \exp(z_j^{(T)}(x)/\tau)}, \quad q_i^{(S)}(x) = \frac{\exp(z_i^{(S)}(x)/\tau)}{\sum_{j=1}^m \exp(z_j^{(S)}(x)/\tau)}.$$

A distillation loss may then penalize the discrepancy between these two distributions, for example by using a Kullback–Leibler divergence term

$$D_{\text{KL}}\left(q^{(T)}(x) \parallel q^{(S)}(x)\right) = \sum_{i=1}^m q_i^{(T)}(x) \log \frac{q_i^{(T)}(x)}{q_i^{(S)}(x)}.$$

In practice, implementations may combine such a distillation term with an ordinary supervised loss against the original labels or target tokens.

For local LLMs, the word *distilled* is sometimes used more broadly than this classical logit-level formulation. A model card may describe a model as distilled when a smaller or more convenient base model has been fine-tuned on outputs, reasoning traces, or other data generated by a stronger teacher model. For example, the DeepSeek-R1 documentation describes distilled models based on Qwen and Llama families and states that reasoning data generated by DeepSeek-R1 was used to fine-tune smaller dense models [76, 77]. Therefore, when reading a model card, one should check what *distilled* means in that particular case: logit distillation, response distillation, reasoning-trace distillation, or another post-training procedure.

A distilled model should not be confused with a quantized model. Quantization changes the numerical representation of an already trained model’s weights, for example by storing them with fewer bits. Distillation, by contrast, changes the training history of the model - the student has been trained to imitate or learn from a teacher.

For mathematical reasoning, a distilled model can be useful because it may carry some problem-solving patterns from a larger teacher while being easier to run locally. However, a distilled model can inherit errors, stylistic habits, overconfident explanations, or hidden assumptions from the teacher. It can also introduce new errors because the student has smaller capacity, a different architecture, different training data, or a different post-training procedure. Hence, a distilled model may be a useful explorer or proof-draft

generator, but it remains subject to the same previously stated rules as any other local LLM.

Typical role in mathematical work. DeepSeek-R1-style models are useful for adversarial proof exploration, i.e., ask for a proof, ask for counterexamples, then ask the model to attack its own argument. Distilled variants may be useful when the full model is too large for local hardware, but the user should remember that distillation changes behavior.

Strengths. The family is associated with explicit reasoning behavior and with a large ecosystem of derivatives and distilled models. This makes it useful for experimental comparisons between a general model, a reasoning model, and a distilled reasoning model.

Cautions. A distilled model may inherit some reasoning style from a larger model while losing robustness, domain coverage, or calibration.

What should be verified. One should verify whether the model is the original release, a later release, a distilled model, a third-party quantization, or a fine-tuned derivative. Moreover, a user should also verify the license and the base model of a distilled release.

Suitable workflow role. DeepSeek-R1-family models are natural *explorers*, *skeptics*, and *counterexample searchers*. They should not be treated as proofreaders.

4.7 Phi-4 Reasoning

Phi-4 Reasoning is a Microsoft open-weight reasoning model family entry. The model card for Phi-4-reasoning describes it as an open-weight reasoning model fine-tuned from Phi-4. It also states intended use cases involving reasoning and logic, and also records limitations such as possible fabrication of inaccurate information [30]. The exact release terms and intended-use statements should be read from the current card.

Typical role in mathematical work. Phi-4 Reasoning is a candidate for small-to-medium local reasoning tasks such as elementary analysis proofs, short proof audits, definition checking, and concise explanations. It can be useful when a very large model is impractical on available hardware.

Strengths. A smaller reasoning model might be pedagogically useful because its mistakes are often visible. The user can compare a Phi-4 Reasoning output with a larger model output and ask which proof is shorter, which proof is more explicit, and which proof actually uses the hypotheses.

Cautions. A smaller model may skip quantifier structure, omit edge cases, or produce a correct theorem with an incomplete proof. The model card’s limitation statements should be taken seriously.

What should be verified. One should verify whether the model is a reasoning variant or a general Phi model, whether the local version is official or converted, and whether the license permits the intended use. On proofs, one should identify each theorem invoked by name and check its assumptions.

Suitable workflow role. Phi-4 Reasoning can serve as an *explorer*, *editor*, or *short proof auditor*. For nontrivial proofs, it should be checked by an independent model or by a human audit.

4.8 Nemotron and OpenReasoning Nemotron

NVIDIA has released several Nemotron-related model cards, including OpenReasoning-Nemotron and OpenMath-Nemotron variants. The OpenReasoning-Nemotron card describes it as a reasoning model post-trained for math, code, and science, while OpenMath-Nemotron is more explicitly connected with mathematical reasoning [31, 32].

Typical role in mathematical work. OpenReasoning-style Nemotron models are candidates for difficult mathematical exploration, proof decomposition, and code-related reasoning. OpenMath-style variants are candidates for math-specific proof attempts and comparison with general reasoning models.

Strengths. The main strength is domain orientation. In an experiment, one can ask whether a math-oriented model handles definitions, hypotheses, and proof structure better than a general chat model on the same task. Then, the result should be measured by audit quality.

Cautions. Nemotron cards may use licensing combinations that need direct reading. Some cards state Creative Commons Attribution governing terms with additional Apache-related information, and other cards in the broader ecosystem may differ. One should not infer one license from another Nemotron name. The user should also distinguish a math-specialized model

from a general-purpose assistant, since specialization can improve some tasks and weaken others.

What should be verified. One should verify the exact Nemotron family member, base model, intended domain, governing terms, local inference path, and quantization source. For a mathematical proof, it should be tested whether the model handles definitions precisely.

Suitable workflow role. Nemotron reasoning models can serve as *explorers*, *counterexample searchers*, or *programmers* for mathematically flavored code.

4.9 gpt-oss

The GPT-oss model is OpenAI’s open-weight model family. OpenAI’s model card describes it as open-weight reasoning models, and the OpenAI help material states that these models run on infrastructure controlled by the user rather than through ChatGPT or the OpenAI API [34, 35, 36].

Typical role in mathematical work. gpt-oss models can be used as reasoning assistants in the same way as other local open-weight families. Hence, for the proof sketching, proof criticism, structured extraction of assumptions, and programming support. They may also be useful if one wants to compare local open-weight behavior against other local families.

Strengths. The primary practical strength is that the release is documented by official OpenAI model-card and support materials. The help documentation also makes clear that self-hosted use is operationally different from using an OpenAI-hosted service, including different support and data-flow assumptions [36].

Cautions. The same rules apply as for the other local LLMs. The user should also read the usage policy and the exact license conditions. Open-weight access does not make the training pipeline fully reproducible by default.

What should be verified. One should verify the exact model, release date, license, usage policy, local runtime, quantization, and hardware requirements. One should not mix claims about ChatGPT, the OpenAI API, and self-hosted gpt-oss runs unless a source explicitly connects them.

Suitable workflow role. gpt-oss can serve as an *explorer*, *skeptic*, or *editor*.

4.10 OLMo and open research models

OLMo represents an open research orientation. It cards, and project materials emphasize release of code, checkpoints, training details, and research artifact [37, 38]. Similarly to the other models discussed before, the exact claims vary by release and should be checked on the model card.

Typical role in mathematical work. OLMo-family models might be helpful for methodological comparison. So not only answering the question “Can this model solve the problem?” but also “What does a more transparent research release allow us to inspect?” They are good candidates for discussions of reproducibility, dataset documentation, model cards, and limitations.

Strengths. The superiority of the family is the openness of the research artifacts relative to many other releases. This could help to separate mathematical verification from empirical model development.

Cautions. An open research release can still hallucinate or give invalid proofs. Transparency of artifacts helps scientific audit.

What should be verified. One should verify the exact OLMo release, base/instruct/think variant, license, and intended use. Moreover, one should also distinguish reproducibility of training artifacts from reproducibility of a particular local inference run.

Suitable workflow role. OLMo-style models are suitable as *comparison models*, *editors*, and sometimes *skeptics*. They are especially useful when the problem includes documentation quality and research openness.

4.11 Gemma, Mistral, GLM

Not every useful local model family is primarily a mathematical reasoning family. Sometimes, general and multilingual models can still play useful roles in a mathematical workflow. In particular, they can be useful for translating an explanation into clearer prose, rewriting a proof in L^AT_EX, identifying ambiguous definitions, or serving as a skeptical reader.

Gemma. Google’s Gemma model cards describe Gemma as a family of open-weight models and require acceptance of Google’s usage license for file access on Hugging Face [39]. This makes Gemma a useful example of a model family that may be locally usable but is not simply described by the word “permissive”. In mathematical work, Gemma-style models can be tested as

editors, general explainers, or small-device comparison models. The user should verify the license, file access requirements, local quantization source, and whether the model is text-only or multimodal.

Mistral and Magistral. Mistral releases include general, coding, and reasoning-oriented families. In mathematical workflows, Mistral-family models can serve as general reasoners, code assistants, or independent editors. One should verify whether they are using an official GGUF card, an original safetensors release, or a third-party conversion.

GLM. GLM model cards are useful for explaining the difference between open-weight availability and practical classroom usability. The GLM-4.5 card describes large MoE releases, active-parameter counts, and substantial hardware requirements for full-featured inference [42, 43]. This illustrates a general point that a model can be open-weight and still not be a realistic choice for ordinary laptops unless a suitable local quantization and runtime exist.

4.12 Code-oriented local models

Mathematicians often use local models not only for prose proofs, but also for mathematical programming. A code-oriented model may help with Python experiments, Sage computations, simple Lean syntax, unit tests, or the conversion of a handwritten calculation into executable form. Examples of code-oriented model cards include Qwen3-Coder, DeepSeek-Coder-V2, Devstral, and OpenCodeReasoning-Nemotron [26, 29, 41, 33].

The role of a code model differs from that of a proof model. When the output is Python or Sage, the first check is execution. When the output is Lean, the first check is compilation. When the output is a proof explanation written in natural language, execution and compilation are usually unavailable, so the human user must perform a mathematical audit.

Typical role in mathematical work. Code models are suitable for writing test scripts, generating small examples, checking numerical evidence, translating a recurrence into code, or producing a first Lean skeleton for an elementary theorem.

Strengths. The strength of a code model is that many of its outputs can be tested mechanically/numerically. A wrong Python function may fail a unit test. A wrong Lean proof may fail to compile. This gives a clearer feedback loop than natural-language proof prose.

Cautions. Executable code can still answer the wrong mathematical question, and simulation can support an intuition without proving a theorem. A Lean snippet that compiles may formalize a weaker or different statement than the intended theorem. Finally, a calculation may hide assumptions about numeric precision.

What should be verified. One should verify the statement encoded in the program, the test cases, the symbolic assumptions, the numeric precision, and the distinction between evidence and proof. For Lean, one should verify that the theorem statement is the intended theorem, not merely that some code compiles.

Suitable workflow role. Code-oriented models are *programmers* and sometimes *formalizers*. They should be paired with a mathematical review and, when possible, with automated tests.

4.13 Small, medium, and large local models

Small models are easier to run, easier to compare in class, and often fast enough for iterative prompting. Medium models may provide a better balance between mathematical fluency and practicality. Large models may produce more elaborate proof sketches on some tasks, but they can be slow, memory-intensive, and harder to deploy consistently on laptops. A larger model may be better at maintaining a multi-step plan, but it may also produce a longer invalid proof. A smaller model may fail more visibly, which is useful pedagogically. A medium code model may be more useful than a larger general model for a Python verification task. A math-specialized model may outperform a general model of similar size on a narrow proof task, but may be weaker as a general editor.

Summarizing:

Small local model. Useful for quick classroom demonstrations, short prompts, definitions, short rewrites, and visible failure cases. It is rarely enough for final proof auditing.

Medium local model. Useful for proof sketches, counterexample search, and moderate mathematical programming. It may be the main working model on a capable laptop.

Large local model. Useful for harder exploration, long audits, and comparison against smaller models, if hardware permits. It should be used deliberately because longer outputs can hide more errors.

Quantization complicates these categories. A quantized larger model may fit into memory where an unquantized smaller model does not. A 32B

Q4 model may outperform a 14B Q8 model on some mathematical tasks because architecture, training, specialization, and scale interact. But this is a possibility, and it must be checked empirically on the intended tasks and then audited mathematically.

4.14 Reproducibility checklist

Before a model is used in lecture notes, a classroom exercise, or a reported experiment, we propose to record the following:

1. **Exact model source.** Record the organization, repository name, and model-card URL.
2. **Exact artifact.** Record whether the run used original weights, an official GGUF, or a third-party quantization.
3. **License.** Record the license named on the model card and link to the license text or governing terms.
4. **Access restrictions.** Record whether a click-through license, login, gated access, regional restriction, or usage policy applies.
5. **Runtime.** Record the tool used, such as Ollama, llama.cpp, LM Studio, MLX, vLLM, Transformers, or another runtime.
6. **Version information.** Record the runtime version, operating system, and, where possible, the model digest or file checksum.
7. **Inference parameters.** Record context length, temperature, seed if supported, maximum output length, and other non-default parameters.
8. **Hardware.** Record CPU, RAM, GPU, VRAM, and offloading settings if relevant.
9. **Prompt and output.** Save the exact prompt and the exact model output. Do not paraphrase a failed proof when auditing it.
10. **Independent verification.** Record the human audit, computational test, second-model critique, or formal proof-system check used after generation.

4.15 Model roles in a mathematical workflow

A local model should be assigned a role, since a model without a role tends to become an unexamined authority. The following table gives practical roles for local models in a mathematical workflow.

Role	Useful for	Required check
Explorer	Generating proof strategies, examples, related lemmas, and possible reductions.	Select a viable path manually. No theorem is accepted merely because it was suggested.
Skeptic	Attacking a candidate proof and searching for hidden assumptions, missing cases, or invalid implications.	Decide whether each criticism is mathematically real and whether it actually affects the stated theorem.
Counter-example searcher	Testing whether a proposed statement may be false by varying assumptions and edge cases.	Verify directly that the proposed example satisfies all hypotheses and violates the conclusion.
Formalizer	Translating a simple theorem into Lean or another proof assistant, or drafting a formal statement.	Compile the file and check that the formal statement matches the intended informal theorem.
Programmer	Writing Python, Sage, or shell scripts for numerical, symbolic, or finite experimentation.	Run the code, inspect tests and precision, and separate computation from proof.
Editor	Rewriting text, improving notation, converting prose into L ^A T _E X, or shortening an explanation.	Check after editing that assumptions, quantifiers, definitions, and conclusions were not changed.

Remark 4.1. *For complicated mathematical proofs, do not use the same model as author, referee, and final editor.*

Chapter 5

Choosing Models for Mathematical Work

The guiding rule is simple, i.e., choose a model for a specific role. A local model may help explore a proof, find a counterexample, write a short Python script, translate a statement into $\text{\LaTeX}$, or critique a draft. None of these roles turns the model into a proof checker.

Exact tags, local quantizations, licenses, context windows, and hardware behavior are fast-changing and should be verified from current model cards, local runtime documentation, and the actual machine used. Model-card documentation is a source for provenance, intended use, limitations, and licensing fields [18].

5.1 Task-based model selection

A useful task description has four main components:

1. **Mathematical object.** Is the input a definition, theorem, conjecture, false claim, proof attempt, program, plot, or $\text{\LaTeX}$ draft?
2. **Expected model role.** Should the model explain, explore, search for counterexamples, audit, formalize, program, or edit?
3. **Verification method.** How will the output be checked: direct proof reading, numerical test, symbolic computation, compilation, unit tests, or a formal proof assistant?
4. **Local constraints.** What are the available RAM, VRAM, disk space, context length, and time budget?

The fourth component is practical, but it should not dominate the first three. A model that fits easily into memory but cannot maintain the hypotheses of the theorem is not a good model for that theorem. Conversely, a

larger model that produces fluent but unaudited text is not mathematically safer merely because it is larger.

The following table gives a first task-to-model map. It is intentionally expressed in model types and workflow roles rather than exact names. Exact local candidates should be selected only after checking current model cards, available quantizations, licenses, local runtime support, etc.

Task	Usually suitable model type and use	Required verification
Explanation	Small or medium instruction model, but sometimes even a small reasoning model is enough. Useful for restating definitions, giving examples, separating assumptions from conclusions, and adapting exposition to a suitable level.	Check that definitions are not simplified incorrectly and that examples satisfy the stated assumptions.
Proof search	Medium or larger reasoning model, math-oriented model when available. Useful for proposing lemmas, possible strategies, and reductions, and for comparing direct proof, contradiction, induction, or counterexample search.	Every lemma and implication must be proved independently - the generated proof is only a candidate.
Counter-example search	Reasoning model or general instruction model with an explicit adversarial prompt. Useful for varying hypotheses, testing boundary cases, etc.	Verify directly that the proposed example satisfies all hypotheses and violates the conclusion.
Proof audit	Independent reasoning model, preferably not the model that wrote the proof. Useful for locating hidden assumptions, missing cases, invalid theorem use, and quantifier errors.	Decide manually whether each criticism is mathematically valid, since false alarms are common.
Python/SageMath coding	Code-oriented model or strong instruction model. Useful for scripts for numerical checks, symbolic computations, finite searches, plotting, or simple experiments.	Run the code, inspect tests, numerical precision, assumptions, and failure cases.
Lean assistance	Code-oriented model or model tested on Lean-like syntax. Useful for drafting theorem statements, imports, and proof skeletons.	Compile the file and check that the formal statement matches the intended theorem.
LaTeX exposition	Small or medium instruction model in an editor role. Useful for rewriting prose, formatting theorem environments, normalizing notation, and improving readability.	Check that assumptions, quantifiers, symbols, and conclusions were not altered.
Final editing	Ordinary instruction model, preferably after the proof is already verified. Useful for grammar, order, notation consistency, and cross-references.	Do not allow the model to change mathematical content unless each change is audited.

5.2 Explanation, proof search, proof audit, coding, and editing

The most common error in model selection is to merge several distinct activities into one conversation. For example, the user may ask for an explanation, receive a proof sketch, then ask for a correction, ask for L^AT_EX, and then finally treat the polished result as a proof. This is a rather bad experimental design. The stages of explanation, proof search, proof audit, coding, and editing should be separated.

Explanation

An explanation task asks the model to clarify meaning, but not to establish a theorem. Suitable prompts include:

Explain the difference between a local minimizer
and a global minimizer. Give three examples on $\mathbb{R}$.
Do not prove any new theorem.

For such a role, a smaller model may be sufficient. The main risk is oversimplification. For example, a model may explain that a critical point is *where the minimum occurs*, which is false without additional assumptions.

Proof search

A proof-search task asks the model to propose possible routes, but it is not yet a request for a finished proof. An exemplary prompt might be:

We want to prove the following statement.
[PUT THE STATEMENT HERE]
List possible proof strategies, but do not write
an attempted final proof yet.
For each strategy, state the key lemma needed
and the hypotheses on which it depends.

This role often benefits from a reasoning model or a mathematically oriented model. The desired output is often a menu of possible approaches. The user then chooses one approach and verifies whether the required lemmas are true.

Proof audit

A proof audit can be seen as an adversarial reading. The model should be asked to identify possible failure points, but not to make the proof sound better. A good audit prompt is quite specific:

Audit the proof below. Do not rewrite it.
 Number every step that uses a hypothesis.
 Identify hidden assumptions, invalid implications,
 missing cases, and named theorems used without
 their hypotheses.
 [STATE THE PROOF HERE]

The audit model should ideally be independent from the model that generated the proof. Namely, use a different family, different tuning, or at least a separate run with an explicitly skeptical prompt. Agreement between two models is still not a proof, however, disagreement is often useful because it exposes places for human inspection.

Coding

Coding tasks include Python checks, SageMath experiments, simple search procedures, and plotting. A code-oriented model may be more useful than a larger general reasoning model, because syntax, library habits, and test scaffolding matter. For Python work, the documentation of Python and its tools remains the source of syntax and behavior [49]. For SageMath, the Sage documentation is the appropriate reference for supported mathematical computation [51].

It is important to remember that the executable computation can refute many false statements and suggest many conjectures, but it usually does not prove a universal theorem. A plot of

$$f_a(x) = x^2 + a \sin^2 x$$

may suggest a convexity threshold or a nonconvex example. It is not a proof of either statement. So, the computational output must be followed by exact derivative analysis and quantifier control.

Editing

Editing is the safest role only when the mathematical content has already been checked. A model can improve sentence structure, remove repetition, and convert a proof into simple L^AT_EX. However, it can also accidentally change a quantifier, remove a necessary condition, or replace with "there exists" by "for all". For this reason, final editing should be performed on a verified proof, and the edited version should be compared line by line with the verified mathematical content.

5.3 Small fast models versus larger reasoning models

Local model choice is often constrained by the available user’s hardware. A small model may be the only model that runs comfortably on a user’s laptop. But small models are often useful for short explanations, notation cleanup, simple examples, prompt demonstrations, and visible failure cases.

The good information is that a small model might often be sufficient when the task has one or more of the following properties:

- the mathematical content is definitional or elementary;
- the output is short and easy to audit;
- the student already knows the proof and wants an alternative explanation;
- the task is formatting rather than discovery;
- the goal is to demonstrate why verification is necessary.

A larger reasoning model may help when the task requires maintaining a longer chain of dependencies. Examples include decomposing a proof into lemmas, searching for a counterexample after several failed attempts, auditing a proof with several cases, or comparing alternative strategies. The word "may" is important, since larger models can produce longer, more persuasive, but still incorrect arguments.

The relevant trade-off for mathematics is:

How much useful mathematical structure does the model produce per unit of human audit effort?

Small and medium models have an additional advantage. Namely, they allow repeated experiments. One can run the same prompt under different temperatures, quantizations, or prompt formulations and see how outputs change. This is useful for understanding local models as experimental instruments.

5.4 Quantization versus mathematical reliability

Quantization affects how model weights are stored and used during inference. Chapter 3 described common GGUF labels such as FP16, Q8_0, Q6_K, Q5_K_M, and Q4_K_M. Hugging Face and llama.cpp documentation give technical context for GGUF and quantization workflows [15, 20]. *It is good to remember that for model selection, a quantization label is not a mathematical reliability label.* Higher precision may improve stability on some tasks, reduce some approximation artifacts, or preserve more of the base model’s behavior.

But it may also require more memory and run more slowly. Lower precision may make a larger model practical on local hardware. Neither choice makes the model a proof checker.

Below are two common mistakes that should be avoided:

1. **Mistake 1: “Q8 is twice as intelligent as Q4.”** This is false. The label concerns numerical representation, but not intelligence. It does not express a linear scale of mathematical ability.
2. **Mistake 2: “The highest precision that fits is always the best choice.”** This is not a rigorously proven theorem. A higher-precision smaller model may be less useful than a lower-precision larger or more specialized model on a particular proof-search task. For example, a 32B model in a usable Q4 quantization may outperform a 14B model in Q8 on some mathematical prompts. And this is not because Q4 is better than Q8, but because base-model capability, training, architecture, and specialization may dominate the storage-precision difference. The result depends on architecture, training, tuning, prompt, inference implementation, and the mathematical problem.

We use the following practical selection rule:

Use a quantization that makes the model responsive enough for the task, then evaluate the mathematical output by audit, but not by the quantization label.

When recording experiments, write down the exact quantization label, the source of the GGUF or local artifact, and the local runtime. Ollama’s model-inspection interfaces and API examples show metadata such as format, family, parameter size, and quantization level for a local model [13]. These fields are useful for reproducibility.

5.5 When to use several models

A multi-model workflow is useful when the task has enough mathematical risk that self-auditing by the same model is weak evidence.

Use several models when:

- the claim is false or suspected to be false, and counterexample search is important,
- the proof uses several named theorems or nontrivial quantifier structure,
- one model produced a fluent proof too quickly and without exposing dependencies,

- the task involves both mathematics and code,
- the final output will be used in teaching material, a report, or a manuscript draft,
- the result would be costly to repair if an error is discovered late.

A useful division of labor is:

1. **Explorer.** Generates possible proof strategies or examples.
2. **Skeptic.** Attacks the selected strategy and searches for hidden assumptions.
3. **Counterexample searcher.** Tries to falsify the statement or its intermediate lemmas.
4. **Programmer.** Writes code for finite checks, symbolic experiments, or plots.
5. **Editor.** Rewrites the already verified result in clean prose or $\text{\LaTeX}$.

The same local model should not occupy all roles in a nontrivial task. A model that wrote a proof is often poor at refuting its own hidden assumptions, especially when the audit prompt is vague. A separate skeptical model may still miss the error, but it increases the chance that a different path through the problem is explored.

When models disagree, do not choose the answer that sounds more confident. Convert the disagreement into a mathematical subproblem. For example:

- Which statement has the correct quantifiers?
- Which proof step uses suitable assumptions like compactness, completeness, convexity, differentiability, existence of a minimizer, etc.?
- Is there a counterexample if one assumption is removed?
- Can the disputed lemma be proved independently?
- Can a finite or one-dimensional example decide the issue?

A multi-model workflow is successful when it improves the human audit.

5.6 When not to use an LLM

There are situations in which a local LLM is the wrong instrument. Refusing to use the model is sometimes the mathematically responsible decision.

Do not use an LLM as the primary tool when:

- the task is to decide the final correctness of a proof,
- the user has not yet written down the precise statement and assumptions,
- the result is high-stakes and there is no independent verification plan,
- the output would be submitted as the user's own proof without disclosure or understanding,
- the problem is mainly about exact computation, and a deterministic symbolic or numerical tool is available,
- the task is to produce citations, licenses, model tags, hardware requirements, or benchmark claims without current source checking,
- the conversation would include confidential material that should not be entered into that particular local interface, log file, or shared machine,
- the model's response would replace reading the relevant textbook, paper, documentation, or proof assistant error message.

The most important case is proof correctness. A model can suggest a proof, but cannot certify the proof by just saying that it is correct. Even a statement such as "I have checked every step" is only generated text. The human user must still inspect the argument.

Remark 5.1. *There is also a pedagogical reason not to use an LLM too early. If the task is designed to teach a particular idea, the student should first attempt the proof independently. The model can then be used as a second reader, source of alternative exposition, or generator of flawed proofs to audit.*

5.7 Recommended minimal local setup

A minimal role-based setup is:

1. **One small reasoning or instruction model.** Used for quick explanations, short examples, and prompt demonstrations.
2. **One medium reasoning model.** Used for proof search, longer mathematical explanations, and candidate proof generation.

3. **One independent audit model.** Used as a skeptic. Prefer a different model family or substantially different tuning from the main proof-search model.
4. **One code-oriented model.** Used for Python, SageMath, shell commands, and possibly Lean-adjacent syntax assistance.
5. **One ordinary instruction model for comparison.** Used to show the difference between general instruction following and reasoning-oriented behavior.

For each selected model, we recommend to record:

- exact source and model card,
- exact local artifact or Ollama identifier,
- license and any access restrictions,
- quantization label and source,
- context setting and relevant parameters,
- local runtime version,
- hardware used for the test,
- three or four representative mathematical prompts,
- known failure cases observed during preparation.

5.8 A model comparison worksheet

A comparison worksheet is a structured record of a local mathematical experiment. Its purpose is to make student observations reproducible and auditable.

The worksheet below can be copied for each model and each task. The field “correct conclusion” should be filled by the human reviewer after checking the mathematics, but not copied from the model output. Furthermore, the field “final assessment” should be referred to the model’s usefulness for the specified task.

Field	Entry
Model name	Exact local name, identifier, or repository used.
Family	General family or source organization, if known from the model card.

Field	Entry
Quantization	Exact label, for example Q4_K_M, Q5_K_M, Q8_0, or FP16. Record only what is reported by the source or local tool.
Size on disk	Local file size or installed model size, measured on the machine used.
Context length	Configured or reported context length. Record how it was set or observed, because context handling is runtime-dependent [3].
Prompt	Exact prompt used. Do not paraphrase.
Task type	Explanation, proof search, counterexample search, proof audit, coding, Lean assistance, LaTeX exposition, or editing.
Correct conclusion	The mathematically correct conclusion, supplied after human verification.
Main error	The most important mathematical or operational error in the model output, if any.
Useful insight	A useful lemma, example, reformulation, code fragment, or warning produced by the model, if any.
Final assessment	Suitability for this task under this prompt and setup: useful, partially useful, misleading, failed, or needs more testing.

We advise using the same prompt across several models, however, do not force the same conclusion. The point is to observe how different models fail and succeed. A good worksheet often contains mixed results like: a model may give the correct conclusion but a weak proof, a wrong conclusion but a useful counterexample idea, or clean $\text{\LaTeX}$ around a mathematically invalid argument.

A final comparison should answer four questions:

1. Did the model preserve the exact assumptions?
2. Did the model reach the correct conclusion?
3. Was the reasoning auditable by a student?
4. Did the model reduce or increase the amount of human verification work?

Remark 5.2. *The last question is often the most important. A local model is considered to be useful for mathematical work when it improves disciplined exploration and verification.*

Chapter 6

Prompting Principles for Mathematics

In mathematical work, A prompt for a local language model is closer to a short experimental protocol. It states the object under investigation, fixes the assumptions, specifies the role assigned to the model, and describes the form in which the answer should be returned. A good prompt makes the model's output easier to inspect, reject, repair, or reuse.

In this chapter, we give practical prompting principles for mathematicians. The model should not be invited to guess the problem you meant to ask. It should be given a carefully delimited mathematical task and a clear instruction to expose its assumptions and weak points. This is especially important because language models can produce fluent but false mathematical text. Empirical work on mathematical reasoning and factuality documents that correctness cannot be inferred from fluency alone [64, 63, 65].

For a compact copy-ready version of the main prompt templates, see Appendix C.

6.1 Anatomy of a mathematical prompt

A mathematical prompt should address the following six questions before the local reasoning model starts generating text:

1. **What is the mathematical object?** Is it a theorem, conjecture, definition, proof attempt, counterexample candidate, numerical experiment, code fragment, or L^AT_EX draft?
2. **What role should the model play?** Explorer, skeptic, counterexample searcher, proof auditor, programmer, formalization assistant, or editor?
3. **What assumptions are fixed?** Domain, regularity, boundedness,

topology, algebraic structure, measure-theoretic setting, etc., all conventions should be stated explicitly.

4. **What definitions must be used?** If a definition is important, include it verbatim. Do not rely on the model's preferred convention.
5. **What is forbidden?** For example: do not assume compactness, do not invoke differentiability if only continuity is given, do not use numerical evidence as proof, do not cite unnamed theorems.
6. **What output format is wanted?** Strategy list, candidate proof, audit table, counterexample checklist, $\text{\LaTeX}$ source, Python script, or uncertainty report.

A compact general template is the following:

You are assisting with a mathematics problem.
 Role: [explorer / skeptic / proof auditor / programmer / editor].
 Statement: [exact statement].
 Domain: [space, field, topology, order, probability space, etc.].
 Assumptions: [numbered list].
 Definitions to use verbatim: [definitions].
 Forbidden shortcuts: [forbidden assumptions or theorems].
 Task: [what the model should do].
 Output format: [strategy list / proof / audit table / code / $\text{\LaTeX}$].
 Verification requirement: mark every step that needs human checking.

The last is very important, since a local model will often write as if a step is settled when it is only plausible. Asking for explicit verification points makes the output more useful for study. It also discourages the common pattern in which the model writes a smooth proof and hides the only difficult step in one sentence.

Remark 6.1. *When an output is used in a proof attempt, exercise, project, or manuscript draft, record the prompt, model identifier, local settings, and date. This improves reproducibility.*

6.2 Stating assumptions precisely

Many failures in model-assisted mathematics are failures of problem specification. The model silently strengthens an assumption, changes the domain, assumes the existence of a minimizer, replaces a local conclusion by a global one, or treats a numerical pattern as exact. For a mathematician, the first discipline is therefore to separate the statement into assumptions and conclusions. Avoid prompts such as:

Prove the theorem about critical points and minima.

The prompt does not say what class of functions is being considered, what the domain is, whether the function is convex, whether the critical point exists, or what kind of minimum is meant. A better prompt is:

Investigate the following statement.
 Let $f: \mathbb{R} \rightarrow \mathbb{R}$ be differentiable. If $f'(x_0)=0$, then x_0 is a global minimizer of f .
 Do not assume convexity unless it is explicitly stated.
 Either prove the statement or find a counterexample.
 Verify every hypothesis for any proposed counterexample.

The second prompt does not force the model toward a proof. It also blocks a common hidden shortcut, namely, adding convexity without permission.

A useful assumption block has the following form:

Assumptions:

1. The domain is exactly [state domain].
2. The codomain is exactly [state codomain].
3. The regularity assumptions are exactly [state regularity].
4. The relevant constants satisfy [state inequalities].
5. No compactness, convexity, boundedness, or existence of a minimizer may be assumed unless it appears above.
6. If you use a theorem, state all hypotheses of that theorem and check them against the assumptions above.

A simple final instruction is effective:

Before attempting the proof, rewrite the assumptions and the desired conclusion as a numbered list. Do not add any assumption that is not present in the problem statement.

This will not prevent every error, but it gives the user a first object to audit.

6.3 Stating definitions explicitly

A definition block can be written as follows:

Definitions that must be used verbatim:

1. A sequence (x_n) of real numbers converges to L if for every $\epsilon > 0$ there exists N such that for all $n \geq N$, $|x_n - L| < \epsilon$.
2. A function $f: \mathbb{R}^d \rightarrow \mathbb{R}$ is convex if for all x, y and all t in $[0, 1]$, $f(tx + (1-t)y) \leq t f(x) + (1-t) f(y)$.
3. A point x^* is a global minimizer of f on D if $x^* \in D$ and $f(x^*) \leq f(x)$ for every x in D .

Then give the task.

Use only the definitions above unless you explicitly state and justify an additional theorem. If your proof uses a theorem, first state the theorem and list its hypotheses.

Definitions should also be stated when asking for an audit. Otherwise the model may audit the proof against its own conventions rather than yours.

Audit the proof below using exactly these definitions. Do not replace these definitions by equivalent-looking versions unless you prove the equivalence under the stated assumptions.

In reasoning model-assisted work, the risk is larger because the model may replace a precise course definition with a common but incompatible paraphrase.

6.4 Asking for proof or counterexample

When the truth of a statement is not known, do not ask only for proof. That prompt biases the conversation toward confirmation. A better instruction is to ask for both possibilities.

Investigate the statement below. Do not assume it is true.

Try both directions:

A. Search for a proof.

B. Search for a counterexample.

If you find a proof, identify the key assumptions used.

If you find a counterexample, verify every hypothesis and show exactly where the conclusion fails.

If neither is complete, report the obstruction honestly.

The following “problem card” is a reusable template for mathematical exploration.

Problem card

1. Statement to investigate:

[Write the exact statement.]

2. Domain:

[State the domain, codomain, topology, algebraic structure, probability space, etc.]

3. Exact assumptions:

[List all assumptions. Do not include desired conclusions.]

4. Desired conclusion:

[State exactly what should be proved or refuted.]

5. Known related facts:

- [List facts that may be relevant, with hypotheses.]
6. Definitions that must be used verbatim:
[Paste the definitions.]
 7. Forbidden shortcuts:
[List assumptions or theorems that must not be used without proof.]
 8. Try both proof and counterexample:
[Ask for a proof attempt and an independent counterexample search.]

For example:

Problem card

1. Statement to investigate:
Every convergent sequence of real numbers is bounded.
2. Domain:
Sequences (x_n) with x_n in $\mathbb{R}$ for every n .
3. Exact assumptions:
There exists L in $\mathbb{R}$ such that $x_n \rightarrow L$.
4. Desired conclusion:
There exists $M > 0$ such that $|x_n| \leq M$ for every n .
5. Known related facts:
Triangle inequality in $\mathbb{R}$.
6. Definitions that must be used verbatim:
Use the epsilon- N definition of convergence.
7. Forbidden shortcuts:
Do not say “convergent sequences are bounded” as a known theorem.
8. Try both proof and counterexample:
First try to prove it. Then check whether a counterexample is possible under the stated assumptions.

When the statement has already survived preliminary counterexample search, a narrower proof-generation prompt can be used.

Write a candidate proof of the statement below.

Use only the listed assumptions and definitions.

Number the proof steps.

After each nontrivial step, write which assumption, definition, or previous step justifies it.

If a step is plausible but not fully justified, mark it as [GAP] instead of hiding the gap.

End with a list of all places where human verification is still needed.

Statement:

[paste statement]

Assumptions:

[paste assumptions]

Definitions:

[paste definitions]

6.5 Asking for strategy before proof

Almost always asking immediately for complete proof of a complicated task is a bad idea. The model may produce a long argument before the user knows whether the strategy is appropriate. The following strategy-first prompt separates exploration from proof writing:

Do not write a full proof yet.

Give three possible proof strategies for the statement below.

For each strategy, provide:

1. the main idea;
2. the key lemma needed;
3. the assumptions used;
4. the likely failure point;
5. whether the strategy could also reveal a counterexample.

After listing the strategies, recommend one strategy to try first and explain why.

Mostly, a strategy is shorter than a proof and easier to audit. Hence, one can reject an invalid strategy before investing effort in a polished but wrong proof.

For a problem involving definitions, use a dependency map as follows:

Create a dependency map before proving anything.

Rows: proof step, statement needed, assumptions used, theorem used, possible gap.

Do not write the final proof until the dependency map is complete.

A useful follow-up prompt is:

For the strategy you recommended, identify the first nontrivial lemma. Prove only that lemma, or explain why it may be false under the stated assumptions.

This keeps the interaction local. Instead of producing a two-page proof, the model must confront one mathematical subproblem.

6.6 Asking for proof audit

A proof audit is not the same as asking “Is this proof correct?”. A proof audit should require line-by-line analysis.

An audit prompt might be like this:

Audit the proof below. Do not rewrite it first.

Use the following format:

1. Restate the theorem exactly.
2. List all assumptions.
3. Number the proof steps.
4. For each step, state whether it follows from:
 - (a) a definition,
 - (b) a previous step,
 - (c) a named theorem,
 - (d) an unstated assumption,
 - (e) an invalid implication.
5. For every named theorem, state all its hypotheses and check them.
6. Identify missing cases and quantifier changes.
7. Give a final verdict: correct, correct with minor exposition changes, incomplete, or false.
8. If incomplete or false, identify the first serious error.

For local models, the instruction “identify the first serious error” is often more useful than asking for a complete repair. Once the first serious error is found, later steps may be irrelevant.

An audit report can be requested in table form:

Return the audit as a table with columns:

Step number	Claim in the proof	Justification given	Required justification	Status	Comment
-------------	--------------------	---------------------	------------------------	--------	---------

Use status labels: OK, unclear, missing hypothesis, invalid, not checked.

At the end, the human must still audit the audit. The purpose of the model audit is to generate a checklist for human reading, but not to replace human reading. When possible, use an independent model for the audit. However, do not use the same model as the author, referee, and final editor for a nontrivial proof. As always, this rule does not guarantee correctness, but it merely reduces one avoidable source of circularity.

6.7 Asking for repair of a flawed proof

After a proof audit identifies a gap, the next task is repair. The repair prompt must require the model to isolate the error before proposing a fix.

A repair prompt should contain the original statement, the flawed proof, and the audit finding:

Repair task.

Original statement:

[paste statement]

Assumptions:
[paste assumptions]

Flawed proof:
[paste proof]

Known audit finding:
[paste the suspected error or gap]

Task:

1. Explain the error in one paragraph.
2. Decide whether the original statement is still true.
3. If the statement is true, give a repaired proof and mark the new step that replaces the flawed step.
4. If the statement is false, give a counterexample and verify all assumptions.
5. If the statement becomes true only after adding assumptions, state the weakest additional assumptions you can justify.
6. Do not hide the gap by changing notation or wording.

For a suspected false theorem, add an adversarial instruction:

Before repairing the proof, try to falsify the theorem under the exact assumptions. Search for one-dimensional, finite, boundary, and degenerate examples. Only attempt a repaired proof if the counterexample search does not succeed.

A repair may have the following outcomes:

1. **Local repair.** The theorem is true and the proof only needs a missing justification.
2. **Structural repair.** The theorem is true, but the proof strategy must change.
3. **Statement repair.** The original statement is false, but a corrected theorem is true under stronger or different hypotheses.
4. **Rejection.** The statement is false and should not be repaired into a theorem without changing its content.

Concerning the fourth outcome, a model-assisted workflow should make it easy to abandon a false claim.

6.8 Asking for LaTeX exposition

A safe L^AT_EX rewriting prompt is:

Rewrite the text below as simple LaTeX.
 Do not change the mathematical content.
 Do not add assumptions.
 Do not strengthen or weaken the conclusion.
 Preserve all quantifiers and constants.
 Use only standard LaTeX, amsmath, amssymb, and theorem environments.
 Do not use minted or packages requiring shell escape.
 If a sentence is mathematically unclear, mark it with:
 [citation needed: mathematical clarification needed]
 Do not silently repair unclear mathematics.

For a theorem-proof block, use a stricter format like this:

Convert the following verified proof into LaTeX.
 Output only LaTeX source.
 Use this structure:

$$\begin{proposition}$$

 [statement]

$$\end{proposition}$$

$$\begin{proof}$$

 [proof]

$$\end{proof}$$

 Do not add new lemmas unless they are already present in the text.
 Do not replace definitions by paraphrases.

For manuscript editing, request a change log:

Edit the exposition for clarity, but return two parts:
 Part A: the edited LaTeX.
 Part B: a change log listing every mathematical phrase that was changed.
 If you changed a quantifier, assumption, constant, domain, or conclusion,
 mark it as a mathematical change requiring human approval.

6.9 Asking for computational experiments

A prompt for computation should state what is being tested and what would count as evidence against the statement.

A general computational experiment prompt is as follows:

Design a computational experiment for the following mathematical claim:

[PUT THE CLAIM HERE]

Do not present the experiment as a proof.
 Return:

1. the mathematical claim being tested;
2. the finite or numerical proxy being used;
3. assumptions that are not captured by the computation;
4. Python code with comments;
5. tests or sanity checks;
6. possible numerical failure modes;
7. what outcome would suggest a counterexample;
8. what outcome would still not prove the theorem.

For Python code, request reproducibility details:

Write Python code for the experiment.
 Use only standard libraries unless a library is explicitly requested.
 Set a random seed if randomness is used.
 Print all parameter values used in the run.
 Separate the mathematical claim from the numerical test.
 Include assertions for simple cases where the expected answer is known.

For symbolic or exact computation, request exactness conditions:

If you use symbolic computation, state which operations are exact and which are numerical approximations. Do not infer a universal theorem from a finite sample unless you also provide a proof that the finite sample covers all cases.

The output of a computational model should be stored with the prompt, code, environment, and seed when relevant. Without reproducibility the numerical evidence might be difficult to discuss.

6.10 Asking for uncertainty and weak points

Asking the model to say whether it is "sure" is usually less useful than asking it to identify specific weak points. We aim to transform vague uncertainty into checkable mathematical tasks.

An uncertainty prompt may look like this:

Analyze your previous answer for weak points.
 Do not simply say whether you are confident.
 Return:

1. the three most fragile steps;
2. any hidden assumptions you may have used;
3. any theorem whose hypotheses were not checked;
4. any case split that may be incomplete;
5. any definition that may have been used with a different convention;
6. possible counterexamples or boundary cases;
7. the shortest independent check a human should perform first.

For proof, ask for dependency pressure points:

Mark every sentence in the proof as one of:

definition, algebra, previous step, named theorem, intuition, or gap.
 For every sentence marked named theorem, state the theorem and all its hypotheses. For every sentence marked intuition or gap, explain what would be needed to turn it into a proof.

For an answer involving a parameter, ask for boundary analysis:

The answer depends on a parameter. Check the boundary cases separately. Identify whether the inequalities are strict or non-strict, and explain what happens at equality. Do not infer the boundary case from nearby numerical examples.

The model may still miss the true weak point. Nevertheless, this prompt often produces a useful audit agenda.

6.11 Bad prompts and improved prompts

The following exemplary comparisons illustrate the difference between vague prompting and mathematically controlled prompting.

Bad prompt	Improved prompt	Why the second is better
Prove this theorem.	Investigate the statement below. Do not assume it is true. Try both proof and counterexample. Verify every hypothesis of any counterexample.	Removes confirmation bias and permits falsification.
Show that every critical point is a minimum.	Let $f : \mathbb{R} \rightarrow \mathbb{R}$ be differentiable. Investigate whether $f'(x_0) = 0$ implies that x_0 is a global minimizer. Do not assume convexity.	Fixes the domain and blocks the hidden convexity assumption.
Is my proof correct?	Audit the proof line by line. For each step, identify the definition, previous step, theorem, or hidden assumption used. State the first serious error, if any.	Replaces a vague judgment by a checkable audit.
Fix the proof.	First identify the exact gap. Then decide whether the statement is true, false, or true only with extra assumptions. Repair only after this diagnosis.	Prevents cosmetic rewriting around a false step.

Bad prompt	Improved prompt	Why the second is better
Write this in LaTeX.	Convert the verified proof into simple $\text{\LaTeX}$. Do not change mathematical content, quantifiers, constants, assumptions, or conclusion. Provide a change log for any mathematical phrase changed.	Separates formatting from mathematical revision.
Test this numerically.	Design a computational experiment. State the mathematical claim, the finite or numerical proxy, code, sanity checks, failure modes, and what the experiment cannot prove.	Keeps numerical evidence distinct from proof.
Are you sure?	Identify the three most fragile steps, any unchecked theorem hypotheses, possible boundary cases, and the shortest independent check a human should perform.	Converts confidence language into verifiable tasks.

Here is a prompt that combines several principles in one reusable form:

You are assisting with a mathematics problem. Treat your answer as a candidate, not as an authority.

Problem:

[paste exact statement]

Definitions to use verbatim:

[paste definitions]

Assumptions:

[paste assumptions]

Forbidden shortcuts:

- Do not assume compactness, convexity, differentiability, boundedness, or existence of a minimizer unless explicitly stated.
- Do not use numerical evidence as proof.
- Do not cite a theorem without stating and checking its hypotheses.

Task:

1. Restate the assumptions and conclusion.
2. Try to find a counterexample.
3. If no counterexample is found, propose a proof strategy.
4. Prove the key lemma needed for that strategy.

5. Mark every step that requires human verification.
6. End with a list of weak points and boundary cases.

For a finished proof draft, use a different prompt.

Audit the proof below as an adversarial reader.
 Do not improve the writing until the audit is finished.
 Return an audit table and a final verdict.
 If the proof is incomplete, identify the first serious error and stop.
 If the proof is correct, list the assumptions used in each nontrivial step and then suggest only exposition improvements.

For a final L^AT_EX handoff, use a narrower prompt.

The mathematical proof has already been verified.
 Rewrite it as clean L^AT_EX source.
 Do not change the statement, assumptions, constants, definitions, quantifiers, or conclusion.
 Use simple L^AT_EX only.
 After the L^AT_EX source, provide a change log of wording changes.

A common failure mode is to merge the above stages into one chat and accept the last polished answer.

Checklist before sending a mathematical prompt

1. Have I stated the exact mathematical object?
2. Have I separated assumptions from the conclusion?
3. Have I specified the domain and the conventions?
4. Have I included definitions that may be ambiguous?
5. Have I told the model whether to prove, refute, audit, repair, compute, or edit?
6. Have I forbidden common invalid shortcuts?
7. Have I requested an output format that I can audit?
8. Have I planned how to verify the output?

If the answer to the last question is no, the prompt is premature. The user should first decide how the output will be checked.

Chapter 7

Mathematical Workflows with Several Models

This chapter describes workflows with several models. The aim of using several models is to produce drafts, objections, counterexamples, computations, and formalization attempts that are easier for a human mathematician to inspect, [64, 63, 65].

7.1 The single-model workflow

The simplest model-assisted workflow has only one model and one prompt.

Problem

```
-> Prompt to one model
    -> Model output
        -> Human reads the output
```

This workflow is sometimes sufficient for low-stakes tasks. For example, it may be reasonable when asking for a paraphrase of a definition, a list of possible proof strategies, a reminder of standard notation, or a first $\text{\LaTeX}$ rewrite of a proof that the student already understands. It can also be useful for elementary exercises where the human reader can check every line immediately.

A safe version of the single-model workflow has the following components:

1. **A precise prompt.** The prompt states the domain, assumptions, definitions, and desired output.
2. **A constrained role.** The model is asked to be an explorer, proof drafter, auditor, programmer, or editor, not all roles at once.
3. **A visible verification layer.** The output is required to mark non-trivial steps, hidden assumptions, and possible gaps.

4. **A human decision.** The student checks the mathematical content independently.

A minimal single-model prompt might be:

You are helping with a mathematics exercise.

Role: proof drafter.

Statement: Every convergent sequence of real numbers is bounded.

Definitions: Use the epsilon-N definition of convergence.

Task: Give a candidate proof. Number the steps. After the proof, state exactly where convergence is used and which parts still require human checking.

7.2 Why single-model workflows are risky

A single-model workflow might be risky because it tends to collapse several logically different tasks into one conversation. The model is asked to discover a proof, write the proof, audit the proof, repair the proof, and polish the proof. However, these tasks require different kinds of pressure. Namely, a proof drafter is rewarded, by the structure of the prompt, for producing a coherent route to the conclusion. A proof auditor should do the opposite, i.e., it should search for unsupported implications, missing hypotheses, incorrect theorem use, false algebraic transformations, and possible counterexamples. A final editor should not change the mathematics at all unless explicitly instructed. Combining these roles makes errors harder to see.

There are several common failure modes:

1. **Self-confirmation.** After proposing an argument, the model may continue defending it even when the argument contains a real gap.
2. **Hidden strengthening.** The model may silently replace an assumption such as continuity by differentiability, local convexity by global convexity, or finite infimum by attained minimum.
3. **Fluent compression.** The only difficult step may be hidden in a phrase such as “it follows by standard arguments”.
4. **Definition drift.** A model may switch between different conventions for a term such as “PL condition”, “smooth”, “bounded”, or “almost surely”.
5. **Citation or theorem invention.** A model may invent a name, reference, or theorem statement unless sources are checked.
6. **Overcorrection during editing.** A model asked to improve exposition may alter quantifiers, hypotheses, or conclusions.

So, the issue is that one model is not an adversarial proof checker merely because it is asked to “check” its own proof. A stronger workflow separates the generation step from the criticism step.

Working rule. Use one model conversation to create an object, and a separate conversation, preferably with a different model, to attack that object. Then check the result yourself.

7.3 The two-model adversarial workflow

A useful next step is the two-model adversarial workflow. Model A is assigned the role of proof drafter, while Model B is assigned the role of skeptic. The human reader controls both prompts and does not allow either model to decide final correctness.

Problem

- > Model A: produce a candidate proof
- > Candidate proof
- > Model B: audit the proof adversarially
- > Audit report
- > Model A or human: repair attempt
- > Human mathematical verification

In prompts, the roles should be stated explicitly. So, a proof-generation prompt for Model A may be:

Role: proof drafter.
Write a candidate proof of the statement below.
Use only the stated assumptions and definitions.
Number all steps.
Mark any step that is plausible but not fully justified as [GAP].
Do not cite unnamed standard theorems.

An audit prompt for Model B ask for a precise failure search:

Role: adversarial proof auditor.
Audit the proof below. Do not try to repair it first.
For each proof step, classify it as:
valid as written,
valid but underjustified,
false,
irrelevant,
circular,
or dependent on an unstated assumption.
For every problem, quote the step number and explain the exact issue.
Then search for a counterexample to the theorem as stated.

If the proof is correct, say what theorem or definition justifies each nontrivial step.

The repair step should be kept separate from the audit step. Otherwise the model may smooth over the gap too quickly.

Role: repair assistant.

Use the audit report below. Do not ignore any audit item.

For each item, either:

- repair the proof,
- weaken the conclusion,
- strengthen the assumptions explicitly,
- or state that no repair is currently available.

Return a revised proof and a list of remaining gaps.

The value of the workflow is that it creates more things for the human user to check.

7.4 The cascade workflow for hard problems

For harder problems, a two-model workflow may still be too compressed. A cascade workflow breaks the work into smaller stages. Each stage produces a concrete artifact: a problem card, a list of definitions, a strategy list, a counterexample report, a proof draft, an audit, a repair, a $\text{\LaTeX}$ write-up, or a formalization attempt.

Problem card

- > Definition normalization
- > Related facts and forbidden shortcuts
- > Strategy search
- > Counterexample search
- > Candidate proof
- > Independent audit
- > Repair or theorem revision
- > $\text{\LaTeX}$ write-up
- > Human or formal verification

Stage 1: Problem card

The problem card fixes and describes the mathematical object before the model runs:

Problem card

1. Statement to investigate:
2. Domain:

3. Exact assumptions:
4. Desired conclusion:
5. Known related facts:
6. Definitions that must be used verbatim:
7. Forbidden shortcuts:
8. Try both proof and counterexample:

Stage 2: Definition normalization

Ask a model to restate the definitions exactly and to identify possible ambiguities.

Normalize the definitions in the problem card.

Do not solve the problem.

Return:

1. the definitions copied exactly,
2. possible convention conflicts,
3. notation that must be fixed before proof search,
4. assumptions that are missing if the intended conclusion is to hold.

This stage is especially useful for terms such as “smooth”, “globally Lipschitz”, “one-sided Lipschitz”, “PL inequality”, “global minimizer”, “weak convergence”, and “almost surely”. A definition-normalization stage can prevent a model from solving the wrong problem.

Stage 3: Related facts and forbidden shortcuts

A related-facts prompt asks for candidate lemmas without treating them as true.

List facts that might be relevant to the problem.

For each fact, state all hypotheses.

Classify each fact as:

- definition,
- elementary lemma to prove,
- named theorem requiring citation,
- or conjectural claim to verify.

Do not use any fact as proved unless it is explicitly proved later.

The output should be read as a to-do list. If a named theorem, model detail, software command, or source claim appears, it needs verification against a reliable source.

Stage 4: Strategy search

In a strategy search, we ask for routes:

Generate several possible proof strategies for the problem.

Do not write the full proof yet.

For each strategy, list:

1. main idea,
2. assumptions used,
3. likely obstruction,
4. possible counterexample pattern,
5. what must be verified by hand.

This is often more useful than asking for one polished proof.

Stage 5: Counterexample search

Counterexample search should be independent of the proof search.

Search for counterexamples to the statement as written.

Do not strengthen the assumptions.

Try simple cases first:

[THIS IS AN EXEMPLARY SET OF CASES: finite sets,
one-dimensional real functions,
boundary cases,
constant functions,
piecewise-defined functions,
nonattained infima,
nonconvex examples]

For each candidate, verify every assumption and then check the conclusion.

The instruction “as written” matters, since a model may rescue a false statement by adding hypotheses.

Stage 6: Candidate proof

The model should be asked for a full proof draft only after definitions, strategies, and counterexamples have been explored.

Write a candidate proof using the chosen strategy.

Number every step.

Use only the assumptions in the problem card.

At each step, state the exact justification.

If a step depends on a lemma, state and prove the lemma separately.

Mark unresolved points as [GAP].

Of course, a candidate proof is allowed to have gaps.

Stage 7: Independent audit

The audit model should receive the problem card, definitions, and proof draft, but not the proof author's self-assessment unless the human deliberately wants to audit that as well. The following step aims to localize errors.

Audit the proof line by line.

Use the problem card as the only statement of assumptions.

For each line, check:

1. Does the line follow from previous lines?
2. Does it use an unstated assumption?
3. Does it use a theorem whose hypotheses are not checked?
4. Is there a quantifier error?
5. Is there a local/global confusion?
6. Is the conclusion exactly the desired conclusion?

Return an audit table and a final verdict: correct, incorrect, or unresolved.

The verdict "unresolved" is acceptable. A model that admits uncertainty can be more useful than a model that is 100% sure.

Stage 8: Repair or theorem revision

The proof may be repaired under the original assumptions or the theorem may need a stronger assumption. The conclusion may need to be weakened, while the statement may be false.

Use the audit report to revise the result.

For each audit issue, choose exactly one action:

- A. repair the proof under the original assumptions,
- B. add a new assumption and state it explicitly,
- C. weaken the conclusion,
- D. provide a counterexample,
- E. leave the issue unresolved.

Do not silently change the theorem.

Stage 9: L^AT_EX write-up

The L^AT_EX write-up should be only a formatting and exposition stage, but not a new proof-discovery stage.

Rewrite the verified proof in clean L^AT_EX.

Do not change the statement, assumptions, or mathematical content.

Use theorem/proof style.

Preserve all constants, quantifiers, and notation.

If you think a mathematical correction is needed, do not make it silently; write it as a separate note marked [MATHEMATICAL CHANGE].

This prompt protects the proof from being altered during stylistic editing. It is especially useful when preparing lecture notes or a scientific preprint, where exposition may be revised several times.

Stage 10: Human or formal verification

The final stage is outside the authority of the language model. The human mathematician checks the proof in ordinary mathematical language, or a formal proof system checks a precise formalization. A formal proof system can verify a formal theorem, but only after the theorem has been translated correctly into the formal language. Lean and mathlib provide a framework for such formalization, but generated Lean code must be compiled and the formal statement must be compared with the intended mathematical statement [52, 53, 54, 55, 66].

7.5 Separating roles: explorer, skeptic, formalizer, programmer, editor

The following table summarizes common roles in a multi-model workflow. A single local model may be used for more than one role in a low-stakes setting, but the roles should still be conceptually separated.

Role	Main task	Useful output	Main caution
Explorer	Generate definitions to check, simple examples, possible lemmas, and proof strategies.	Strategy list, candidate lemmas, examples, obstruction list.	Exploration is not evidence that a theorem is true.
Skeptic	Attack a statement or proof.	Audit table, hidden assumptions, counterexample patterns, gap list.	The skeptic can miss errors or invent objections.
Counterexample searcher	Test the statement against simple and boundary cases.	Candidate counterexamples with hypothesis checks.	A failed search is not a proof of truth.
Formalizer	Suggest formal theorem statements or Lean proof fragments.	Candidate Lean statement, import suggestions, proof skeleton.	Formal code must compile, and the formal statement must match the intended theorem.
Programmer	Write Python, Sage-Math, or shell code for experiments and reproducibility.	Scripts, notebooks, parameter scans, plots, test cases.	Numerical evidence and passing tests are not mathematical proof.

Role	Main task	Useful output	Main caution
Editor	Improve notation, structure, and L ^A T _E X exposition.	Cleaner manuscript text, theorem/proof formatting, consistency suggestions.	Editing must not silently change mathematical content.

The role assignment should be visible in the prompt since a vague prompt such as “check this proof and improve it” mixes skepticism and editing. A better sequence is:

1. audit the proof without repairing it,
2. repair the proof without polishing it,
3. polish the verified proof without changing it.

This sequence is slower, but it produces a better mathematical record.

7.6 Human-in-the-loop verification

Human-in-the-loop verification means that a human mathematician checks and takes responsibility for the mathematical claims obtained from the reasoning model. This usually requires reconstructing the proof independently enough to know why each step follows.

A practical verification procedure is the following:

1. **Restate the theorem.** Write the theorem without model prose. Check the domain, assumptions, constants, and conclusion.
2. **List the definitions.** Identify which definitions are used and verify that their conventions match the course.
3. **Check existence claims.** If the proof uses a minimizer, supremum, limit, compactness argument, derivative, or inverse, verify that it exists under the assumptions.
4. **Check every implication.** For each line, ask whether it follows from previous lines or from an explicitly stated theorem.
5. **Check theorem hypotheses.** If a named theorem is used, list all its hypotheses and verify them.
6. **Search for boundary cases.** Test constants equal to zero, empty sets when allowed, nonattained infima, nonconvex functions, finite examples, one-dimensional reductions, etc.
7. **Separate notation from substance.** Ignore the polishing until the logic is checked.

8. **Record corrections.** If a model-generated proof is repaired, keep a note of what was changed and why.

Remark 7.1. *A useful model answer can still be mathematically wrong. For example, it may suggest the correct counterexample family but verify the hypotheses incorrectly. Conversely, a model may give a poor explanation of a valid idea. The human task is to separate useful insight from correctness.*

7.7 Optional formal verification with Lean

Formal verification is a different kind of check from asking another language model. A proof assistant such as Lean checks a proof term against a formal statement in a formal environment. This can be valuable for simple theorems and for carefully selected examples, especially when the theorem is close to material already present in mathlib [52, 53, 54, 55]. Formal verification has two separate layers:

1. **Formal correctness.** Does the Lean file compile? Does Lean accept the proof of the formal theorem?
2. **Faithfulness.** Does the formal theorem express the original informal theorem that the user intended to prove?

The first layer is checked by Lean, while the second layer is a mathematical and interpretive task. A model can suggest a Lean theorem statement, but it may formalize a weaker theorem, use a different definition, add a hidden assumption, or prove a trivialized version. For example, a theorem about all real sequences may be formalized only for a special sequence, while a theorem about a global minimizer may be formalized with an extra existence hypothesis. A cautious Lean workflow is:

Informal theorem

```
-> Human writes intended definitions and statement
    -> Model suggests Lean statement
        -> Human compares Lean statement with intended theorem
            -> Model suggests proof skeleton
                -> Lean compiler checks formal code
                    -> Human checks faithfulness of formalization
```

A prompt for the formalizer role should be restrictive:

Role: Lean formalization assistant.

Do not claim that the code is correct unless it has been compiled.

Task: suggest a Lean 4 theorem statement for the informal theorem below.

Also explain, in ordinary mathematical language, what the formal statement says. List any assumptions that were added, removed, or changed during formalization.

7.8 From model output to publication-quality proof

A useful conversion procedure might have the following passes:

1. **Extraction pass.** Extract the mathematical skeleton from the model output: theorem, assumptions, lemmas, proof steps, and conclusion. Remove motivational prose.
2. **Validity pass.** Check whether each step is true under the stated assumptions. Mark gaps explicitly.
3. **Repair pass.** Repair only those gaps that can be repaired without changing the theorem. If the theorem must change, state the change separately.
4. **Exposition pass.** Rewrite the verified argument in clean mathematical language and L^AT_EX.
5. **Final audit pass.** Read the final proof as if it were written by someone else. Check whether any corrections introduced a new problem.

The following table distinguishes the model artifact from the final mathematical artifact.

Model artifact	Useful content	Required human action
Strategy list	Possible proof routes and obstructions.	Decide which strategies are mathematically plausible, discard irrelevant routes.
Candidate proof	Sequence of proposed implications.	Verify every implication and theorem use, identify gaps.
Audit report	Possible errors, hidden assumptions, and counterexamples.	Check whether the objections are valid, repair or reject them.
Counterexample candidate	A possible object refuting the statement.	Verify all assumptions and show exactly where the conclusion fails.
Python or Sage code	Numerical or symbolic evidence.	Check code, reproduce results, and keep computation separate from proof.
Lean suggestion	Candidate formal statement or proof script.	Compile the code and compare the formal statement with the intended theorem.
L ^A T _E X rewrite	Better exposition and notation.	Confirm that no mathematical content was changed.

7.9 Documentation of prompts and model versions

Model-assisted work is easier to discuss when the workflow is documented. It might be helpful to make the experiment inspectable and reproducible

enough for a student, instructor, or coauthor to understand what happened.

For each significant model interaction, record at least the following fields:

Field	What to record
Model name	The displayed model name or repository name used locally.
Model version or tag	The exact local tag, repository revision, or file identifier when available. If unknown, write explicitly “unknown”.
Quantization	The quantization label, such as Q4 or Q8, if known. If it was not checked, write this fact explicitly.
Date	The date of the interaction.
Prompt	The exact prompt or a pointer to the saved prompt file.
Temperature	The generation temperature or local setting, if known.
Context length	The configured or reported context length if known. If not verified, leave it blank or mark it as not verified.
Output	The raw output or a pointer to the saved output file.
Human corrections	Mathematical corrections, rejected claims, changed assumptions, and final decisions.

A compact log entry can be kept in plain text:

Model interaction log

Date:

Model name:

Model tag or file:

Quantization:

Temperature:

Context length:

Prompt file:

Output file:

Task role: explorer / skeptic / counterexample searcher / formalizer /
programmer / editor

Human corrections:

Final status: useful / partially useful / incorrect / unresolved

The same discipline applies to software and formal verification. If Python, SageMath, or Lean was used, record the software version and relevant files. If a Lean proof was not compiled, it should be labeled as an unverified Lean suggestion. If a numerical experiment was not reproduced, it should be labeled as preliminary evidence.

Final advice: For nontrivial mathematical problems, do not use the same model as the author, referee, and final editor.

Chapter 8

Worked Mathematical Examples

The previous chapters described local language models as tools for mathematical work. This chapter begins to apply that position to concrete problems. The examples are deliberately elementary. Their purpose is not to impress the reader with difficult mathematics, but to make the verification discipline visible. In each case the user should separate three objects:

1. the mathematical problem,
2. the candidate reasoning produced or suggested by a model,
3. the verified mathematical argument accepted by the student,

and these objects should not be collapsed. Every example treats model text as conjectural until the mathematics has been checked by the user. The examples in this chapter illustrate a progression: elementary proof checking, false-claim detection, convexity reasoning, Polyak–Łojasiewicz definition discipline, numerical exploration, flawed-proof auditing, LaTeX repair, and optional Lean formalization. Throughout the chapter, a “model-output pattern” means a summarized description of a plausible response from a local model.

8.1 Example 1: Every convergent real sequence is bounded

Problem statement

Let $(x_n)_{n \geq 1}$ be a sequence of real numbers. Prove that if (x_n) converges in $\mathbb{R}$, then (x_n) is bounded.

In explicit quantifier form, the conclusion is:

$$\exists M > 0 \forall n \geq 1 \quad |x_n| \leq M.$$

The assumption is that there exists $L \in \mathbb{R}$ such that $x_n \rightarrow L$.

Why this is a useful LLM test

This is a standard theorem from elementary analysis, but it is a good first test because the proof contains a small structural point that models sometimes compress too much. Convergence gives control only for all sufficiently large indices. Boundedness requires control of all indices. The proof must therefore handle the finitely many initial terms separately. A model that writes only “since $x_n \rightarrow L$, the sequence is close to L , hence bounded” has the right idea but has not supplied a complete proof. The missing part is not deep, but it is exactly the kind of omitted detail that the user should detect.

Initial prompt

Prove that every convergent sequence of real numbers is bounded.
Give a rigorous proof and explicitly state where convergence is used.
Use the epsilon-N definition of convergence.
Do not cite the theorem "convergent sequences are bounded".

A plausible model-output pattern

A plausible model response will introduce the limit L , choose $\varepsilon = 1$, and conclude that for all sufficiently large n , $|x_n - L| < 1$. A good response then uses the triangle inequality to bound the tail by $|L| + 1$ and takes a maximum over the finitely many initial terms.

A weaker response may stop after bounding the tail. It may say that all terms are bounded because all sufficiently large terms are bounded. That statement does not yet address the initial segment $x_1, \dots, x_{N-1}$. The omitted step is easy but necessary.

What should be checked

1. Was the definition of convergence stated correctly?
2. Was a specific value of ε chosen? The usual choice $\varepsilon = 1$ is sufficient.
3. Did the proof distinguish the tail $n \geq N$ from the initial segment $1 \leq n < N$?
4. Was the finite maximum over initial terms well-defined?
5. Did the final bound work for every index $n \geq 1$, not only for large n ?

Correct mathematical solution

Assume that $x_n \rightarrow L$ for some $L \in \mathbb{R}$. By the definition of convergence, for $\varepsilon = 1$ there exists an integer $N \geq 1$ such that

$$n \geq N \implies |x_n - L| < 1.$$

For such n , the triangle inequality gives

$$|x_n| \leq |x_n - L| + |L| < 1 + |L|.$$

Thus the tail of the sequence is bounded by $|L| + 1$.

It remains to include the finitely many initial terms. Define

$$M = \max\{|L| + 1, |x_1|, |x_2|, \dots, |x_{N-1}|\},$$

where the list $|x_1|, \dots, |x_{N-1}|$ is omitted if $N = 1$. This is the maximum of a finite nonempty set of real numbers, so it is well-defined and satisfies $M > 0$.

If $1 \leq n < N$, then $|x_n| \leq M$ by the definition of M . If $n \geq N$, then $|x_n| < |L| + 1 \leq M$. Therefore $|x_n| \leq M$ for every $n \geq 1$. Hence (x_n) is bounded.

Proof-audit prompt

Audit the proof below.

Problem: prove that every convergent real sequence is bounded.

Definition of bounded sequence: there exists $M > 0$ such that

$|x_n| \leq M$ for every $n \geq 1$.

Definition of convergence: $x_n \rightarrow L$ means that for every $\varepsilon > 0$ there exists N such that $n \geq N$ implies $|x_n - L| < \varepsilon$.

Tasks:

1. Identify where convergence is used.
2. Check whether the proof handles finitely many initial terms.
3. Check whether the final bound applies to every n .
4. Mark any hidden assumption or gap.

Key lesson

When auditing model output, look for quantifier changes such as “exists” becoming “for all”.

8.2 Example 2: A false claim about critical points

Problem statement

Investigate the following claim.

If a differentiable function has exactly one critical point, must that point be a global minimum?

For this example we interpret the claim as follows: let $f : \mathbb{R} \rightarrow \mathbb{R}$ be differentiable, and call x_0 a critical point if $f'(x_0) = 0$. If f has exactly one critical point, is that point necessarily a global minimizer of f ?

Why this is a useful LLM test

The claim sounds plausible if one remembers examples such as $f(x) = x^2$. It is also close to a true convex statement: for a differentiable convex function, a zero gradient implies a global minimum. Without convexity, however, the statement is false. This example tests whether a model silently imports extra assumptions. A model may reason as if every differentiable function behaves like a convex function or as if a first-derivative test automatically gives a global minimum. Both moves are invalid.

Initial prompt

Investigate the following claim. Do not assume it is true.

Claim: If a differentiable function $f: \mathbb{R} \rightarrow \mathbb{R}$ has exactly one critical point, then that critical point is a global minimum.

Either prove the claim or give a concrete counterexample.

If you give a counterexample, verify:

1. f is differentiable on $\mathbb{R}$;
2. f has exactly one critical point;
3. that point is not a global minimizer.

Do not assume convexity unless it is explicitly stated.

A plausible model-output pattern

A useful model response may propose $f(x) = x^3$. It may compute $f'(x) = 3x^2$, observe that the only critical point is 0, and then note that values of f below 0 occur for negative x . This is the correct counterexample if all details are checked. A defective response may instead discuss $f(x) = x^2$, which is not a counterexample because its unique critical point is indeed a global minimum. Another defective response may say that a unique critical point “must be” a minimum because the derivative changes sign. But no derivative sign change has been assumed.

What the student should check

For any proposed counterexample, the student should verify all hypotheses before looking at the conclusion.

1. Is the function defined on all of $\mathbb{R}$, as required?
2. Is it differentiable everywhere?
3. Are all critical points found, not only one of them?
4. Does the proposed unique critical point fail to be a global minimizer?
5. Is the argument relying on a graph or on exact inequalities?

Correct counterexample and verification

Let

$$f(x) = x^3, \quad x \in \mathbb{R}.$$

Then f is differentiable on $\mathbb{R}$, and

$$f'(x) = 3x^2.$$

Therefore $f'(x) = 0$ if and only if $x = 0$. Hence 0 is exactly one critical point of f . However, 0 is not a global minimizer. We have $f(0) = 0$, while for any negative number, for example $x = -1$,

$$f(-1) = -1 < 0 = f(0).$$

Thus there exists a point $x \in \mathbb{R}$ with $f(x) < f(0)$. Consequently 0 is not a global minimizer. The claim is false.

In fact, 0 is not even a local minimizer: for every $\delta > 0$, one may choose $x \in (-\delta, 0)$, and then $f(x) = x^3 < 0 = f(0)$. This additional observation is not needed to refute the global-minimum claim, but it helps identify the geometric error behind the false statement.

Proof-audit prompt

Audit the proposed counterexample below for the claim:

"If a differentiable function $f: \mathbb{R} \rightarrow \mathbb{R}$ has exactly one critical point, then that point is a global minimum."

Definition: a critical point is a point x_0 such that $f'(x_0) = 0$.

Definition: x_0 is a global minimizer if $f(x_0) \leq f(x)$ for every x in $\mathbb{R}$.

Tasks:

1. Check differentiability on all of $\mathbb{R}$.
2. Compute the derivative and solve $f'(x) = 0$ exactly.
3. Verify that there is exactly one critical point.
4. Verify that the critical point is not a global minimizer.
5. Identify any hidden use of convexity, monotonicity, or a graph.

Key lesson

The number of critical points alone does not determine global optimality. The false claim becomes tempting because the user may remember a correct theorem with an extra hypothesis, such as convexity, and then forget that the hypothesis is essential. A local model can make the same mistake. The right response is to force the reasoning model to verify every assumption and every failure of the conclusion.

8.3 Example 3: Convexity and global minimizers

Problem statement

Prove the following statement.

Let $f : \mathbb{R}^d \rightarrow \mathbb{R}$ be differentiable and convex. If $\nabla f(x^*) = 0$, then x^* is a global minimizer of f , i.e.,

$$f(x^*) \leq f(y) \quad \text{for every } y \in \mathbb{R}^d.$$

Why this is a useful LLM test

This example should be read together with Example 8.2. There, a unique critical point did not imply a global minimum. Here, a zero gradient does imply a global minimum, but only because convexity is present. The example tests whether a reasoning model can distinguish a true theorem from a false overgeneralization. It also tests whether the model explains where convexity is used. A proof that says only "the gradient is zero, therefore the point is a minimum" is not acceptable.

Initial prompt

Prove the following theorem rigorously.

Let $f: \mathbb{R}^d \rightarrow \mathbb{R}$ be differentiable and convex.

If $\text{grad } f(x^*) = 0$, prove that x^* is a global minimizer.

Requirements:

1. State exactly where convexity is used.
2. Do not use a second derivative test.
3. Do not assume strict convexity.
4. Prove the inequality $f(y) \geq f(x^*)$ for an arbitrary y in $\mathbb{R}^d$.

A plausible model-output pattern

A good model response will use the first-order inequality for differentiable convex functions,

$$f(y) \geq f(x) + \langle \nabla f(x), y - x \rangle,$$

with $x = x^*$. Since $\nabla f(x^*) = 0$, this gives $f(y) \geq f(x^*)$ for all y .

A response that merely cites this inequality may be acceptable in a later course if the inequality has already been proved. In these notes, the user should also be able to prove the needed inequality directly from convexity. This prevents the model from hiding the only real step behind a named or unnamed theorem.

What should be checked

1. Is convexity explicitly used?
2. Is the proof valid in $\mathbb{R}^d$, not only in one dimension?
3. Does the proof work without strict convexity?
4. Does it prove global minimality, not merely local minimality?
5. Does the proof show the inequality for an arbitrary point $y \in \mathbb{R}^d$?

It is especially important not to replace the hypothesis $\nabla f(x^*) = 0$ by the stronger and different hypothesis that x^* is already known to minimize f .

Correct mathematical solution

Let $y \in \mathbb{R}^d$ be arbitrary. Define

$$g(t) = f(x^* + t(y - x^*)), \quad 0 \leq t \leq 1.$$

Because f is convex, the function g is convex on $[0, 1]$. Indeed, for $s, t \in [0, 1]$ and $\lambda \in [0, 1]$, the point corresponding to $\lambda s + (1 - \lambda)t$ is the same convex combination of the points corresponding to s and t , so convexity of f gives convexity of g .

Convexity of g gives, for every $t \in (0, 1]$,

$$g(t) \leq (1 - t)g(0) + tg(1).$$

Rearranging,

$$\frac{g(t) - g(0)}{t} \leq g(1) - g(0).$$

Since f is differentiable, g is differentiable at 0 from the right, and

$$g'(0) = \langle \nabla f(x^*), y - x^* \rangle.$$

Letting $t \downarrow 0$ in the previous inequality yields

$$\langle \nabla f(x^*), y - x^* \rangle \leq f(y) - f(x^*).$$

By the assumption $\nabla f(x^*) = 0$, the left-hand side is zero. Hence

$$0 \leq f(y) - f(x^*),$$

or equivalently

$$f(x^*) \leq f(y).$$

Because $y \in \mathbb{R}^d$ was arbitrary, x^* is a global minimizer of f .

Proof-audit prompt

Audit the proof below for the theorem:

Let $f: \mathbb{R}^d \rightarrow \mathbb{R}$ be differentiable and convex.

If $\text{grad } f(x^*) = 0$, then x^* is a global minimizer.

[PASTE PROOF HERE]

Tasks:

1. Identify the step where convexity is used.
2. Check whether the argument works for every y in $\mathbb{R}^d$.
3. Check whether the proof uses strict convexity, compactness, or a second derivative test without permission.
4. Verify the passage from the one-dimensional restriction to the gradient expression.
5. Confirm that the conclusion is global minimality, not only local minimality.

Key lesson

Example 8.2 and Example 8.3 form a pair. The statement “zero derivative implies global minimum” is false in general. The statement becomes true under convexity. The workflow lesson is therefore to identify exactly which hypothesis changes a false statement into a true one. A model that gives the right conclusion but does not isolate the role of convexity has not given a satisfactory mathematical explanation.

8.4 Example 4: The Polyak–Łojasiewicz inequality and minimizers

Problem statement

Let $f: \mathbb{R}^d \rightarrow \mathbb{R}$ be differentiable and let

$$f^* = \inf_{x \in \mathbb{R}^d} f(x) > -\infty.$$

In this example we use the following convention for the global Polyak–Łojasiewicz condition: there exists a constant $\mu > 0$ such that

$$\frac{1}{2}\|\nabla f(x)\|^2 \geq \mu(f(x) - f^*) \quad \text{for every } x \in \mathbb{R}^d.$$

Investigate what this condition implies about critical points and global minimizers. In particular, decide which of the following statements are justified:

1. every critical point is a global minimizer
2. the function must be convex
3. the function must be strongly convex
4. the global minimizer must be unique

Why this is a useful LLM test

This example tests definition discipline. Local models sometimes write the Polyak–Łojasiewicz inequality in the wrong direction. The direction is not a cosmetic issue. The inequality used here says that the squared gradient controls the function-value gap from below:

$$\frac{1}{2}\|\nabla f(x)\|^2 \geq \mu(f(x) - f^*).$$

If a model reverses the inequality and writes

$$f(x) - f^* \geq \mu\|\nabla f(x)\|^2,$$

then setting $\nabla f(x) = 0$ gives only the trivial statement $f(x) - f^* \geq 0$. It does not imply that x is a global minimizer. Thus a reversed inequality destroys the main conclusion of this example.

The example also separates several notions that are often conflated: convexity, strong convexity, the PL inequality, and uniqueness of minimizers. They are related in some familiar examples, but they are not the same property. For background on the PL condition and its relationship to strong convexity and convexity, see [69].

Initial prompt

Consider a differentiable function $f: \mathbb{R}^d \rightarrow \mathbb{R}$ with finite infimum f^* . Use the following definition of the global Polyak-Łojasiewicz condition: there exists $\mu > 0$ such that

$$(1/2) \|\text{grad } f(x)\|^2 \geq \mu (f(x) - f^*)$$

for every x in $\mathbb{R}^d$.

Tasks:

1. Prove what this condition implies about critical points.
2. Explain why the direction of the inequality matters.
3. Decide whether PL implies convexity, strong convexity, or uniqueness of minimizers. Give examples or counterexamples where possible.
4. Do not assume convexity unless it is explicitly stated.

A plausible model-output pattern

A useful response will set $\nabla f(x) = 0$ in the PL inequality and derive $f(x) = f^*$. A defective response may reverse the inequality, then still claim that critical points must minimize. Another defective response may state that PL functions are strongly convex or have unique minimizers. Those claims require separate hypotheses and are false as stated.

The safest student response is to ask the model to state the exact inequality first, then audit every conclusion against that inequality.

What should be checked

1. Is f^* the infimum of f , and is it finite?
2. Is the PL inequality written in the correct direction?
3. Is the factor $1/2$ only a convention, or has it changed the mathematical claim? Constants may be renamed, but the direction may not be reversed.
4. Does the argument prove only “critical point $\Rightarrow$ global minimizer”, or does it also assert existence of a critical point?
5. Are convexity, strong convexity, and uniqueness being smuggled in without proof?

Correct mathematical solution and discussion

Assume that f satisfies the PL inequality above. Let $x_0 \in \mathbb{R}^d$ be a critical point, so that $\nabla f(x_0) = 0$. Substituting $x = x_0$ into the PL inequality gives

$$0 = \frac{1}{2} \|\nabla f(x_0)\|^2 \geq \mu(f(x_0) - f^*).$$

Since $\mu > 0$, it follows that

$$f(x_0) - f^* \leq 0.$$

On the other hand, by the definition of f^* as an infimum,

$$f(x_0) - f^* \geq 0.$$

Therefore $f(x_0) = f^*$. Hence x_0 is a global minimizer.

This proves:

$$\nabla f(x_0) = 0 \implies f(x_0) = f^*.$$

It does not prove that a critical point exists. It says that if a critical point exists, then it is globally minimizing.

Conversely, if f is differentiable on all of $\mathbb{R}^d$ and x_0 is a global minimizer, then ordinary first-order calculus gives $\nabla f(x_0) = 0$. Indeed, for every direction $v \in \mathbb{R}^d$, the one-variable function $t \mapsto f(x_0 + tv)$ has a global minimum at $t = 0$, so its derivative at 0 is zero. Thus $\nabla f(x_0) \cdot v = 0$ for all v , and hence $\nabla f(x_0) = 0$.

Therefore, for a differentiable PL function on $\mathbb{R}^d$, the set of global minimizers coincides with the set of critical points; both sets may be empty. The PL condition alone does not guarantee uniqueness. Consider

$$g(x_1, x_2) = \frac{1}{2}x_1^2, \quad (x_1, x_2) \in \mathbb{R}^2.$$

Then $g^* = 0$,

$$\nabla g(x_1, x_2) = (x_1, 0), \quad \frac{1}{2}\|\nabla g(x_1, x_2)\|^2 = \frac{1}{2}x_1^2 = g(x_1, x_2).$$

Thus g satisfies the PL inequality with $\mu = 1$. But every point of the line

$$\{(0, x_2) : x_2 \in \mathbb{R}\}$$

is a global minimizer. The minimizer is not unique.

The same example shows that PL does not imply strong convexity. If g were strongly convex with some parameter $m > 0$, then $(x_1, x_2) \mapsto g(x_1, x_2) - \frac{m}{2}(x_1^2 + x_2^2)$ would be convex. On the line $x_1 = 0$, this restriction is $-\frac{m}{2}x_2^2$, which is not convex for $m > 0$. Hence g is not strongly convex.

Convexity also does not imply the PL inequality. Let $h(x) = x^4$ on $\mathbb{R}$. The function is convex and has $h^* = 0$. If it satisfied the PL inequality with some $\mu > 0$, then for $x \neq 0$,

$$\frac{1}{2}|h'(x)|^2 = \frac{1}{2}(4x^3)^2 = 8x^6 \geq \mu x^4.$$

Dividing by $x^4 > 0$ gives $8x^2 \geq \mu$ for all $x \neq 0$, which is impossible as $x \rightarrow 0$. Thus h is convex but does not satisfy a global PL inequality with positive constant.

A nonconvex PL example is given in Example 8.5, using $f(x) = x^2 + 2\sin^2 x$. The point here is not to memorize a taxonomy, but to keep the logical implications separate.

Proof-audit prompt

Audit the following answer about the Polyak-Lojasiewicz condition.
Use this definition:

$$(1/2) \|\text{grad } f(x)\|^2 \geq \mu (f(x) - f^*)$$

where $\mu > 0$ and $f^* = \inf f$.

Tasks:

1. Check the direction of the PL inequality.
2. Check what happens when $\text{grad } f(x) = 0$.
3. Check whether the answer proves existence of a minimizer or only says that critical points are minimizers.
4. Identify any unsupported claim that PL implies convexity, strong convexity, or uniqueness.
5. Give a corrected statement.

Key lesson

In this example, the correct PL direction forces every critical point to have value f^* . The reversed direction does not. A local model that writes the inequality in the wrong direction may still produce fluent paragraphs, but the proof has lost its mathematical content.

8.5 Example 5: A function of the form $f(x) = x^2 + a \sin^2(x)$

Problem statement

For $a \geq 0$, consider

$$f_a(x) = x^2 + a \sin^2 x, \quad x \in \mathbb{R}.$$

Use this family to study symbolic differentiation, convexity, numerical exploration, and the difference between computational evidence and proof. The goal is not to obtain every possible property of this family. The goal is to demonstrate a disciplined workflow.

Why this is a useful LLM test

This example is useful because it contains both easy and tempting steps. The function is visibly nonnegative when $a \geq 0$, so its global minimum is easy to identify. The derivative and second derivative are also elementary.

But convexity depends on a parameter, and a plot can easily mislead the reader into overgeneralizing from a few sampled points.

A model may correctly compute derivatives but then make an unsupported global claim, such as “the function is convex because it is a sum of squares”. The phrase “sum of squares” is not enough: $\sin^2 x$ is not globally convex.

Initial prompt

Let $a \geq 0$ and $f_a(x) = x^2 + a \sin^2(x)$ on $\mathbb{R}$.

Analyze the following questions rigorously:

1. Compute $f_a'(x)$ and $f_a''(x)$.
 2. Find the global minimum value.
 3. Determine for which $a \geq 0$ the function is convex.
 4. For $a = 2$, decide whether the function is convex.
 5. Clearly separate numerical evidence from proof.
- Do not conclude convexity from a plot.

A plausible model-output pattern

A useful response will compute

$$f_a'(x) = 2x + a \sin(2x), \quad f_a''(x) = 2 + 2a \cos(2x),$$

and then use $-1 \leq \cos(2x) \leq 1$ to analyze convexity. A weaker response may compute the derivatives correctly but claim convexity for all $a \geq 0$. That conclusion is false for $a > 1$, because the second derivative is then negative at $x = \pi/2$.

Another plausible response is to run a numerical scan, observe a minimum near 0, and infer that 0 is the global minimizer. The conclusion is correct, but the proof is simpler and exact: for $a \geq 0$, both x^2 and $a \sin^2 x$ are nonnegative.

What should be checked

1. Were the trigonometric derivatives computed correctly?
2. Was the parameter range $a \geq 0$ kept fixed throughout?
3. Was convexity proved using a global condition, not by plotting?
4. Was nonconvexity for $a > 1$ verified by an exact point where $f_a'' < 0$?
5. Did the model confuse global minimality with convexity?

Correct mathematical analysis

For $a \geq 0$, we have

$$f_a(x) = x^2 + a \sin^2 x \geq 0 \quad \text{for every } x \in \mathbb{R},$$

and $f_a(0) = 0$. Hence

$$f_a^* = 0,$$

and $x = 0$ is a global minimizer for every $a \geq 0$. This conclusion does not require convexity.

Differentiating gives

$$f'_a(x) = 2x + 2a \sin x \cos x = 2x + a \sin(2x),$$

and

$$f''_a(x) = 2 + 2a \cos(2x).$$

Since $\cos(2x) \in [-1, 1]$, the smallest possible value of $2 + 2a \cos(2x)$ for $a \geq 0$ is $2 - 2a$. Therefore, if $0 \leq a \leq 1$, then

$$f''_a(x) \geq 0 \quad \text{for all } x \in \mathbb{R},$$

so f_a is convex.

If $a > 1$, then

$$f''_a(\pi/2) = 2 + 2a \cos(\pi) = 2 - 2a < 0.$$

A twice differentiable convex function on an interval cannot have a negative second derivative at a point. Hence f_a is not convex for $a > 1$. Thus, for $a \geq 0$,

$$f_a \text{ is convex} \quad \Longleftrightarrow \quad 0 \leq a \leq 1.$$

This statement uses the standing assumption $a \geq 0$. If the same formula is studied with a allowed to range over all real numbers, the answer is $-1 \leq a \leq 1$; see Project 4 in Chapter 10 for the corresponding sign split.

The case $a = 2$ is a useful nonconvex example. Let

$$f(x) = x^2 + 2 \sin^2 x.$$

Then $f''(\pi/2) = -2$, so f is not convex. Nevertheless f is nonnegative and has global minimum value 0 at $x = 0$. Moreover, this specific function satisfies a global PL inequality, although the proof gives only existence of a positive constant, not the best constant.

Indeed,

$$f'(x) = 2x + 2 \sin(2x) = 2(x + \sin(2x)).$$

For $x > 0$, if $0 < x < \pi/2$, then $\sin(2x) > 0$, hence $x + \sin(2x) > 0$. If $x \geq \pi/2$, then

$$x + \sin(2x) \geq x - 1 > 0,$$

because $\pi/2 > 1$. Thus $f'(x) > 0$ for $x > 0$. Since f' is odd, it follows that $f'(x) < 0$ for $x < 0$. Therefore the only critical point is $x = 0$.

For $x \neq 0$, define

$$R(x) = \frac{|f'(x)|^2}{2f(x)}.$$

The denominator is positive for $x \neq 0$, and R is continuous on $\mathbb{R} \setminus \{0\}$. The preceding paragraph shows that $f'(x) \neq 0$ for $x \neq 0$, so $R(x) > 0$ there. Taylor expansion at 0 gives

$$f(x) = 3x^2 + O(x^4), \quad f'(x) = 6x + O(x^3),$$

so $R(x) \rightarrow 6$ as $x \rightarrow 0$. Also $R(x) \rightarrow 2$ as $|x| \rightarrow \infty$, since $f(x) \sim x^2$ and $f'(x) \sim 2x$. These two limits supply the endpoint control needed for a compactness argument. Choose $\delta > 0$ such that $R(x) \geq 3$ whenever $0 < |x| < \delta$. Choose $A > \delta$ such that $R(x) \geq 1$ whenever $|x| > A$. On the compact set

$$[-A, -\delta] \cup [\delta, A],$$

R is continuous and positive, so it attains a positive minimum $m > 0$. With $\mu = \min\{1, 3, m\}$, the three regions $0 < |x| < \delta$, $\delta \leq |x| \leq A$, and $|x| > A$ cover $\mathbb{R} \setminus \{0\}$ with no gap. Therefore

$$\frac{1}{2}|f'(x)|^2 \geq \mu f(x) \quad \text{for every } x \neq 0.$$

At $x = 0$, both sides are 0, so the inequality also holds at 0. Thus this particular nonconvex function satisfies the PL condition used in Example 8.4. This is a proof by elementary calculus and compactness, not by plotting.

Proof-audit prompt

Audit the following analysis of $f_a(x) = x^2 + a \sin^2(x)$, $a \geq 0$.

Tasks:

1. Check $f_a'(x)$ and $f_a''(x)$.
2. Check the argument for the global minimum value.
3. Check whether convexity is proved for all x , not only near 0.
4. If nonconvexity is claimed, identify an exact point where $f_a'' < 0$.
5. Separate any numerical observation from a mathematical proof.
6. If a PL claim is made, check the exact inequality and every step used to prove a positive lower bound.

Key lesson

For $f_a(x) = x^2 + a \sin^2 x$, the derivatives are easy to compute, the global minimum is immediate from nonnegativity, and convexity follows from a second-derivative calculation. A plot may suggest these facts, but the proof uses exact inequalities.

8.6 Example 6: Numerical evidence versus proof

Problem statement

Use a numerical experiment to investigate the function

$$f(x) = x^2 + 2 \sin^2 x.$$

Then decide which conclusions are proved and which are only suggested by the computation.

Why this is a useful LLM test

Local models are often helpful when writing small Python scripts for plots, grids, symbolic checks, or finite searches. That help is useful, but it can also blur the boundary between evidence and proof. A grid check over $[-10, 10]$ is not a proof over $\mathbb{R}$. A floating-point minimum near zero is not a theorem about the exact global minimum. A graph with no visible extra critical point does not prove that no extra critical point exists.

Initial prompt

Write a small Python experiment for $f(x)=x^2+2 \sin^2(x)$.
Estimate the minimum on a grid, inspect $f'(x)$, and inspect $f''(x)$.
Then clearly state which observations are numerical evidence and which claims still require proof.
Do not say that the plot proves a theorem.

A plausible model-output pattern

A plausible model response may produce code similar to the following.

```
import numpy as np

xs = np.linspace(-10.0, 10.0, 4001)
f = xs**2 + 2.0*np.sin(xs)**2
fp = 2.0*xs + 2.0*np.sin(2.0*xs)
fpp = 2.0 + 4.0*np.cos(2.0*xs)

print(xs[np.argmin(f)], np.min(f))
print(np.min(fpp), np.max(fpp))
```

Such a script may suggest that the minimum is attained near 0, that the function is not convex, and that the derivative changes sign at 0. These are useful observations. They are not yet proofs.

What should be checked

1. What interval was sampled?
2. What grid spacing was used?
3. Were all computations floating-point computations?
4. Does the conclusion concern only sampled points, or all real points?
5. Is there an exact symbolic argument that replaces the numerical observation?

Correct mathematical discussion

The numerical observation that $f(x) \geq 0$ near the sampled points is not the proof. The proof is

$$x^2 \geq 0, \quad 2 \sin^2 x \geq 0,$$

so $f(x) \geq 0$ for every real x , and $f(0) = 0$. Hence the global minimum value is exactly 0.

The numerical observation that the second derivative is sometimes negative is also not the proof. The proof is the exact calculation

$$f''(x) = 2 + 4 \cos(2x),$$

and therefore

$$f''(\pi/2) = 2 + 4 \cos(\pi) = -2 < 0.$$

Thus f is not convex.

The numerical observation that f' appears to vanish only at 0 is not the proof. A proof was given in Example 8.5: for $x > 0$, one has $x + \sin(2x) > 0$, hence $f'(x) > 0$, and by oddness $f'(x) < 0$ for $x < 0$. Therefore the only critical point is 0.

Proof-audit prompt

Audit the numerical discussion below.

Problem: $f(x) = x^2 + 2 \sin^2(x)$ on $\mathbb{R}$.

Tasks:

1. List every conclusion supported only by a grid or plot.
2. List every conclusion proved by an exact symbolic argument.
3. For each numerical claim, say what theorem or inequality would be needed to turn it into a proof.
4. Check whether the interval $[-10, 10]$ was incorrectly treated as all of $\mathbb{R}$.
5. Check whether floating-point output was incorrectly treated as exact.

Key lesson

Numerical evidence is often valuable for conjecture formation. A model-generated computation is best viewed as a way to find what should be proved.

8.7 Example 7: Auditing a flawed model-generated proof

Problem statement

Audit the following intentionally flawed proof.

Flawed proof. Let $f(x) = x^2 + 2\sin^2 x$. The function x^2 is convex. Also, $\sin^2 x$ is a square, so it is convex. A nonnegative multiple of a convex function is convex, and a sum of convex functions is convex. Therefore f is convex.

Find the first invalid step and give a corrected discussion.

Why this is a useful LLM test

The flawed proof has the style of a reasonable mathematical argument. It uses true closure rules for convex functions: nonnegative multiples and sums of convex functions are convex. The error is hidden in the assertion that $\sin^2 x$ is convex because it is a square. That assertion is false.

A model may make exactly this kind of mistake: it uses a familiar rule in a place where one of the hypotheses has not been verified.

Initial prompt

Audit the following proof. Find the first invalid step.
Do not merely say whether the conclusion is true or false.
Explain exactly which assertion fails and provide a corrected discussion.

Claim: $f(x)=x^2+2 \sin^2(x)$ is convex on $\mathbb{R}$.

Proof: x^2 is convex. $\sin^2(x)$ is a square, so it is convex.

A nonnegative multiple of a convex function is convex, and a sum of convex functions is convex. Therefore f is convex.

A plausible model-output pattern

A useful model response will identify the first invalid step as the claim that $\sin^2 x$ is convex because it is a square. A weaker response may say that “sums of convex functions are convex” is the problem. That closure rule is not the problem; it is true. The problem is that one of the summands has not been shown to be convex.

What should be checked

1. Is the conclusion actually true or false?
2. Which line is the first invalid line?
3. Are later true closure rules being blamed for an earlier false premise?
4. Can the error be exposed by differentiating?
5. Does the corrected discussion distinguish nonnegativity from convexity?

Corrected discussion

The first invalid step is:

$\sin^2 x$ is a square, so it is convex.

The square of a function is not necessarily convex. In this case,

$$\frac{d^2}{dx^2} \sin^2 x = 2 \cos(2x),$$

which is negative, for example, at $x = \pi/2$. Thus $\sin^2 x$ is not convex on $\mathbb{R}$.

For the whole function $f(x) = x^2 + 2 \sin^2 x$, we have

$$f''(x) = 2 + 4 \cos(2x),$$

so

$$f''(\pi/2) = 2 - 4 = -2 < 0.$$

Therefore f is not convex.

Notice the separate true statement: $f(x) \geq 0$ for all x , and $f(0) = 0$. Thus 0 is a global minimizer. Nonnegativity and convexity are different properties. The flawed proof confused a useful nonnegativity observation with a global shape property.

Proof-audit prompt

Audit a proof line by line.

For each line, classify it as:

- correct;
- correct but irrelevant;
- unsupported;
- false;
- true only under missing assumptions.

Then identify the first invalid step and explain how to repair the argument. Do not replace the argument by a new proof until the first invalid step has been located.

Key lesson

When auditing a proof, try to Find the first invalid step. This is often more informative than simply producing a counterexample or a corrected proof. It teaches which rule was misapplied and prevents the same error from reappearing in a different problem.

8.8 Example 8: From informal solution to $\text{\LaTeX}$ proof

Problem statement

Use a local model to rewrite a verified mathematical argument into clean $\text{\LaTeX}$ prose. The rewriting step should occur only after the mathematics has already been checked.

As a concrete example, use the following verified statement:

For $a \geq 0$, the function $f_a(x) = x^2 + a \sin^2 x$ is convex on $\mathbb{R}$ if and only if $0 \leq a \leq 1$.

Why this is a useful LLM test

Local models can be useful editors. They can improve sentence flow, normalize notation, and produce a readable $\text{\LaTeX}$ proof. But editing is dangerous if the model silently changes assumptions or conclusions. The model should not be asked to repair the mathematics and rewrite the exposition in the same undisciplined step.

Initial prompt

Rewrite the verified argument below as a concise $\text{\LaTeX}$ proof.
Do not change the theorem, assumptions, notation, or conclusion.
Do not add a stronger claim.
Do not remove the condition $a \geq 0$.
If you think a mathematical step is missing, mark it with `[CHECK]` instead of inventing a new theorem.

Verified argument:

For $a \geq 0$, $f_a(x) = x^2 + a \sin^2(x)$.
Then $f_a''(x) = 2 + 2a \cos(2x)$.
Since $\cos(2x)$ is between -1 and 1 , $f_a''(x) \geq 2 - 2a$.
If $0 \leq a \leq 1$, then $f_a''(x) \geq 0$ for all x , so f_a is convex.
If $a > 1$, then $f_a''(\pi/2) = 2 - 2a < 0$, so f_a is not convex.
Therefore f_a is convex exactly for $0 \leq a \leq 1$.

A plausible model-output pattern

A useful model response will produce a proposition and proof without changing the parameter range. A defective response may write “for all real a ” or may replace the if-and-only-if statement by only one implication. Another common defect is to write a polished proof that says “clearly convex” without showing the second-derivative calculation.

What should be checked

1. Did the model preserve the assumption $a \geq 0$?
2. Did it preserve the conclusion “if and only if $0 \leq a \leq 1$ ”?
3. Did it compute the second derivative correctly?
4. Did it prove both directions?
5. Did it introduce an unproved stronger claim?

A clean L^AT_EX version after verification

After the mathematics has been checked, the proof may be written as follows.

```
\begin{proposition}
Let  $a \geq 0$  and define

$$f_a(x) = x^2 + a \sin^2 x, \quad x \in \mathbb{R}.$$

Then  $f_a$  is convex on  $\mathbb{R}$  if and only if  $0 \leq a \leq 1$ .
\end{proposition}

\begin{proof}
We have

$$f'_a(x) = 2x + a \sin(2x),$$


$$f''_a(x) = 2 + 2a \cos(2x).$$

If  $0 \leq a \leq 1$ , then  $\cos(2x) \geq -1$  gives

$$f''_a(x) \geq 2 - 2a \geq 0$$

for every  $x \in \mathbb{R}$ . Hence  $f_a$  is convex.

If  $a > 1$ , then
```

```

\l
  f_a''(\pi/2)=2+2a\cos(\pi)=2-2a<0.
\r
Thus \f_a cannot be convex on \(\mathbb{R}\). Therefore \f_a is
convex exactly for \((0\leq a\leq 1)\).
\end{proof}

```

The displayed proof is an exposition of already checked mathematics. The model may help with formatting, but the human reader must still verify that no assumption or quantifier has changed.

Proof-audit prompt

Compare the LaTeX proof with the verified argument.

Tasks:

1. Check that the assumptions are identical.
2. Check that the conclusion is identical.
3. Check every displayed formula.
4. Identify any added theorem or omitted condition.
5. Decide whether the LaTeX version is a faithful rewrite or a mathematical modification.

Key lesson

A model should be used as an editor only after the mathematical content has been stabilized. The correct workflow is: solve, audit, repair, then ask for a faithful L^AT_EX rewrite.

8.9 Example 9: Optional Lean formalization of a simple theorem

Problem statement

Use a local model to help draft a Lean formalization of a very simple theorem. This example is optional. It is included to illustrate the boundary between informal mathematics and formal verification.

The target informal statement is deliberately modest:

For every real number L , the constant sequence $x_n = L$ is bounded.

This is weaker than the theorem that every convergent real sequence is bounded. The weaker statement is chosen because it is closer to a small first formalization exercise.

Why this is a useful LLM test

Lean is not another language model. A proof assistant checks a formal proof against a formal statement. This can provide a much stronger kind of check than asking a second model for an opinion. However, the formal statement must faithfully represent the intended mathematical statement, and the file must actually compile in the chosen Lean and mathlib environment [52, 53, 54, 55].

In these notes, the following Lean block is not compiled. It is therefore only a draft sketch, not a verified formal proof.

Initial prompt

Draft a Lean theorem for the following elementary statement:
For every real number L , the constant sequence $x_n = L$ is bounded.

Requirements:

1. Keep the theorem modest.
2. State the formal theorem explicitly.
3. Do not claim that the code is verified unless it compiles.
4. Explain what informal statement the formal theorem represents.
5. Mark any imports or tactics that must be checked locally.

A plausible model-output pattern

A useful model response may propose an import, a theorem statement using $\mathbb{R}$, an existential bound $|L| + 1$, and a short proof using nonnegativity of absolute value. A defective response may state a theorem about all convergent sequences without defining convergence, or may claim that Lean has verified the proof even though no file has been compiled.

What should be checked

1. Does the formal statement match the intended informal theorem?
2. Is the theorem about the constant sequence, not all convergent sequences?
3. Are the imports correct for the local Lean/mathlib version?
4. Does the code compile?
5. Does the proof rely on a tactic that is unavailable in the local environment?

Optional draft Lean sketch, not verified here

The following is a pseudo-Lean-style draft. It is intentionally marked as a draft and has not been checked as part of these notes. The ASCII words and symbols such as `exists`, `forall`, `/\`, and `<=` should be treated as placeholders for the exact Lean 4 syntax accepted by the installed environment. The file must be checked in a real Lean project before being called a formal verification.

```
import Mathlib

example (L : Real) :
  exists M : Real, 0 < M /\ forall n : Nat,
    abs ((fun _ : Nat => L) n) <= M := by
  refine ⟨abs L + 1, ?_, ?_⟩
  · have hnonneg : 0 <= abs L := abs_nonneg L
    linarith
  · intro n
    change abs L <= abs L + 1
    linarith
```

This sketch formalizes only a boundedness statement for a constant real sequence. It does not formalize the theorem that every convergent sequence is bounded. To formalize the latter, one must choose exact definitions of sequence convergence and boundedness in the Lean environment and then prove that the formal theorem matches the intended textbook theorem.

Proof-audit prompt

Audit the Lean draft below.

Tasks:

1. State in ordinary mathematical language what theorem is formalized.
2. Check whether this matches the intended informal theorem.
3. Identify imports, tactics, or names that must be verified locally.
4. Do not claim Lean verification unless the file compiles.
5. If the statement is too weak or too strong, propose a corrected formal statement before attempting a proof.

Key lesson

Formal verification checks the formalized statement. A local model can help draft a formalization, but the formal proof system and the human reader have distinct roles. Namely, Lean checks the formal proof, while the human checks whether the formal statement is the right theorem.

Chapter 9

Limitations, Hallucinations, and Verification

In mathematics, hallucination need not look bizarre. It may look like a plausible definition, a plausible theorem citation, a plausible but false application of a standard result, or a polished proof step that is simply invalid. The literature on mathematical reasoning and factuality in large language models gives useful context for this cautious position [64, 63, 65]. The operational rule in these notes is simpler than any benchmark. Namely, a model answer is just a candidate object for inspection.

Local LLMs are useful mathematical instruments, but they are not mathematical authorities. Their outputs should be treated as conjectural until verified.

9.1 Common mathematical failure modes

The following table summarizes recurring failures.

Failure mode	Typical model pattern	Why it is dangerous	Basic check
Definition drift	Replaces the stated definition by a nearby convention.	The proof may solve a different problem.	Compare each use of a definition with the problem statement.
Hidden compactness	Invokes a minimum, maximum, or convergent subsequence without compactness or a replacement hypothesis.	Existence claims may fail on noncompact domains.	Identify the set and the exact existence theorem being used.

Failure mode	Typical model pattern	Why it is dangerous	Basic check
Hidden convexity	Treats a critical point or local minimizer as global.	Stationarity is not global optimality outside special classes.	Locate where convexity or another global principle enters.
Direction error	Reverses an inequality or loses a sign condition.	The logical implication may disappear.	Test the inequality at special points and check signs.
Smoothness inflation	Differentiates more times than the assumptions allow.	The proof may use derivatives that do not exist.	Count differentiability assumptions before every derivative.
False theorem invocation	Names a theorem but omits its hypotheses.	The theorem label creates an illusion of rigor.	State the theorem and verify hypotheses line by line.
Local-global confusion	Uses a local estimate as a global conclusion.	Neighborhood statements do not automatically extend to the full domain.	Audit the quantifiers: local, global, pointwise, uniform.
Numerical evidence as proof	Treats plots, grids, or simulations as theorem proofs.	A finite grid, plot, or uncertified numerical simulation, by itself, cannot certify an infinite universal statement.	Separate conjecture generation from proof.

9.2 Definition drift

Definition drift occurs when the solution begins with the stated definition but then substitutes a different one. This is common when several nearby conventions exist.

Example: the Polyak–Łojasiewicz inequality

Suppose the problem states the global Polyak–Łojasiewicz condition as

$$\frac{1}{2}\|\nabla f(x)\|^2 \geq \mu(f(x) - f^*), \quad x \in \mathbb{R}^d,$$

where $\mu > 0$ and $f^* = \inf f$. This direction is essential. If $\nabla f(x) = 0$, then

$$0 \geq \mu(f(x) - f^*).$$

Since $f(x) - f^* \geq 0$, it follows that $f(x) = f^*$. Thus every critical point is a global minimizer under this condition.

A common drift is to write instead

$$f(x) - f^* \geq \mu \|\nabla f(x)\|^2.$$

At a critical point this gives only $f(x) - f^* \geq 0$, which is true by definition of the infimum. The central conclusion is lost. A proof using the reversed inequality is not a proof of the intended statement. For standard optimization terminology around the PL condition, see [69].

Example: smoothness conventions

In optimization, an L -smooth differentiable function usually means that ∇f is L -Lipschitz:

$$\|\nabla f(x) - \nabla f(y)\| \leq L\|x - y\|.$$

For $f \in C^2(I)$ on an interval $I \subset \mathbb{R}$, the bound

$$\sup_{x \in I} |f''(x)| \leq L$$

implies that f' is L -Lipschitz on I . Conversely, if f' is L -Lipschitz and $f''(x)$ exists at a point x , then $|f''(x)| \leq L$. Thus second-derivative arguments require hypotheses that justify both the second derivative and the needed bound. If the problem assumes only C^1 , a proof cannot freely use f'' .

Audit prompt

Audit the solution for definition drift. List every definition used in the argument. For each definition, compare it with the exact definition in the problem statement. Identify any place where the solution silently replaces the stated definition by a stronger, weaker, or different one.

9.3 Hidden compactness assumptions

Many elementary existence statements require compactness, or a substitute such as coercivity. A model may remember a conclusion but omit the hypothesis.

Incorrect pattern

A typical invalid sentence is:

Since f is continuous on $\mathbb{R}$, it attains a global minimum.

This is false. The function $f(x) = e^x$ is continuous on $\mathbb{R}$, has infimum 0, and does not attain this infimum at any real point.

A correct compactness statement is different: a continuous function on a compact set attains its minimum and maximum. On $\mathbb{R}^d$, one needs additional information. For instance, a continuous coercive function $f : \mathbb{R}^d \rightarrow \mathbb{R}$, with $f(x) \rightarrow +\infty$ as $\|x\| \rightarrow \infty$, does attain a global minimum. The proof reduces the search to a sufficiently large compact ball. Coercivity cannot be inserted silently.

Sequence version

A model may also write that every sequence has a convergent subsequence. This is false. The sequence $x_n = n$ in $\mathbb{R}$ has no convergent subsequence. In $\mathbb{R}^d$, boundedness is the key additional hypothesis. The word “bounded” is not a detail; it is the compactness mechanism.

Audit prompt

Find every existence claim in the proof: minimum, maximum, convergent subsequence, optimizer, fixed point, or limit point. For each one, identify the compactness, coercivity, completeness, boundedness, closedness, or other hypothesis that justifies it. If no such hypothesis is present, mark the claim as unsupported.

9.4 Hidden convexity assumptions

Convexity is a global geometric assumption. When it is present, first-order conditions can imply global conclusions. When it is absent, the same first-order conditions may say very little.

Stationary does not imply globally minimal

The function

$$f(x) = x^3$$

is differentiable on $\mathbb{R}$, and $f'(0) = 0$. The point 0 is the only critical point. It is not a local minimum and not a global minimum. The function is unbounded below. Thus the statement “a critical point is a global minimum” is false without additional hypotheses.

What convexity adds

If $f : \mathbb{R}^d \rightarrow \mathbb{R}$ is differentiable and convex, then

$$f(y) \geq f(x) + \langle \nabla f(x), y - x \rangle, \quad x, y \in \mathbb{R}^d.$$

If $\nabla f(x^*) = 0$, then $f(y) \geq f(x^*)$ for every y . Hence x^* is a global minimizer. The global conclusion comes from convexity, not from stationarity alone.

Audit prompt

Audit the proof for hidden convexity. Identify every step where the argument passes from a critical point, local minimum, or first-order condition to a global conclusion. For each step, state whether convexity or another global hypothesis is explicitly assumed. If not, provide a counterexample or mark the step invalid.

9.5 Direction errors in inequalities

The direction of an inequality is part of its meaning. A reversed inequality may look similar on the page while losing the implication needed in the proof.

PL as a diagnostic example

For the PL inequality

$$\frac{1}{2}\|\nabla f(x)\|^2 \geq \mu(f(x) - f^*),$$

substituting a critical point gives useful information. For the reversed inequality, substituting a critical point gives only a tautology. This simple check detects the error immediately.

Sign errors under multiplication and division

If an argument multiplies an inequality by a quantity whose sign is unknown, the direction may change or the step may be invalid. For example, from $a \leq b$ one may conclude $ca \leq cb$ only when $c \geq 0$. If $c < 0$, the direction reverses. If the sign of c has not been established, the step is unsupported.

Monotonicity does not imply boundedness

A model may write that a sequence is increasing and therefore bounded above. This is false. The sequence $a_n = n$ is increasing and unbounded above. Monotonicity plus boundedness can imply convergence for real sequences, but monotonicity alone is not enough.

Audit prompt

Audit all inequalities in the proof. For each important inequality, state which side controls which quantity, whether any multiplication or division uses a sign condition, and whether the direction is preserved under every operation. Test the inequality at special points when possible.

9.6 Smoothness inflation

Smoothness inflation occurs when a proof uses more differentiability than the problem assumes. This is common because many familiar arguments are easier in C^2 form than in C^1 form.

Using second derivatives under a C^1 assumption

Assume only that $f : \mathbb{R} \rightarrow \mathbb{R}$ is C^1 . A proof cannot freely use f'' . The function

$$f(x) = x|x|$$

is C^1 , with $f'(x) = 2|x|$, but f'' does not exist at 0. Any argument that requires $f''(0)$ is invalid for this class.

Taylor expansion without hypotheses

A formula such as

$$f(x+h) = f(x) + f'(x)h + \frac{1}{2}f''(\xi)h^2$$

requires hypotheses. If only first differentiability or continuity is assumed, this expression cannot be introduced without proof or an additional assumption. A polished Taylor expansion can hide a serious logical error.

Audit prompt

Audit the proof for smoothness inflation. List every derivative, Taylor expansion, Hessian, Lipschitz-gradient statement, and differentiability-based theorem used in the proof. For each item, state the exact smoothness assumption needed and whether it appears in the problem statement.

9.7 False theorem invocation

A model may cite a real theorem but apply it in a setting where the hypotheses are not satisfied. This is especially dangerous because theorem names create a surface appearance of rigor.

Intermediate value theorem

The sentence “by the intermediate value theorem, there exists a zero” is valid only after the relevant hypotheses have been checked: continuity on an interval and values with opposite signs, or a target value lying between two function values. Without these hypotheses, the theorem name contributes nothing.

Contraction mapping theorem

The contraction mapping theorem requires a complete metric space and a map with Lipschitz constant strictly less than one. Continuity is not enough. Nonexpansiveness with constant one is not enough. If a model verifies only continuity and then invokes contraction, the invocation is invalid.

Plausible citations

A mathematical hallucination may also be bibliographic: a plausible title, a plausible author list, a theorem number that does not exist, or a source that supports a nearby but different statement. This is why the project maintains a source registry and a manual bibliography. A citation must support the exact claim for which it is used.

Audit prompt

Audit every theorem invocation. For each invoked theorem, write:

- (1) the theorem name, if any;
- (2) the exact hypotheses needed;
- (3) where each hypothesis is verified in the proof;
- (4) the exact conclusion obtained.

If any hypothesis is missing, mark the invocation invalid.

9.8 Local-global confusion

Local-global confusion occurs when a statement proved in a neighborhood is used as if it held on the whole domain.

Local minimum need not be global

Let

$$f(x) = x^2 - x^3.$$

Then $f(0) = 0$. If $|x| < 1/2$, then

$$f(x) = x^2(1 - x) \geq \frac{1}{2}x^2 \geq 0 = f(0).$$

Thus 0 is a local minimizer. It is not a global minimizer, since

$$f(2) = 4 - 8 = -4 < 0 = f(0).$$

Indeed, $f(x) \rightarrow -\infty$ as $x \rightarrow +\infty$.

Local Lipschitz need not be global Lipschitz

The function $f(x) = x^2$ is locally Lipschitz on $\mathbb{R}$, but it is not globally Lipschitz on $\mathbb{R}$. If $|x^2 - y^2| \leq L|x - y|$ held for all x, y , then taking $y = 0$ would give $x^2 \leq L|x|$ for all x , impossible for large $|x|$.

Quantifier audit

A proof audit should distinguish:

- local: there exists a neighborhood where the property holds;
- global: the property holds on the full stated domain;
- pointwise: a constant or index may depend on the point;
- uniform: one constant or index works for all points in the stated set.

A model-generated proof that blurs these distinctions should be rejected or rewritten.

9.9 Numerical evidence mistaken for proof

Numerical computation is useful for generating conjectures and finding errors. It is not, by itself, a proof of a universal mathematical statement.

Symbolic derivative versus numerical check

For

$$f_a(x) = x^2 + a \sin^2 x,$$

a finite-difference calculation may suggest the derivative formulas. The proof is symbolic:

$$f'_a(x) = 2x + a \sin(2x), \quad f''_a(x) = 2 + 2a \cos(2x).$$

A numerical check at finitely many points is a diagnostic for mistakes, not a replacement for differentiation.

Finite grid versus global nonnegativity

A plot may suggest that $g(x) \geq 0$ for all $x \in \mathbb{R}$. A finite grid verifies only finitely many values and may miss a narrow negative region. This limitation can be made exact. Fix an integer $N \geq 1$, set

$$G_N = \left\{0, \frac{1}{N}, \frac{2}{N}, \dots, 1\right\}$$

and define

$$q_N(x) = - \prod_{k=0}^N \left(x - \frac{k}{N} \right)^2.$$

Then $q_N(k/N) = 0$ at every grid point, while $q_N(x) < 0$ for every $x \in [0, 1] \setminus G_N$. The grid therefore completely misses the strict negativity between grid points. A proof of global nonnegativity needs an argument covering all points, such as a factorization, monotonicity analysis, convexity argument, interval estimate, or another rigorous method.

Disciplined computational workflow

A disciplined use of computation has the following form:

1. state the conjecture suggested by the computation;
2. record the code, parameter values, software environment, and output;
3. identify exactly which finite cases were checked;
4. state what remains unproved;
5. provide a separate proof or counterexample.

The phrase “the plot proves” should be avoided. The phrase “the plot suggests” is usually more accurate.

9.10 Verification checklist

The following checklist should be used after a model produces a candidate solution. It is intentionally detailed because many errors are small local moves inside a plausible global narrative.

A. Statement and assumptions

1. Is the theorem statement copied correctly?
2. Are the domain, codomain, and quantifiers correct?
3. Are all assumptions used exactly as stated?
4. Did the model add compactness, convexity, boundedness, differentiability, measurability, independence, or positivity assumptions?
5. Did the model weaken the conclusion without saying so?

B. Definitions

1. Are all relevant definitions stated explicitly?
2. Is each definition used in the same form throughout the proof?
3. Are there multiple conventions for the term?
4. Does the proof depend on a convention different from the problem statement?

C. Algebra and inequalities

1. Are displayed formulas algebraically correct?
2. Are signs and inequality directions correct?
3. Is division by a possibly zero quantity avoided?
4. Is multiplication by a negative quantity handled correctly?
5. Are limiting operations justified?

D. Theorem invocations

1. Which theorems are invoked?
2. Are their hypotheses stated?
3. Are their hypotheses verified in the current problem?
4. Is the conclusion exactly what the proof uses?
5. Is the theorem applied globally when it is only local, or uniformly when it is only pointwise?

E. Examples and counterexamples

1. Does a proposed counterexample satisfy all assumptions?
2. Does it actually violate the claimed conclusion?
3. Are derivatives, limits, and extrema in the counterexample computed correctly?
4. Is the example defined on the required domain?

F. Computation

1. Is the computation symbolic, numerical, or both?
2. If numerical, what finite set of cases was actually checked?
3. Are floating-point errors or grid effects relevant?
4. Is computation used only as evidence, or incorrectly as proof?
5. Is the code recorded sufficiently for reproduction?

G. Writing and responsibility

1. Does the final write-up distinguish model suggestions from verified arguments?
2. Are unsupported citations removed or marked?
3. Are all remaining gaps explicitly listed?
4. Has a human checked the proof line by line?
5. If formal verification is claimed, has the formal statement been checked, and does it faithfully represent the informal theorem?

9.11 The human mathematician as final arbiter

A useful local model can make mathematical work faster. However, it can also make a mistake look more polished. Hence, the right response is to place them in a disciplined workflow where their outputs are inspected, challenged, corrected, and documented. The final arbiter of a mathematical proof still the human mathematician reading the arguments, or a formal proof system checking a faithful formalization. The second part of this statement matters. A proof assistant such as Lean checks only a formalized theorem. If the formal statement is weaker than the intended theorem, or if an assumption is accidentally added during formalization, then the formal proof may be correct but irrelevant to the original mathematical question. Formal verification is a stronger standard than model agreement, but it still requires faithful translation into the formal language [66, 53, 54].

For ordinary work with local LLMs, the practical conclusion is:

1. use models to generate ideas,
2. use independent models to criticize ideas,
3. use computation to test ideas,
4. use $\text{\LaTeX}$ to make corrected arguments readable,

5. use formal verification where it is appropriate and feasible,
6. never confuse any of these steps with unexamined mathematical truth.

Chapter 10

Student Projects and Assignments

This chapter gives course projects that can be run with free local or open-weight models. The projects are meant to train disciplined mathematical use, such as precise prompting, careful logging, independent checking, and separation of useful model output from verified mathematical proof.

All projects should be carried out locally, for example through the local setup described earlier in these notes. Exact command syntax and model metadata should be checked against the local installation and the relevant documentation before the course session [1, 18].

General rules for all projects

For every model interaction, students should keep a reproducibility record. At minimum, record the following fields.

Field	What to record
Model name	The displayed model name used in the local tool.
Model tag/version	The exact tag or version string, if available. If unavailable, write “not visible from local interface”.
Quantization	The local quantization label, if visible, for example a Q4, Q5, Q8, or FP16 variant. Do not infer it from memory.
Context length	The configured or reported context length, if known. If it is not known, write “not recorded”.
Prompt	The exact prompt submitted to the model.
Temperature	The temperature or sampling setting, if known. If it is not known, write “not recorded”.
Output summary	A short summary of the answer; do not paste pages of unexamined output into the report.
Correctness assessment	The student’s mathematical judgment after checking the answer.

Field	What to record
Human correction	The corrected proof, counterexample, code, or exposition produced after verification.

A report should explain how the answer was checked. A useful model output may be wrong in its final conclusion but still suggest a good counterexample or a missing hypothesis. A simple assessment distribution for these projects is:

- mathematical correctness and verification: 40%,
- reproducibility record: 25%,
- quality of comparison and diagnosis: 20%,
- clarity of writing and L^AT_EX presentation: 15%.

The verification rule for all projects is the same: a model output, a plot, a timing number, a compiled L^AT_EX file, or a formal-looking proof is not accepted until the mathematical claim has been checked. Each project below also names a project-specific hazard.

10.1 Project 1: Compare three local models on elementary analysis

Objective

Compare three local models on a small set of elementary analysis tasks. The goal is not to produce a leaderboard. The goal is to observe how different models handle assumptions, quantifiers, counterexamples, and proof auditing.

Required tools

- A working local LLM setup.
- Three locally runnable models or model variants. They may be from three different families, or from one family in different sizes or quantizations.
- A plain text editor or L^AT_EX editor for the final report.

Task description

Use the same prompts for all three models. Run each prompt in a fresh conversation, unless the instructor explicitly asks for a multi-turn experiment. The following three tasks are suitable.

1. Prove that every convergent sequence in $\mathbb{R}$ is bounded.
2. Decide whether the following claim is true: if a differentiable function $f : \mathbb{R} \rightarrow \mathbb{R}$ has exactly one critical point, then that point must be a global minimum.
3. Prove that if $f : \mathbb{R}^d \rightarrow \mathbb{R}$ is differentiable and convex and $\nabla f(x^*) = 0$, then x^* is a global minimizer.

For each task, first ask for a direct answer. Then ask for an audit of that answer. For example:

Audit your previous answer.
Identify hidden assumptions, missing cases,
incorrect theorem invocations, and places where
local conclusions are treated as global conclusions.

Do not repair the output before recording it. The first output is part of the data.

Deliverable

Submit a short report containing:

- the reproducibility record for each model;
- a comparison table with one row per model and task;
- a corrected human solution or counterexample for each task;
- a paragraph explaining which model output was most useful and why;
- a paragraph explaining which output was most dangerous and why.

Assessment criteria

The report is assessed on the accuracy of the corrected mathematics, the precision of the model logs, and the student's ability to diagnose errors. A model should not receive a high evaluation merely because its answer was long or confident.

Warning about verification

Elementary tasks are deliberately used because their answers can be checked by the student. If a model fails on an elementary theorem, this is not a reason to discard local models completely. It is evidence that verification is not optional. If a model succeeds on these tasks, this is not evidence that it can be trusted on harder mathematics.

10.2 Project 2: Compare Q4 and Q8 quantization

Objective

Study whether two quantizations of a local model behave differently on a fixed mathematical task. The purpose is to understand an experimental protocol, not to prove a universal law about quantization.

Required tools

- Two local variants that differ primarily in quantization, if such variants are available on the student's machine.
- The same prompt file for both runs.
- Optional timing notes, recorded informally and without overinterpreting them.

Task description

Choose one mathematical task and one audit task. For instance:

Problem. Determine for which parameters $a \geq 0$ the function $f_a(x) = x^2 + a \sin^2(x)$ is convex on $\mathbb{R}$. Give a complete proof.
Optional extension: repeat the problem for all real a .
Do not use numerical evidence as a proof.

Then use the audit prompt:

Audit the proof.
Check the derivatives.
Check the range of every trigonometric expression used.
Check the quantifiers over a .
Check whether numerical evidence is mistaken for proof.

Run the same prompts on the two quantizations. Keep all other visible settings as similar as possible. The human-verified answer for the displayed range is $0 \leq a \leq 1$. If the optional all-real extension is used, the answer is $-1 \leq a \leq 1$, because the sign of a changes which endpoint of $[-1, 1]$ is relevant for $\cos(2x)$. If the two local variants are not actually the same base model, state this explicitly. In that case the experiment is not a pure quantization comparison; it is a comparison of two local variants.

Deliverable

Submit a report with:

- exact model names and quantization labels as displayed locally;
- the exact prompts;
- a side-by-side summary of the outputs;
- a human-verified solution of the mathematical task;
- a discussion of whether the higher-precision variant was more stable, equally useful, or less useful in this particular experiment.

Assessment criteria

Credit is given for experimental control, correct mathematical checking, and honest interpretation. It is better to write “the experiment is inconclusive” than to manufacture a ranking from weak evidence.

Warning about verification

A Q8 model is not a proof checker, and a Q4 model is not automatically mathematically unreliable. Higher quantization precision may preserve more numerical detail in model weights, but mathematical correctness still depends on the generated argument and the subsequent verification. One experiment does not prove general superiority.

10.3 Project 3: Audit a flawed proof

Objective

Practice proof auditing by locating the first invalid step in a plausible but wrong argument. The model may be used as a second reader, but the final audit must be the student’s own mathematical judgment.

Required tools

- One local reasoning or instruction model.
- The flawed proof supplied below, or another proof supplied by the instructor.
- A document editor for an annotated solution.

Task description

Consider the following false claim.

If a differentiable function $f : \mathbb{R} \rightarrow \mathbb{R}$ has exactly one critical point, then that critical point is a global minimum.

Here is an intentionally flawed proof.

Let x_0 be the unique critical point. Since $f'(x_0) = 0$ and no other critical point exists, the derivative cannot vanish anywhere else. Hence f' has one sign on $(-\infty, x_0)$ and the opposite sign on (x_0, ∞) . Therefore f decreases before x_0 and increases after x_0 . Thus x_0 is a global minimum.

Ask a local model to audit the proof. A suitable prompt is:

Audit the following proof.

Do not rewrite it.

Find the first invalid step.

For each step, state whether it follows from the assumptions.

If the claim is false, give a concrete counterexample and verify it.

The student must then write an independent audit. The proof fails when it turns “ f' does not vanish away from x_0 ” into a specific global sign pattern. In particular, uniqueness of a critical point does not imply that f' is negative on one side and positive on the other. A concrete counterexample is

$$f(x) = x^3.$$

Then $f'(x) = 3x^2 > 0$ for every $x \neq 0$, so the only critical point is 0, but the derivative has the same sign on both sides of 0. Hence 0 is not a global minimum; the function is unbounded below.

Deliverable

Submit:

- the model’s audit summary;
- an annotated version of the flawed proof;
- the first invalid step;
- the counterexample and its verification;
- a short comparison between the model audit and the student’s audit.

Assessment criteria

The central criterion is whether the student identifies the first invalid step and verifies the counterexample. Additional credit is given for recognizing why the invalid step is tempting: it resembles a true first-derivative test but uses hypotheses that have not been established.

Warning about verification

A model may correctly say that the claim is false while giving a weak or incorrect explanation. Conversely, it may produce a persuasive repair of the flawed proof rather than rejecting it. The student must check the actual logic, not the tone of the answer.

10.4 Project 4: Local LLM plus Python

Objective

Use a local model to help design a small Python experiment, while keeping numerical evidence separate from proof. Python is used here as a tool for exploration and visualization, not as a substitute for mathematical argument [49].

Required tools

- A local model capable of producing or revising simple Python code.
- A Python environment with standard numerical and plotting tools, as chosen by the instructor.
- Optional SageMath if the instructor wants exact symbolic support [51].

Task description

Use the family

$$f_a(x) = x^2 + a \sin^2 x$$

as the experimental object. Ask the model to write Python code that:

- plots f_a for several values of a ;
- plots or samples the second derivative $f_a''(x) = 2 + 2a \cos(2x)$;
- searches numerically for values of x where $f_a''(x) < 0$ for a chosen value of a ;
- prints a warning that the plot is not a proof.

A suitable prompt is:

```
Write simple Python code to explore
f_a(x)=x^2+a sin^2(x).
The code should plot f_a.
It should sample the second derivative for several values of a.
Include comments saying that numerical plots are not proofs.
Do not claim a theorem from the plot.
```

After running or inspecting the code, the student must give the exact mathematical argument for the convexity condition in the range considered. For example, when $a \geq 0$, convexity is equivalent to

$$2 + 2a \cos(2x) \geq 0 \quad \text{for all } x \in \mathbb{R}.$$

Because $-1 \leq \cos(2x) \leq 1$, the worst case for $a \geq 0$ is $\cos(2x) = -1$, so the condition becomes $a \leq 1$. Hence, under the standing assumption $a \geq 0$, convexity holds if and only if $0 \leq a \leq 1$, as in Example 8.5. If all real parameters are allowed, one must split according to the sign of a : for $a \geq 0$ the worst case is $\cos(2x) = -1$, giving $a \leq 1$, while for $a < 0$ the worst case is $\cos(2x) = 1$, giving $a \geq -1$. Thus, over all real a , convexity holds if and only if $-1 \leq a \leq 1$.

Deliverable

Submit:

- the prompt used to obtain or revise the code;
- the code file or a short code listing;
- a short description of the numerical output;
- a proof or counterexample that is independent of the plot;
- a paragraph stating exactly what the numerical experiment did and did not establish.

Assessment criteria

The code should be readable and relevant, but the main assessment is the mathematical separation between exploration and proof. A correct plot with an incorrect theorem receives little credit. A simple plot accompanied by a correct proof receives high credit.

Warning about verification

A finite grid can miss behavior between grid points. A model-generated program can also contain syntax errors, plotting mistakes, or a wrong derivative. The student must check both the code and the mathematics.

10.5 Project 5: Local LLM plus L^AT_EX

Objective

Use a local model as a writing assistant after the mathematics has already been verified. The aim is to improve exposition, not to delegate proof checking. L^AT_EX syntax and package behavior should be checked against documentation or against successful compilation [50].

Required tools

- One local instruction model or editor-role model.
- A corrected mathematical proof written by the student.
- A local L^AT_EX compiler, if available.

Task description

Choose one corrected proof from another project. Give the model a restricted rewriting task. For example:

```
Rewrite the proof below in clean LaTeX.
Do not change the mathematical statement.
Do not change assumptions, quantifiers, or notation.
Preserve every logical step.
If you think a mathematical correction is needed,
do not make it silently.
Write it as a separate comment after the LaTeX draft.
```

The student must then compare the model's L^AT_EX version with the verified proof line by line. The audit should look for changed quantifiers, missing hypotheses, altered notation, and new claims.

Deliverable

Submit:

- the original verified proof;
- the prompt given to the model;

- the model-generated $\text{\LaTeX}$ draft;
- the final student-edited $\text{\LaTeX}$ proof;
- a list of any mathematical or notational changes made by the model.

Assessment criteria

The final proof should compile and should preserve the verified mathematics. Readable exposition matters, but not at the cost of changing the theorem. Students receive credit for detecting and reversing model edits that alter the mathematical content.

Warning about verification

A polished $\text{\LaTeX}$ proof can be wrong. A model may replace “there exists” by “for every”, omit a boundedness assumption, or strengthen a conclusion. The edited document must be checked as mathematics, not only as typography.

10.6 Project 6: Optional local LLM plus Lean

Objective

Explore the difference between informal proof assistance and formal proof checking. Lean can check a formalized statement, but only the formalized statement. If the formalization does not faithfully represent the intended mathematics, the verification is not evidence for the informal theorem [52, 53, 54].

Required tools

- A working Lean environment supplied by the instructor, or a local Lean installation.
- One local model used only as an assistant for drafting or explaining Lean code.
- A very small theorem chosen by the instructor.

Task description

Choose a modest theorem. Suitable examples include a simple transitivity statement for real numbers, a basic algebraic identity already supported by an appropriate tactic, or a simple proposition from propositional logic. Avoid turning this project into a large formalization of analysis unless the course has already prepared the necessary Lean background.

A suitable model prompt is:

I want to formalize a very small theorem in Lean 4.
Please propose a Lean statement and explain each line.
Do not claim that the code is verified unless it has
been compiled in my local Lean environment.
Keep the theorem elementary.

The student then attempts to compile the code locally. If it compiles, record the Lean version, imports, command used, and theorem statement. If it does not compile, submit the error message and an explanation of what was learned.

Deliverable

Submit:

- the informal theorem;
- the proposed Lean statement;
- the model prompt and output summary;
- the exact Lean code attempted;
- the compilation result;
- a short discussion of whether the Lean statement matches the informal theorem.

Assessment criteria

This optional project is assessed for modest scope, accurate logging, and faithfulness of formalization. A non-compiling attempt may still receive credit if the student records the attempt carefully and diagnoses the gap.

Warning about verification

Do not write “Lean verified the theorem” unless the exact code compiled in the recorded environment. Do not write “the intended theorem was verified” unless the formal statement was checked for faithfulness to the informal statement.

10.7 Final report template

Each project report should be concise, but it must contain enough information for another student or instructor to understand what was done. The following structure can be copied into a $\text{\LaTeX}$ document.

`\section*{Project title}`

`\subsection*{Objective}`

State the mathematical and workflow objective.

`\subsection*{Local setup}`

List hardware briefly if relevant.

List the local LLM interface used.

Do not include private information.

`\subsection*{Model record}`

For each model or variant, record:

- model name,
- model tag/version,
- quantization,
- size on disk if relevant and visible,
- context length if known,
- temperature or sampling settings if known,
- date of run.

`\subsection*{Prompts}`

Paste the exact prompts used.

If a prompt was revised, number the versions.

`\subsection*{Model output summary}`

Summarize each output.

Do not paste long unverified transcripts
unless the assignment asks for them.

`\subsection*{Mathematical verification}`

Give the corrected proof, counterexample,
code check, or formalization result.

State explicitly what has been proved
and what has not been proved.

`\subsection*{Error analysis}`

Identify hidden assumptions, definition drift,
inequality direction errors,
unsupported theorem invocations,
or numerical-overclaiming.

`\subsection*{Human correction}`

State what the student changed after verification.

```
\subsection*{Final assessment}
```

Explain what the model was useful for.
 Explain what it failed to do.
 State what should be changed in the next run.

For comparison projects, the following worksheet may be used.

Worksheet field	Student entry
Model name	
Model tag/version	
Family	
Quantization	
Size on disk	
Context length	
Prompt	
Task type	Explanation, proof search, counterexample search, proof audit, Python/SageMath coding, Lean assistance, LaTeX exposition, or editing.
Temperature	if known
Output summary	
Correct conclusion	
Main error	
Useful insight	
Human correction	
Final assessment	

Transition to the tool-support chapters

Several projects above refer to Python, Sage, $\text{\LaTeX}$, or Lean. The next three chapters are optional support chapters for those tools. They are not new mathematical authorities; they describe how to use local models around computation, exposition, and formalization while preserving the same verification discipline.

Chapter 11

Numerical Evidence, Python, and Sage

Local LLMs can be useful when a mathematical question has a computational component. They can suggest experiments, write first versions of small programs, explain error messages, or translate a formula into code. This is valuable, but it does not change the logical status of computation. A plot, a finite grid, or a simulation may suggest a conjecture. It is not, by itself, a proof of a universal mathematical statement.

The role of this chapter is narrow. It explains how to use local models as assistants for computational exploration while preserving the boundary between numerical evidence and proof. This chapter applies the central principle to computation: a local model may help produce code, plots, or symbolic checks, but the logical status of the result is decided by mathematics, not by the model output.

11.1 Numerical evidence as conjecture generation

A numerical experiment is often a good way to decide what to try next. For example, before proving a statement about a one-variable function, it may be reasonable to inspect derivatives, evaluate the function on a grid, or plot the candidate expression for several parameter values. This may reveal a sign change, a likely threshold, or a suspicious exceptional case.

The mistake is to confuse this exploratory role with proof. A finite grid checks finitely many points. A theorem about all real numbers, all sufficiently large integers, all matrices in a class, or all functions satisfying a hypothesis is not established by checking finitely many sampled cases unless an additional rigorous argument reduces the infinite problem to those cases.

A useful local-model prompt should therefore ask for two separate outputs:

I want to explore the following mathematical claim computationally.
Claim: [insert claim].

1. Suggest a small numerical experiment that may reveal possible behavior.
 2. State clearly what the experiment can and cannot prove.
 3. After the experiment, list the additional mathematical steps needed for a proof.
- Do not present numerical evidence as a proof.

The last sentence is not decorative. It changes the task. The model is not being asked to produce rhetorical confidence; it is being asked to separate experimental evidence from deductive verification.

11.2 Using Python for plots and sanity checks

Python is a convenient language for small computational experiments. For a course, students should run Python code in a controlled environment and record how the environment was created. The official Python documentation for virtual environments is a suitable starting point for reproducible local setups [49]. If students use array-based numerical code or plotting libraries, the relevant NumPy and Matplotlib documentation should be checked rather than inferred from a model response [70, 71].

The following code is an example only. It checks a derivative formula and a few sampled values for the family

$$f_a(x) = x^2 + a \sin^2 x.$$

It does not prove any global theorem about this family.

```
# Example only. Numerical output is not a proof.
import math
```

```
def f(x, a):
    return x*x + a*(math.sin(x)**2)
```

```
def fprime_formula(x, a):
    return 2*x + a*math.sin(2*x)
```

```
def finite_difference(x, a, h=1e-6):
    return (f(x+h, a) - f(x-h, a))/(2*h)
```

```
for x in [-2.0, -1.0, 0.0, 1.0, 2.0]:
    print(x, fprime_formula(x, 2.0), finite_difference(x, 2.0))
```

A local model may help explain why the formula

$$f'_a(x) = 2x + a \sin(2x)$$

is expected: the derivative of $\sin^2 x$ is $2 \sin x \cos x = \sin(2x)$. But the model's explanation must still be checked by the student. In particular, a generated program should be reviewed for wrong formulas, wrong parameter values, wrong intervals, silent type conversion, plotting the wrong function, and interpreting floating-point output as exact.

A better prompt than “plot this” is:

Write a short Python experiment for the function $f_a(x)=x^2+a*\sin(x)^2$. The goal is only exploratory.

Requirements:

1. Use clearly named functions.
2. Print the derivative formula used in the code.
3. Sample several values of a and x .
4. Add a warning in a comment that the computation is not a proof.
5. List possible mathematical questions suggested by the output.

This prompt asks for code and for interpretation discipline.

11.3 Using Sage for exact or symbolic support

SageMath can be useful when exact symbolic manipulation is more appropriate than floating-point computation. Its documentation should be used for installation, syntax, and mathematical-library behavior [51]. In these notes, Sage is not treated as a replacement for proof. It is treated as a mathematical computing environment that can help with algebraic simplification, exact arithmetic, and symbolic checks.

For example, Sage may simplify derivatives or solve elementary equations in cases where the symbolic problem is within its scope. A model may suggest Sage commands, but the student should verify both the syntax and the mathematical meaning of the output. A symbolic answer can still be misread. A command can solve a transformed problem rather than the original one. A solution may be conditional on assumptions that were not stated in the code.

A conservative Sage prompt is:

Suggest a SageMath check for the following symbolic derivative.
Function: $f_a(x)=x^2+a*\sin(x)^2$.

Requirements:

1. Keep a as a symbolic parameter.
2. Compute the first and second derivatives.
3. Explain which part is symbolic computation and which part still needs proof.
4. Do not infer global convexity only from a plot or from sampled values.

The important phrase is “which part still needs proof”. This prevents the computational step from being silently promoted to a theorem.

11.4 Worked example: plotting $x^2 + a \sin^2 x$

Consider again

$$f_a(x) = x^2 + a \sin^2 x.$$

A plot for several values of a may suggest that positive values of a make the function more curved near some points and less curved near others. The second derivative is

$$f_a''(x) = 2 + 2a \cos(2x).$$

For $a \geq 0$, this expression is bounded below by $2 - 2a$. Therefore, if $0 \leq a \leq 1$, the second derivative is nonnegative for all x , and f_a is convex. If $a > 1$, then $f_a''(\pi/2) = 2 - 2a < 0$, so f_a is not convex.

A plot may suggest this threshold, but the proof is the inequality $-1 \leq \cos(2x) \leq 1$ together with the second-derivative criterion in one real variable. The plot is useful only because it guides attention to the right expression.

A disciplined local-model workflow for this example is:

1. ask for symbolic derivatives;
2. check the derivatives independently;
3. ask for a numerical exploration over several values of a ;
4. ask the model to state what the numerical exploration does not prove;
5. write the proof using inequalities, not the plot;
6. audit the proof for hidden assumptions about a .

11.5 Reproducible computation logs

Every computational experiment used in a mathematical report should be logged. For these notes, a minimal computation log contains:

Field	What to record
Question	The mathematical question being explored.
Model	The local model name and tag, if visible.
Prompt	The exact prompt used to generate or revise the code.
Settings	Temperature, seed, and context length, if known.
Code	The exact code that was run.
Environment	Python, Sage, package, and operating-system information where relevant.
Output	The relevant numerical or symbolic output.
Interpretation	What the output suggests.
Boundary	What the output does not prove.
Human check	Corrections made after reading the mathematics and the code.

The boundary field is essential. Without it, a computation log easily becomes a rhetorical support for a conclusion that has not been proved.

11.6 Failure modes in generated code

Model-generated code fails in mathematically recognizable ways. Common patterns include:

- differentiating the wrong function;
- replacing a parameter by a fixed numerical value;
- sampling an interval that misses the important behavior;
- using floating-point arithmetic where exact arithmetic is needed;
- treating an optimization routine's output as a certified global optimum;
- omitting edge cases such as boundary points, zero parameters, or negative parameters;
- producing code that runs but answers a different question.

The student should therefore audit generated code like a proof: identify the statement being checked, verify the assumptions, inspect the transformations, and ask whether the output logically implies the claimed conclusion. In many cases, it will not. That is not a failure of the workflow. It is precisely why computation is treated as exploration unless it is embedded in a rigorous argument.

Chapter 12

LaTeX Writing and Manuscript Support

Local models can be useful LaTeX assistants. They can restructure text, convert informal notes into theorem-proof form, detect inconsistent notation, or prepare a first draft of an exposition. They should not be used to launder unchecked mathematics into polished prose. A wrong proof written in elegant LaTeX is still a wrong proof.

The LaTeX material in these notes is deliberately simple. The project uses standard document structure, theorem-like environments, tables, URLs, and verbatim code blocks. General LaTeX documentation should be consulted for language and package behavior [50]. For this course, a minimal and stable LaTeX style is preferable to a fragile style that depends on complex packages or shell escape.

12.1 What local models can help with in LaTeX

A local model may be useful for:

- converting rough notes into coherent paragraphs;
- formatting a verified statement as a theorem, proposition, example, or remark,
- rewriting a proof for readability after the proof has been checked,
- finding notation inconsistencies,
- producing a table from a fixed list of fields,
- translating an audit report into a revised proof plan,
- preparing a BibTeX or manual bibliography draft from verified meta-data.

The model should not be asked to “make the proof rigorous” unless the student also asks it to preserve the exact assumptions and to identify every change it made. The danger is that a model may silently strengthen hypotheses, add missing compactness, replace a definition, or remove a difficult case.

12.2 Definitions used verbatim

When rewriting mathematical text, definitions and assumptions should usually be copied exactly. A small linguistic change may be a mathematical change. For example, replacing “bounded derivative” by “Lipschitz derivative” is not a stylistic edit. Replacing “there exists a minimizer” by “the minimum is attained” may be harmless in some contexts, but it should still be deliberate.

A safe rewriting prompt is:

```
Rewrite the following proof in clear LaTeX prose.
Do not change the theorem statement, assumptions, definitions, notation,
or quantifiers. If a sentence seems mathematically wrong, mark it as
[needs mathematical review] instead of repairing it silently.
```

```
Text to rewrite:
[insert checked text]
```

The instruction “instead of repairing it silently” is important. Silent repair hides the audit trail.

12.3 Theorem, proof, example, and remark environments

The project skeleton defines simple theorem-like environments in `main.tex`. A model may use these environments, but it should not invent new package requirements unless explicitly asked. The intended pattern is:

```
\begin{proposition}
State the mathematical claim precisely.
\end{proposition}

\begin{proof}
Give the verified proof.
\end{proof}
```

For examples and warnings, use prose unless a formal environment is helpful. The aim is readability and checkability, not decorative formalism.

12.4 From audit notes to revised proof

A good LaTeX workflow separates four objects:

1. the original problem statement;
2. the model-generated draft;
3. the audit notes identifying gaps, false steps, and missing assumptions;
4. the revised proof written after the mathematics has been corrected.

The final proof should be based on the fourth object, not directly on the model output. A useful prompt is:

You are given audit notes for a mathematical proof.

Your task is to prepare a clean LaTeX proof only from the corrected argument.

Rules:

1. Do not add new lemmas unless they are explicitly stated in the audit notes.
2. Preserve all hypotheses exactly.
3. Mark any remaining gap as [gap remains].
4. At the end, list every mathematical change made from the draft.

This prompt makes the model an editor of a corrected argument rather than an authority about the theorem.

12.5 LaTeX output as a handoff artifact

For multi-stage workflows, LaTeX files are useful handoff artifacts. They are plain text, versionable, auditable, and easy to compare. A file-based workflow should record which stage produced each file, which sources were cited, which claims remain unsupported, and whether the document compiled.

A minimal handoff for a proof-writing stage should include:

- the modified `.tex` file,
- any bibliography entries added or removed,
- a statement of unresolved mathematical gaps,
- a statement of unresolved source issues,
- the build log if compilation failed,
- the compiled PDF if compilation succeeded.

12.6 Common LaTeX failure modes

Local models often produce LaTeX that looks plausible but fails in practice. Common failure modes include:

- mismatched math delimiters,
- undefined commands or theorem environments,
- package recommendations that are unnecessary or unavailable,
- broken tables that exceed the page width,
- fragile code-listing packages where `verbatim` would suffice,
- bibliography keys that are cited but not defined,
- invented bibliographic metadata,
- changes in notation introduced during rewriting.

Chapter 13

Lean and Formal Verification

Formal verification is different in kind from model agreement. A language model can produce a proof-like text. A second model can agree that the text looks correct. A formal proof system checks whether a precise formal object is accepted according to the rules of the system. *This is a stronger form of verification, but only for the statement that has actually been formalized.*

Remark 13.1. *Lean 4 is a theorem prover and programming language. The Lean project, the standard tutorial material, and the mathlib documentation are the appropriate sources for current usage and background [52, 53, 54, 66]. The mathlib repository is also relevant when a formalization depends on library theorem names, imports, and versioned declarations [55].*

13.1 Why formal verification is different from model agreement

Model agreement is a heuristic, while formal verification is a syntactic and logical check inside a formal system. If Lean accepts a proof, then Lean has checked a formal statement under the imported axioms, definitions, and library dependencies. However, this is not the same as saying that the original informal theorem was formalized faithfully.

Lean can check a formalized theorem. It cannot, by itself, guarantee that the formalized theorem is the theorem the author intended to prove.

This is why a Lean workflow has two verification tasks:

1. check that the formal proof is accepted by Lean,
2. check that the formal statement faithfully represents the informal mathematical claim.

The second task has mathematical and human nature. A model may help draft the formal statement, but it cannot be trusted to decide whether the formalization captures the intended theorem.

13.2 What Lean can and cannot do

In these notes Lean is optional and modest, and it is included to show the difference between proof-like text and formal checking, not to turn the lecture notes into a full proof-assistant course.

Lean can help with:

- checking small formalized statements,
- exposing missing hypotheses in a formal proof attempt,
- forcing exact quantifiers and definitions,
- showing students that informal obviousness is not the same as formal correctness.

Lean cannot verify an informal proof that has not been formalized. It cannot guarantee that a model-generated theorem statement is the intended theorem. And finally, it cannot remove the need to understand the mathematics behind the formal object.

13.3 Installing or accessing Lean

Exact installation instructions change over time and should be taken from the official Lean documentation rather than copied from an old handout [52]. The safest policy is:

- do not print stale installation commands as permanent facts,
- record the Lean version and project toolchain when a formal proof is checked,
- keep formalization exercises small enough that setup problems do not dominate the mathematics,
- distinguish between an installed local environment and a web or cloud environment if one is used.

13.4 Using mathlib

Many mathematical formalizations in Lean rely on mathlib. Library theorem names, import paths, and available lemmas are practical details that must be checked against the installed version [54, 55]. A local LLM may suggest a theorem name that is plausible but nonexistent, or a name that existed in a different version.

A good prompt for Lean assistance is not:

Prove this in Lean.

but:

Help me draft a Lean statement for the following elementary theorem. Do not claim that the code is verified. List all imports and theorem names that must be checked in my local Lean/mathlib version.

The phrase “must be checked” is essential. Generated Lean code should be treated like generated Python code, and it is useful if it compiles and represents the intended mathematics.

13.5 Example: formalizing a simple theorem

A suitable first Lean exercise should be very small. For example, one may first formalize a lemma extracted from the ordinary proof: if a real sequence converges to L , then for $\varepsilon = 1$ there exists N such that $|x_n - L| < 1$ for all $n \geq N$. A separate and larger formalization would then combine this tail estimate with a finite initial-segment bound to prove boundedness. A complete formalization may require definitions and library lemmas that depend on the chosen Lean setup.

The following block is a draft-style prompt, not a verified Lean proof.

Draft a Lean 4 formalization plan for the theorem:

"Every convergent real sequence is bounded."

Requirements:

1. State the theorem using the definition of convergence available in mathlib.
2. State the boundedness conclusion precisely.
3. List all imports and lemmas whose exact names must be checked locally.
4. Do not claim that the proof is verified unless Lean accepts it.

A student deliverable for this optional exercise should include either:

- a Lean file that has actually been checked locally, together with the Lean and mathlib versions; or
- a draft formalization plan explicitly labelled as not yet verified.

Anything else invites confusion between formal verification and formal-looking text.

13.6 LLMs as Lean assistants

Local models can help with Lean in the same way they help with mathematics more generally. So they can propose a structure, suggest imports, explain error messages, or rewrite an informal theorem into a possible formal statement. But they are not Lean. They do not check proofs unless they are connected to an actual Lean environment and the resulting file is accepted.

A useful Lean-assistance workflow is:

1. write the informal theorem carefully,
2. ask a model for a formalization plan,
3. manually inspect the formal statement,
4. test the file in Lean,
5. copy the exact error messages into a repair prompt,
6. accept only code that has been checked in the local Lean project,
7. document the final statement and any mismatch with the informal theorem.

This workflow uses the model as a translator and assistant. The checker remains Lean, and the mathematical responsibility remains with the human user.

13.7 When not to formalize

Formalization has a cost. Moreover, it is often not the right tool for a first exploration of a problem, for a quick counterexample search, or for a proof whose informal details are not yet understood. A local LLM plus Lean workflow is most useful when:

- the statement is small,
- the definitions are clear,
- the student can compare the formal statement with the informal one,
- the goal is to learn verification discipline rather than to outsource understanding.

Formalization is usually not useful when the user does not yet know what theorem is being proved. In that case, begin with ordinary mathematical work.

Chapter 14

Conclusion

These notes have treated local reasoning models as mathematical instruments. The word “instrument” is deliberate. An instrument can extend what a mathematician can try, compute, compare, or write. It does not remove the need for judgment.

14.1 What the human user should retain

1. A local LLM can be useful for exploration, proof search, proof audit, counterexample search, small code generation, LaTeX rewriting, and formalization support.
2. A fluent answer is not a proof.
3. Agreement between models is not a proof.
4. A plot is not a proof of a universal mathematical statement.
5. A reasoning model is not a proof checker.
6. A formal proof system can check a formalized statement, but the formalization must faithfully represent the intended theorem.
7. Exact model tags, licenses, commands, context lengths, and hardware behavior should be checked against current primary sources before a course run.

14.2 A recommended first independent project

A good first independent project should be small enough to verify by hand and rich enough to expose model failure modes. One recommended project is the following.

Choose three local models and test them on five elementary mathematical claims: two true statements, two false statements, and one statement whose answer depends on a missing hypothesis. For each response, record the prompt, model information, quantization, context length if known, output summary, correctness assessment, first error, useful insight, and human correction.

The purpose of this project is to learn how model behavior changes with task type and how human verification enters the workflow.

A minimal final report should include:

- the exact problem statements,
- the prompts used,
- model identifiers and local settings,
- a table of outcomes,
- at least one detailed proof audit,
- one corrected mathematical solution or counterexample,
- a short reflection on which model role was useful: explorer, skeptic, programmer, editor, or formalizer.

Appendix A

Ollama Command Reference

The public command line interface, model-library names, and application behavior can change. Before starting work with Ollama, verify the installed version with `ollama -version` and check command help locally. The command-sensitive material below is based on the official CLI documentation, Modelfile documentation, context-length documentation, and official command source where needed [1, 2, 3, 14].

A.1 Conventions

In this appendix, `modelname` denotes a local model identifier. It may be a name from the Ollama library, a model created from a Modelfile, or a Hugging Face GGUF identifier in the Hub pattern described below. The command examples are written in shell form. On Windows PowerShell the same commands are normally entered without the leading shell prompt. When a command includes angle brackets or placeholders, replace the placeholder first, but do not type the angle brackets literally.

A.2 Version and help commands

Command	Purpose	Notes
<code>ollama -version</code>	Print the installed Ollama version.	Record this in every reproducibility log.
<code>ollama -help</code>	Show top-level help for the installed CLI.	Use this when documentation and local behavior differ.
<code>ollama help run</code>	Show local help for a sub-command.	Replace <code>run</code> by another command name.

A.3 Core model-management commands

Command	Meaning	Use in these notes
<code>ollama pull modelname</code>	Download or update a model.	Use before a planned experiment.
<code>ollama run modelname</code>	Start an interactive session with a model.	Main command for mathematical prompts.
<code>ollama run modelname "prompt"</code>	Run a one-line prompt non-interactively.	Useful for short tests and scripts.
<code>ollama list</code>	List locally available models on installations that accept this spelling.	Verify with <code>ollama help</code> ; current public documentation displays <code>ollama ls</code> for this task.
<code>ollama ls</code>	List locally available models.	This spelling is displayed on the current public CLI page.
<code>ollama show modelname</code>	Show information for a model.	Record model details before an experiment.
<code>ollama show -modelfile modelname</code>	Show the Modelfile associated with a model, when available.	Useful for prompt-template and parameter inspection.
<code>ollama rm modelname</code>	Remove a local model.	Use when disk space is limited.
<code>ollama ps</code>	List currently running or loaded models.	Useful for checking context and processor allocation.

The public CLI page currently shows commands such as `ollama run`, `ollama pull`, `ollama rm`, `ollama ls`, `ollama ps`, and `ollama serve` [1]. The official command source defines `list` with alias `ls`, and defines `show MODEL` for model information [14].

A.4 Running a first mathematical prompt

A minimal first test is a check that the local model runs.

```
ollama run modelname
```

Then enter:

Prove that every convergent sequence of real numbers is bounded.
Give a rigorous proof and explicitly state where convergence is used.

After the model answers, enter:

Audit your previous proof. Identify every hidden assumption.

The successful execution of the command shows that the local software is working.

A.5 PowerShell examples on Windows

The following examples are intentionally generic. They avoid fixed model tags.

```
ollama --version
ollama pull modelname
ollama run modelname
ollama list
ollama show modelname
ollama rm modelname
```

To save output for a reproducibility log, use ordinary PowerShell redirection. For example:

```
ollama --version > ollama_version.txt
ollama list > ollama_models.txt
ollama show modelname > modelname_show.txt
```

For an interactive mathematical session, it is often better to copy the prompt and response into a plain text log than to rely only on terminal history.

A.6 Hugging Face GGUF pattern

Hugging Face documents direct use of GGUF repositories with Ollama through the pattern [16]:

```
ollama run hf.co/username/model-repository
```

For an explicit quantization selection, Hugging Face documents the pattern:

```
ollama run hf.co/username/model-repository:quantization-tag
```

For example, the placeholder form used in these notes is:

```
ollama run hf.co/username/model-repository:Q4_K_M
```

A.7 Context length and loaded models

Ollama documents context length as the maximum number of tokens available to the model in memory, and notes that increasing it can increase memory requirements [3]. The same documentation describes checking allocated context length and processor allocation with:

```
ollama ps
```

When testing mathematical prompts, record the context length if it is known. A proof audit that fails because the earlier proof was outside the effective context is different from a proof audit that fails because the model accepted an invalid step.

A.8 Minimal reproducibility log

For an experiment, we recommend to record at least:

- operating system,
- date and time,
- `ollama -version`,
- exact model name or Hub identifier,
- quantization, if known,
- context length, if known,
- prompt,
- model output summary,
- human correction and final mathematical verdict.

Appendix B

Example Modelfiles

A Modelfile is an Ollama configuration file used to create a customized local model from a base model or local model file. Ollama documents `FROM`, `PARAMETER`, `SYSTEM`, `TEMPLATE`, `ADAPTER`, `LICENSE`, `MESSAGE`, and, in some versions, `REQUIRES` as Modelfile instructions. Verify the accepted instruction set against the installed Ollama version, [2].

B.1 General cautions

A Modelfile can set a system message and runtime parameters, and may make outputs more disciplined. Before using a Modelfile, verify:

- that the `FROM` model exists locally or points to a valid local file,
- that the model license permits the intended educational use,
- that the parameters are supported by the installed Ollama version,
- that the resulting model can be inspected with `ollama show`,
- that the system message does not ask the model to hide uncertainty.

B.2 Minimal mathematical assistant Modelfile

This example uses documented instructions and documented parameters: `FROM`, `PARAMETER temperature`, `PARAMETER num_ctx`, `PARAMETER seed`, and `SYSTEM` [2]. Replace `modelname` by a locally available base model.

```
FROM modelname
```

```
PARAMETER temperature 0  
PARAMETER num_ctx 4096  
PARAMETER seed 42
```

```

SYSTEM """
You are assisting mathematics students.
Treat every proof attempt as provisional until checked.
State assumptions explicitly.
Do not invent theorem names, citations, model capabilities, or software commands.
When a conclusion is uncertain, say so.
For nontrivial claims, separate strategy, proof, and audit.
"""

```

To create a local model from this file, save it as `Modelfile` and run:

```

ollama create math-assistant -f Modelfile
ollama run math-assistant

```

The name `math-assistant` is local and arbitrary.

B.3 Skeptical proof-auditor Modelfile

This variant is intended for proof auditing. It should not be the same model that produced the proof being audited.

```

FROM modelname

PARAMETER temperature 0
PARAMETER num_ctx 4096
PARAMETER seed 17

SYSTEM """
You are a skeptical mathematical proof auditor.
Your task is not to improve style first.
Your task is to find the first invalid step, missing assumption,
wrong definition, false theorem invocation, or unjustified implication.
Return one of these verdicts:
    correct as written,
    correct after minor repair,
    incomplete,
    incorrect.
Do not call a proof correct unless every step has been checked.
"""

```

Use this model role after a proof attempt.

B.4 Counterexample-search Modelfile

This variant asks the model to prefer small, checkable examples.

```
FROM modelname
```

```
PARAMETER temperature 0.2
```

```
PARAMETER num_ctx 4096
```

```
PARAMETER seed 23
```

```
SYSTEM """
```

```
You are helping search for counterexamples in mathematics.
```

```
For every proposed counterexample, check every assumption explicitly.
```

```
Prefer simple finite sets, one-dimensional functions, matrices of small size,  
or elementary sequences when appropriate.
```

```
Do not present a counterexample as valid until all assumptions and the negated  
conclusion have been verified.
```

```
"""
```

A nonzero temperature may sometimes encourage exploration.

B.5 Long-context reading Modelfile

This example raises the context parameter. Ollama documents `num_ctx` as setting the context window size in a Modelfile, and also documents that larger context length can require more memory [2, 3].

```
FROM modelname
```

```
PARAMETER temperature 0
```

```
PARAMETER num_ctx 8192
```

```
PARAMETER seed 42
```

```
SYSTEM """
```

```
You are assisting with careful reading of mathematical notes.
```

```
When asked to summarize, distinguish definitions, theorems, proofs, examples,  
warnings, and open questions.
```

```
Do not infer missing assumptions from context unless you mark them explicitly.
```

```
"""
```

Do not increase `num_ctx` merely just because it is possible. Longer context may be slower and may require more memory.

B.6 Modelfile from a local GGUF file

Ollama documents building from a GGUF file using `FROM ./ollama-model.gguf`, with the file path absolute or relative to the Modelfile location [2]. A placeholder example is:

```
FROM ./model-file.gguf
```

```
PARAMETER temperature 0
```

```
PARAMETER num_ctx 4096
```

```
PARAMETER seed 42
```

```
SYSTEM """
```

```
You are assisting with elementary mathematical reasoning.
```

```
Do not claim that a numerical experiment proves a theorem.
```

```
When asked for a proof, provide a proof or clearly state that you do not have one.
```

```
"""
```

B.7 Modelfile audit checklist

Before distributing a Modelfile to students, check:

1. Is the base model identified precisely?
2. Is the model license recorded from a primary source?
3. Are all parameters documented for the installed Ollama version?
4. Does the system message encourage uncertainty reporting rather than confident invention?
5. Does the system message distinguish proof exploration from proof verification?
6. Is the context length appropriate for the hardware?
7. Has the resulting local model been tested on a trivial mathematical prompt?
8. Has the Modelfile itself been stored with the course materials?

Appendix C

Prompt Templates

This appendix repeats selected templates from Chapter 6 in a copy-ready form. Before using a template, check that the assumptions, definitions, forbidden shortcuts, and verification target have been adapted to the actual mathematical problem. The templates are designed for local models and for experimenting. They do not make the model more reliable, and their purpose is to reduce ambiguity, expose hidden assumptions, and make verification easier.

C.1 Problem card template

Problem card:

1. Statement to investigate:
2. Domain:
3. Exact assumptions:
4. Desired conclusion:
5. Known related facts:
6. Definitions that must be used verbatim:
7. Forbidden shortcuts:
8. Try both: proof and counterexample.

Instructions:

- Do not assume additional hypotheses.
- If a definition is ambiguous, ask for clarification or state the ambiguity.
- Separate heuristic ideas from verified proof steps.
- If you give a counterexample, verify every assumption explicitly.

C.2 Proof or counterexample template

Investigate the following mathematical claim.

Claim:

[insert claim]

Assumptions:

[insert assumptions]

Definitions to use verbatim:

[insert definitions]

Task:

Either give a rigorous proof or give a concrete counterexample.

Before giving the final answer, list possible failure modes of the claim.

After the answer, verify every assumption and every implication.

Do not use unstated compactness, convexity, smoothness, boundedness, or finite-dimensional assumptions.

C.3 Proof-generation template

Prove the following theorem rigorously.

Theorem:

[insert theorem]

Requirements:

- State all quantifiers explicitly.
- Identify where each assumption is used.
- Do not invoke named theorems unless you state the exact version needed.
- If the proof depends on a standard theorem, state that theorem precisely.
- End with a short proof audit of your own argument.

C.4 Strategy-before-proof template

Do not write the proof yet.

For the following problem, propose three possible proof strategies.

For each strategy, state:

1. the main idea,
2. the assumptions it needs,
3. the likely weak point,
4. whether a counterexample search seems more appropriate.

Problem:

[insert problem]

C.5 Counterexample-search template

Search for a counterexample to the following claim.

Claim:

[insert claim]

Rules:

- Start with the simplest possible domain or object.
- For each candidate counterexample, verify every assumption explicitly.
- Then verify that the claimed conclusion fails.
- If a candidate fails, explain exactly which assumption is not satisfied.
- Do not stop at numerical evidence.

C.6 Proof-audit template

Audit the following proof.

Proof to audit:

[insert proof]

Task:

1. Restate the theorem being proved.
 2. List all assumptions used in the proof.
 3. Check each implication line by line.
 4. Identify the first invalid or unjustified step, if any.
 5. Check for hidden compactness, convexity, smoothness, boundedness, or measurability assumptions.
 6. Check the direction of every inequality.
 7. Give a final verdict:
correct as written / correct after minor repair / incomplete / incorrect.
- Do not rewrite the proof before auditing it.

C.7 Proof-repair template

The following proof has been audited and found problematic.

Original statement:

[insert statement]

Original proof:

[insert proof]

Audit findings:
[insert audit findings]

Task:

- Do not ignore the audit findings.
- Decide whether the original statement is true or false.
- If true, repair the proof with all missing assumptions and steps.
- If false, give a counterexample and verify it.
- If the statement needs stronger assumptions, state them clearly.

C.8 LaTeX rewriting template

Rewrite the following corrected mathematical argument in clean LaTeX.

Rules:

- Do not change the mathematical content.
- Do not add new assumptions.
- Do not add citations or theorem names unless they are already present.
- Use simple LaTeX.
- Preserve all definitions and quantifiers.
- If you see a mathematical gap, mark it as [GAP] instead of silently fixing it.

Corrected argument:

[insert corrected argument]

C.9 Computational-experiment template

Design a small computational experiment for the following mathematical question.

Question:

[insert question]

Requirements:

- Use Python code that can be run locally.
- State what the experiment can suggest.
- State explicitly what the experiment cannot prove.
- Include checks for edge cases.
- Do not present plots or floating-point output as a proof.

C.10 Uncertainty-analysis template

Analyze uncertainty in your previous answer.

Return:

1. statements you are confident about and why,
2. statements that depend on unstated assumptions,
3. places where a theorem may have been invoked incorrectly,
4. calculations that should be recomputed by hand,
5. possible counterexamples to test,
6. what a human mathematician should verify next.

C.11 Bad prompt and improved prompt examples

Bad prompt	Improved prompt
Prove this is true.	State the claim, domain, exact assumptions, definitions, and ask for proof or counterexample.
Is this proof OK?	Ask for a line-by-line audit, the first invalid step, hidden assumptions, and a final verdict.
Find a counterexample.	Specify the claim, assumptions to preserve, simplest domains to try, and require verification of every assumption.
Make it rigorous.	Provide the corrected argument and ask the model to mark gaps rather than silently add hypotheses.
Write LaTeX.	Ask for LaTeX rewriting only after the mathematics has been corrected, and forbid changes of content.

Appendix D

Glossary

Term	Meaning in these notes
LLM	A large language model: a statistical model trained to predict and generate text-like sequences.
Local reasoning model	A model executed on the user's own machine or local infrastructure rather than through a paid remote API. Local execution affects privacy, cost, and reproducibility.
Parameter	A numerical weight or related learned quantity in a model. Parameter counts are useful rough size indicators.
Quantization	A reduced-precision representation of model weights or related tensors. Quantization can reduce memory use and speed up local inference, but may affect output quality. For GGUF-related use, consult the relevant format and tool documentation [15, 19, 20].
GGUF	A model file format associated with GGML and llama.cpp-style local inference. Hugging Face documents Hub support for GGUF files and metadata viewing [15].
Context window	The maximum number of tokens that the model has access to in memory for a run. Ollama documents context length as a token limit and notes that larger context length can increase memory requirements [3].
Token	A unit in the model's text representation. A token may be a word, part of a word, punctuation, or another text fragment, depending on the tokenizer.

KV cache	The cached key-value tensors used internally by transformer inference to avoid recomputing earlier context. Larger context usually increases cache memory requirements. For general cache behavior, see transformer documentation [21].
RAM	System memory available to the operating system and CPU. If a model does not fit in GPU memory, some computation may use CPU and RAM instead.
VRAM	GPU memory. For local inference, VRAM often constrains which model size, quantization, and context length can run efficiently.
Offloading	Splitting model computation or storage between GPU and CPU. Practical offloading may allow a larger model to run, but it may reduce speed. Ollama recommends checking processor allocation with <code>ollama ps</code> in its context-length documentation [3].
Prompt	The instruction or mathematical problem given to the model. A good prompt should state all relevant details like domain, assumptions, definitions, requested output type, etc.
System prompt	A higher-level instruction that frames the assistant's behavior, sometimes stored in a Modelfile using <code>SYSTEM</code> [2].
Temperature	A generation parameter affecting randomness in token selection. Ollama documents <code>temperature</code> as a Modelfile parameter [2]. Low temperature may make answers more stable, but not necessarily correct.
Seed	A generation parameter used to make outputs more reproducible under fixed conditions. Ollama documents <code>seed</code> as a Modelfile parameter [2].
Hallucination	A plausible but unsupported output. In mathematics, this may be a plausible definition, a false theorem invocation, a fabricated citation, a wrong inequality direction, or an invalid proof step.
Proof audit	A systematic check of a proposed proof: statement, assumptions, definitions, local implications, calculations, hidden hypotheses, and final verdict.

Formal verification	Checking a formalized statement and proof in a proof assistant such as Lean. Formal verification checks the formal statement actually encoded, not the informal intention, unless the encoding is faithful [52, 53, 66].
Model card	A documentation page describing a model, usually including intended use, limitations, license information, and metadata. Hugging Face documents model cards as part of its Hub documentation [18].
Benchmark	A standardized evaluation task or score. Benchmarks can be useful for orientation, but benchmark performance is not a proof of reliability on a new mathematical problem.
Reasoning model	A model family or checkpoint presented by its authors as specialized or trained for extended reasoning behavior.
Proof checker	A system that checks formal proof objects against formal rules.

Appendix E

Model Comparison Table

This appendix provides a worksheet for comparing local models in a mathematics course. It is not a leaderboard. A model that performs well on one prompt may fail on a closely related prompt, especially when definitions or quantifiers change.

E.1 Principles for comparison

A useful classroom comparison should be:

- task-specific, not prestige-based;
- reproducible enough that another student can rerun it;
- conservative about conclusions;
- explicit about quantization, context, and prompt wording;
- evaluated by human mathematical checking.

Do not compare models by vague impressions such as “smarter”, “better”, or “more mathematical”. Compare them on stated tasks: proof search, counterexample search, proof audit, coding, LaTeX exposition, or formalization assistance.

E.2 Model comparison worksheet

Copy one row per model and per task. If the same model is tested under two quantizations, use two rows.

Field	Entry
Model name	

Exact tag or identifier
Model family
Source of model card
License source yes / no / unclear checked?
Local format Ollama library / GGUF from Hugging Face / local Modelfile / other
Quantization
Size on disk
Context length
Ollama version
Date of experiment
Prompt
Task type explanation / proof search / counterexample search / proof audit / coding / Lean assistance / LaTeX exposition / editing
Temperature, if known
Seed, if known
Output summary
Correct conclusion? yes / no / partially / not applicable
Main mathematical error
Useful insight
Human correction
Final assessment useful / useful with caution / not useful for this task / incorrect or misleading

E.3 Compact table for reports

For a final report, students may compress the results into the following table.

Model	Quant.	Task	Verdict	Main error	Useful insight

E.4 Fields requiring primary sources

The following fields should not be filled from memory:

- license;
- exact model identifier;
- exact quantization file name;
- claimed context length;
- model size or architecture details;
- benchmark numbers;
- special prompting or chat-template requirements.

Use model cards and official documentation where possible. Hugging Face documents the role of model cards and GGUF metadata, but the completeness of individual model cards varies [18, 15]. If a field is unknown, write **unknown** rather than guessing.

Appendix F

Verification Checklist

This checklist is the operational counterpart of the methodological warning repeated throughout the notes. A local model may be useful for generating ideas, but proof responsibility remains with the human mathematician or with a formal proof system checking a faithful formalization.

F.1 Compact checklist

Before accepting a model-generated mathematical answer, check the following.

1. **Statement.** Is the theorem or claim stated exactly as intended?
2. **Domain.** Are the domain, codomain, and ambient space correct?
3. **Quantifiers.** Are all universal and existential quantifiers in the right order?
4. **Definitions.** Did the model use the definitions supplied in the prompt?
5. **Assumptions.** Did the proof introduce extra assumptions such as compactness, convexity, boundedness, smoothness, or finite dimension?
6. **Local implications.** Does each line follow from previous lines?
7. **Inequality direction.** Are all inequalities written in the correct direction?
8. **Boundary cases.** Are endpoints, zero cases, empty sets, and equality cases handled?
9. **Calculations.** Have derivatives, algebraic transformations, limits, and estimates been recomputed independently?

10. **Theorem invocation.** Is every named theorem stated in a version that actually applies?
11. **Counterexamples.** Have simple counterexamples been tested against the assumptions?
12. **Numerics.** Is numerical evidence treated only as evidence, not proof?
13. **References.** Are citations real, relevant, and checked from primary sources where possible?
14. **Formalization.** If Lean or another proof assistant is used, does the formal theorem faithfully express the intended informal statement?
15. **Final verdict.** Is the result correct as written, correct after repair, incomplete, false, or still unresolved?

F.2 Checklist for proof generation

Object to check	Question
Definitions	Are all definitions exactly those required by the problem?
Assumptions	Where is each assumption used? Is any unused assumption suspicious?
Conclusion	Does the proof reach the stated conclusion, or a stronger/weaker/different one?
Named results	Does each named result apply under the current hypotheses?
Calculations	Can every derivative, estimate, or algebraic identity be verified by hand?
Hidden cases	Are exceptional cases excluded without justification?

F.3 Checklist for counterexamples

For a proposed counterexample, verify in this order:

1. The object is well-defined.
2. The object belongs to the required domain.
3. Every assumption of the claim is satisfied.
4. The conclusion of the claim fails.

5. No part of the verification depends only on numerical approximation unless the claim is itself numerical.

Most false counterexamples fail at step 3.

F.4 Checklist for numerical experiments

- What mathematical question is the experiment testing?
- What finite approximation is being computed?
- What parameter ranges were tested?
- What was not tested?
- Could floating-point error affect the conclusion?
- Does the plot suggest a conjecture, a counterexample candidate, or merely a debugging clue?
- What analytic proof or disproof would still be required?

F.5 Checklist for Lean or formal verification

Formal verification is valuable only after the informal-to-formal translation has been checked.

1. Is the formal statement the intended theorem?
2. Are the types and hypotheses faithful to the informal statement?
3. Are implicit assumptions introduced by typeclass instances or imported lemmas understood?
4. Was the proof actually checked by Lean, or only drafted by a model?
5. If a model supplied Lean code, did a human run it locally?
6. Does the formal proof prove a trivialized or weakened statement?

Appendix G

Suggested Reading and Documentation

This appendix lists documentation and reading categories.

G.1 Ollama documentation

Use official Ollama documentation for command-sensitive material:

- CLI reference for `run`, `pull`, `rm`, `ls`, `ps`, and related command-line behavior [1].
- Modelfile reference for `FROM`, `PARAMETER`, `SYSTEM`, and related instructions [2].
- Context-length documentation for memory and context-window cautions [3].
- Importing-model documentation for local GGUF import through a Modelfile [5].
- Hardware-support and troubleshooting documentation for installation issues [6, 12].

G.2 Hugging Face GGUF documentation

Use Hugging Face documentation for Hub-hosted GGUF workflows:

- GGUF documentation for the Hub’s treatment of GGUF files and metadata [15].
- Ollama-on-Hugging-Face documentation for the `hf.co/...` running pattern and quantization-tag syntax [16].

- Model-card documentation for understanding how model authors describe intended use, limitations, licenses, and metadata [18].

G.3 Model cards used in these notes

For model-family discussions, use exact model cards rather than memory or social-media summaries. The current source registry contains primary entries for Qwen/QwQ, DeepSeek-R1, Phi-4 reasoning, Nemotron and Open-Reasoning Nemotron, gpt-oss, OLMo, Gemma, Mistral, GLM, and selected code-oriented families. Examples include [23, 27, 30, 31, 34, 37, 39, 40, 42].

G.4 Python, SageMath, LaTeX, and Lean

For computational and writing support, use primary documentation:

- Python documentation for virtual environments and package management [49].
- SageMath documentation for symbolic and computational mathematics [51].
- LaTeX Project documentation for LaTeX itself [50].
- Lean documentation and *Theorem Proving in Lean 4* for formalization basics [52, 53].
- Mathlib documentation and repository materials for the mathematical library ecosystem [54, 55].

G.5 LLM limitations and hallucinations

Use limitation-focused sources when discussing hallucination, factuality, and mathematical reliability. Useful starting points include the survey by Wang et al. on factuality, the evaluation study by Frieder et al. on mathematical capabilities, and the survey by Ahn et al. on mathematical reasoning with LLMs [65, 64, 63].

G.6 Formal verification

For formal verification, use proof-assistant documentation and system papers rather than model outputs. Lean can check a formal proof, but it checks the formalized statement. For Lean 4, see the Lean project documentation, *Theorem Proving in Lean 4*, and the Lean 4 system paper [52, 53, 66].

G.7 How to update this appendix

At the beginning of a new course edition:

1. Recheck all software documentation URLs.
2. Recheck model cards and licenses for models actually used.
3. Remove model cards that are no longer used.
4. Do not add benchmark claims unless they are sourced and carefully contextualized.
5. Update `bibliography/source_registry.tex` with access dates and reliability notes.

Bibliography

- [1] Ollama. *CLI Reference*. Official software documentation. Ollama. <https://docs.ollama.com/cli>. Accessed 2026-06-22.
- [2] Ollama. *Modelfile Reference*. Official software documentation. Ollama. <https://docs.ollama.com/modelfile>. Accessed 2026-06-22.
- [3] Ollama. *Context Length*. Official software documentation. Ollama. <https://docs.ollama.com/context-length>. Accessed 2026-06-22.
- [4] Ollama. *Ollama API Introduction*. Official API documentation. Ollama. <https://docs.ollama.com/api>. Accessed 2026-06-22.
- [5] Ollama. *Importing a Model*. Official software documentation. Ollama. <https://docs.ollama.com/import>. Accessed 2026-06-22.
- [6] Ollama. *Hardware Support*. Official software documentation. Ollama. <https://docs.ollama.com/gpu>. Accessed 2026-06-22.
- [7] Ollama. *Windows*. Official software documentation. Ollama. <https://docs.ollama.com/windows>. Accessed 2026-06-22.
- [8] Ollama. *Download Ollama on Windows*. Official download page. Ollama. <https://ollama.com/download/windows>. Accessed 2026-06-22.
- [9] Ollama. *macOS*. Official software documentation. Ollama. <https://docs.ollama.com/macOS>. Accessed 2026-06-22.
- [10] Ollama. *Linux*. Official software documentation. Ollama. <https://docs.ollama.com/linux>. Accessed 2026-06-22.
- [11] Ollama. *FAQ*. Official software documentation. Ollama. <https://docs.ollama.com/faq>. Accessed 2026-06-22.
- [12] Ollama. *Troubleshooting*. Official software documentation. Ollama. <https://docs.ollama.com/troubleshooting>. Accessed 2026-06-22.
- [13] Ollama. *Show Model Details*. Official API reference. Ollama. <https://docs.ollama.com/api-reference/show-model-details>. Accessed 2026-06-22.

- [14] Ollama. *ollama/cmd/cmd.go*. Official source code. GitHub / Ollama repository. <https://github.com/ollama/ollama/blob/main/cmd/cmd.go>. Accessed 2026-06-22; moving-branch source, recheck against the installed version before teaching.
- [15] Hugging Face. *GGUF*. Official platform documentation. Hugging Face Hub. <https://huggingface.co/docs/hub/gguf>. Accessed 2026-06-22.
- [16] Hugging Face. *Use Ollama with any GGUF Model on Hugging Face Hub*. Official platform documentation. Hugging Face Hub. <https://huggingface.co/docs/hub/ollama>. Accessed 2026-06-22.
- [17] Hugging Face. *Local Apps*. Official platform documentation. Hugging Face Hub. <https://huggingface.co/docs/hub/local-apps>. Accessed 2026-06-22.
- [18] Hugging Face. *Model Cards*. Official platform documentation. Hugging Face Hub. <https://huggingface.co/docs/hub/model-cards>. Accessed 2026-06-22.
- [19] ggml-org. *GGUF File Format Specification*. Official repository documentation. GitHub / ggml repository. <https://github.com/ggml-org/ggml/blob/master/docs/gguf.md>. Accessed 2026-06-22; moving-branch source, recheck if exact format details matter.
- [20] ggml-org / llama.cpp. *llama.cpp Quantize README*. Official repository documentation. GitHub / llama.cpp repository. <https://github.com/ggml-org/llama.cpp/blob/master/tools/quantize/README.md>. Accessed 2026-06-22; moving-branch source, recheck if exact labels or behavior matter.
- [21] Hugging Face. *Cache Strategies*. Official library documentation. Hugging Face Transformers. https://huggingface.co/docs/transformers/en/kv_cache. Accessed 2026-06-22.
- [22] Hugging Face. *Tokenization Algorithms*. Official library documentation. Hugging Face Transformers. https://huggingface.co/docs/transformers/en/tokenizer_summary. Accessed 2026-06-22.
- [23] Qwen. *Qwen/QwQ-32B Model Card*. Official model card. Hugging Face Hub. <https://huggingface.co/Qwen/QwQ-32B>. Accessed 2026-06-22.
- [24] Qwen. *Qwen/Qwen3-8B Model Card*. Official model card. Hugging Face Hub. <https://huggingface.co/Qwen/Qwen3-8B>. Accessed 2026-06-22.
- [25] Qwen. *Qwen/Qwen3.6-35B-A3B Model Card*. Official model card. Hugging Face Hub. <https://huggingface.co/Qwen/Qwen3.6-35B-A3B>. Accessed 2026-06-23; repository existence rechecked during final revision.

- [26] Qwen. *Qwen/Qwen3-Coder-30B-A3B-Instruct Model Card*. Official model card. Hugging Face Hub. <https://huggingface.co/Qwen/Qwen3-Coder-30B-A3B-Instruct>. Accessed 2026-06-22.
- [27] DeepSeek-AI. *deepseek-ai/DeepSeek-R1 Model Card*. Official model card. Hugging Face Hub. <https://huggingface.co/deepseek-ai/DeepSeek-R1>. Accessed 2026-06-22.
- [28] DeepSeek-AI. *deepseek-ai/DeepSeek-R1-0528 Model Card*. Official model card. Hugging Face Hub. <https://huggingface.co/deepseek-ai/DeepSeek-R1-0528>. Accessed 2026-06-22.
- [29] DeepSeek-AI. *deepseek-ai/DeepSeek-Coder-V2-Instruct Model Card*. Official model card. Hugging Face Hub. <https://huggingface.co/deepseek-ai/DeepSeek-Coder-V2-Instruct>. Accessed 2026-06-22.
- [30] Microsoft. *microsoft/Phi-4-reasoning Model Card*. Official model card. Hugging Face Hub. <https://huggingface.co/microsoft/Phi-4-reasoning>. Accessed 2026-06-22.
- [31] NVIDIA. *nvidia/OpenReasoning-Nemotron-32B Model Card*. Official model card. Hugging Face Hub. <https://huggingface.co/nvidia/OpenReasoning-Nemotron-32B>. Accessed 2026-06-22.
- [32] NVIDIA. *nvidia/OpenMath-Nemotron-32B Model Card*. Official model card. Hugging Face Hub. <https://huggingface.co/nvidia/OpenMath-Nemotron-32B>. Accessed 2026-06-22.
- [33] NVIDIA. *nvidia/OpenCodeReasoning-Nemotron-32B Model Card*. Official model card. Hugging Face Hub. <https://huggingface.co/nvidia/OpenCodeReasoning-Nemotron-32B>. Accessed 2026-06-22.
- [34] OpenAI. *gpt-oss-120b and gpt-oss-20b Model Card*. Official model card. OpenAI. <https://openai.com/index/gpt-oss-model-card/>. Accessed 2026-06-22.
- [35] OpenAI. *Open models by OpenAI*. Official product/model page. OpenAI. <https://openai.com/open-models/>. Accessed 2026-06-22.
- [36] OpenAI. *OpenAI open-weight models: gpt-oss*. Official help-center article. OpenAI Help Center. <https://help.openai.com/en/articles/11870455-openai-open-weight-models-gpt-oss>. Accessed 2026-06-22.
- [37] Allen Institute for AI. *OLMo Project Page*. Official project page. Allen Institute for AI. <https://allenai.org/olmo>. Accessed 2026-06-22.

- [38] Allen Institute for AI. *allenai/Olmo-3-1025-7B Model Card*. Official model card. Hugging Face Hub. <https://huggingface.co/allenai/Olmo-3-1025-7B>. Accessed 2026-06-22.
- [39] Google. *google/gemma-3-12b-it Model Card*. Official model card. Hugging Face Hub. <https://huggingface.co/google/gemma-3-12b-it>. Accessed 2026-06-22.
- [40] Mistral AI. *mistralai/Magistral-Small-2509 Model Card*. Official model card. Hugging Face Hub. <https://huggingface.co/mistralai/Magistral-Small-2509>. Accessed 2026-06-22.
- [41] Mistral AI. *mistralai/Devstral-Small-2507 Model Card*. Official model card. Hugging Face Hub. <https://huggingface.co/mistralai/Devstral-Small-2507>. Accessed 2026-06-22.
- [42] zai-org / Z.ai. *zai-org/GLM-4.5 Model Card*. Official model card. Hugging Face Hub. <https://huggingface.co/zai-org/GLM-4.5>. Accessed 2026-06-22.
- [43] zai-org / Z.ai. *zai-org/GLM-4.5-Air Model Card*. Official model card. Hugging Face Hub. <https://huggingface.co/zai-org/GLM-4.5-Air>. Accessed 2026-06-22.
- [44] Open Source Initiative. *The Open Source Definition*. Official definition page. Open Source Initiative. <https://opensource.org/osd>. Accessed 2026-06-22.
- [45] Open Source Initiative. *The Open Source AI Definition – 1.0*. Official definition page. Open Source Initiative. <https://opensource.org/ai/open-source-ai-definition>. Accessed 2026-06-22.
- [46] Apache Software Foundation. *Apache License, Version 2.0*. Official license text. Apache Software Foundation. <https://www.apache.org/licenses/LICENSE-2.0.txt>. Accessed 2026-06-22.
- [47] Software Package Data Exchange. *MIT License*. License reference page. SPDX. <https://spdx.org/licenses/MIT>. Accessed 2026-06-22.
- [48] Creative Commons. *Creative Commons Attribution 4.0 International Legal Code*. Official license legal code. Creative Commons. <https://creativecommons.org/licenses/by/4.0/legalcode.en>. Accessed 2026-06-22.
- [49] Python Software Foundation. *Python Tutorial: Virtual Environments and Packages*. Official language documentation. Python Software Foundation. <https://docs.python.org/3/tutorial/venv.html>. Accessed 2026-06-22.

- [50] The LaTeX Project. *LaTeX Project Documentation*. Official project documentation. The LaTeX Project. <https://www.latex-project.org/>. Accessed 2026-06-22.
- [51] SageMath. *SageMath Documentation*. Official software documentation. SageMath. <https://doc.sagemath.org/>. Accessed 2026-06-22.
- [52] Lean project. *Lean Programming Language*. Official project documentation. Lean FRO. <https://lean-lang.org/>. Accessed 2026-06-22.
- [53] Jeremy Avigad, Leonardo de Moura, Soonho Kong, Sebastian Ullrich, and the Lean community. *Theorem Proving in Lean 4*. Official / standard tutorial book. Lean community. https://leanprover.github.io/theorem_proving_in_lean4/. Accessed 2026-06-22.
- [54] Lean project / Mathlib community. *Mathlib: A Foundation for Formal Mathematics Research and Verification*. Official project documentation. Lean FRO / Mathlib community. <https://lean-lang.org/use-cases/mathlib/>. Accessed 2026-06-22.
- [55] Leanprover-community / Mathlib community. *leanprover-community/mathlib4 GitHub Repository*. Official community repository. GitHub / mathlib4. <https://github.com/leanprover-community/mathlib4>. Accessed 2026-06-22; theorem names and imports are version-dependent.
- [56] Microsoft. *Get-PSDrive (Microsoft.PowerShell.Management)*. Official command documentation. Microsoft Learn. <https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.management/get-psdrive?view=powershell-7.6>. Accessed 2026-06-22.
- [57] Microsoft. *Get-CimInstance (CimCmdlets)*. Official command documentation. Microsoft Learn. <https://learn.microsoft.com/en-us/powershell/module/cimcmdlets/get-ciminstance?view=powershell-7.6>. Accessed 2026-06-22.
- [58] NVIDIA. *Nvidia-smi Manual*. Official hardware/software documentation. NVIDIA. <https://docs.nvidia.com/deploy/nvidia-smi/index.html>. Accessed 2026-06-22.
- [59] Xcode Manual Pages mirror. *df(1) Manual Page*. Operating-system manual-page mirror. Xcode Manual Pages mirror. <https://keith.github.io/xcode-man-pages/df.1.html>. Accessed 2026-06-22.
- [60] Xcode Manual Pages mirror. *sysctl(8) Manual Page*. Operating-system manual-page mirror. Xcode Manual Pages mirror. <https://keith.github.io/xcode-man-pages/sysctl.8.html>. Accessed 2026-06-22.

- [61] GNU Project. *df invocation*. Official software documentation. GNU Coreutils. https://www.gnu.org/software/coreutils/manual/html_node/df-invocation.html. Accessed 2026-06-22.
- [62] Linux man-pages project. *free(1) Linux manual page*. Manual page. man7.org / Linux man-pages. <https://man7.org/linux/man-pages/man1/free.1.html>. Accessed 2026-06-22.
- [63] Janice Ahn, Rishu Verma, Renze Lou, Di Liu, Rui Zhang, and Wenpeng Yin. *Large Language Models for Mathematical Reasoning: Progresses and Challenges*. Peer-reviewed conference paper. Association for Computational Linguistics. <https://aclanthology.org/2024.eacl-srw.17/>. Accessed 2026-06-22.
- [64] Simon Frieder, Luca Pinchetti, Alexis Chevalier, Ryan-Rhys Griffiths, Tommaso Salvatori, Thomas Lukasiewicz, Philipp Christian Petersen, and Julius Berner. *Mathematical Capabilities of ChatGPT*. Conference paper. OpenReview / NeurIPS 2023 Datasets and Benchmarks Track. <https://openreview.net/forum?id=xJ7YWXQ0rg>. Accessed 2026-06-22.
- [65] Yuxia Wang, Minghan Wang, Muhammad Arslan Manzoor, Fei Liu, Georgi Nenkov Georgiev, Rocktim Jyoti Das, and Preslav Nakov. *Factuality of Large Language Models: A Survey*. Peer-reviewed conference paper. Association for Computational Linguistics. <https://aclanthology.org/2024.emnlp-main.1088/>. Accessed 2026-06-22.
- [66] Leonardo de Moura and Sebastian Ullrich. *The Lean 4 Theorem Prover and Programming Language*. Conference paper. Springer. https://link.springer.com/chapter/10.1007/978-3-030-79876-5_37. Accessed 2026-06-22.
- [67] William Fedus, Barret Zoph, and Noam Shazeer. *Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity*. Journal article / preprint. Journal of Machine Learning Research / arXiv. <https://arxiv.org/abs/2101.03961>. Accessed 2026-06-22.
- [68] Albert Q. Jiang et al.. *Mixtral of Experts*. Technical report / preprint. arXiv. <https://arxiv.org/abs/2401.04088>. Accessed 2026-06-22.
- [69] Hamed Karimi, Julie Nutini, and Mark Schmidt. *Linear Convergence of Gradient and Proximal-Gradient Methods Under the Polyak-Lojasiewicz Condition*. Conference paper / preprint. Springer / arXiv. <https://arxiv.org/abs/1608.04636>. Accessed 2026-06-22.
- [70] NumPy developers. *NumPy Documentation*. Official software documentation. NumPy. <https://numpy.org/doc/>. Accessed 2026-06-22.

- [71] Matplotlib Development Team. *Matplotlib Documentation*. Official software documentation. Matplotlib. <https://matplotlib.org/stable/index.html>. Accessed 2026-06-22.
- [72] Hugging Face. *Generation*. Official library documentation. Hugging Face Transformers. https://huggingface.co/docs/transformers/en/main_classes/text_generation. Accessed 2026-06-24.
- [73] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. *The Curious Case of Neural Text Degeneration*. arXiv preprint arXiv:1904.09751, 2020. <https://arxiv.org/abs/1904.09751>. ICLR 2020.
- [74] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. *Distilling the Knowledge in a Neural Network*. arXiv preprint arXiv:1503.02531, 2015. <https://arxiv.org/abs/1503.02531>. Accessed 2026-06-24.
- [75] Hugging Face. *Knowledge Distillation for Image Classification*. Official Transformers documentation. Hugging Face. https://huggingface.co/docs/transformers/en/tasks/knowledge_distillation_for_image_classification. Accessed 2026-06-24.
- [76] DeepSeek-AI. *DeepSeek-R1-Distill-Qwen-32B*. Model card. Hugging Face. <https://huggingface.co/deepseek-ai/DeepSeek-R1-Distill-Qwen-32B>. Accessed 2026-06-24.
- [77] DeepSeek-AI. *DeepSeek-R1*. Official repository and documentation. GitHub. <https://github.com/deepseek-ai/DeepSeek-R1>. Accessed 2026-06-24.